

IDS Sentinel Terminal

A research-style technical assessment of a productized intrusion-analysis CLI and GUI

Prepared from repository evidence and web research on May 15, 2026

This report examines the IDS Sentinel Terminal repository as both a software product and a research artifact. The current codebase bundles two KDD-derived CSV assets with 494,021 training rows and 311,029 test rows, ships a pure-Python self-learning model, exposes a Tk-based GUI, and records multiple historical classical and deep-learning runs. The report also situates the project against NIST guidance, MITRE ATT&CK, and more modern benchmark datasets such as CIC-IDS2017 and UNSW-NB15.

- Purpose: determine what this repository already delivers as an analyst-facing tool and where its scientific and engineering limits remain.
- Method: combine local repo inspection, executable command evidence, live GUI capture, and authoritative external sources.
- Scope: architecture, datasets, scoring logic, training artifacts, operational commands, packaging, risks, and optimization priorities.

Abstract

IDS Sentinel Terminal is an attempt to recast a machine-learning-oriented intrusion detection repository into a user-facing defensive product. Instead of stopping at model training scripts, the project now exposes an installable command-line interface, a native graphical console, cached downloadable reports, local network and file triage commands, and a pure-Python behavioral model that can score CSV traffic rows without the heavy scientific stack. This report studies the repository in the manner of a software-centric research paper: first by grounding the design against established IDS literature and operational guidance, then by analyzing the codebase, data assets, packaged artifacts, and historical training runs, and finally by identifying the engineering and research gaps that still separate the tool from a production-grade security platform. Two observations dominate the evidence. First, the repository's own learned profile finds high separation in features such as count, srv_count, and logged_in, which is consistent with classical KDD-style connection statistics. Second, the installable package surface is materially ahead of the dependency story: the primary CLI can run in a light Python environment, but legacy training and API modules still require undeclared dependencies such as joblib, numpy, FastAPI, Pydantic, and TensorFlow. The report therefore concludes that IDS Sentinel Terminal is already useful as a local analysis shell and demonstrator of CSV-based traffic triage, yet it remains scientifically constrained by its benchmark choices and operationally constrained by partial packaging and cross-platform assumptions.

Research Questions

- How closely does the repository's current design align with established IDS operational guidance and contemporary research expectations?
- What can be verified directly from the code and historical artifacts about the product's capabilities, limitations, and reproducibility?
- Which defects or architectural pressures most urgently limit the project's credibility as a cross-platform downloadable tool?

1. Introduction and Research Framing

The repository under study no longer behaves like a bare academic notebook or a one-off training script collection. It has been evolved into a named product, IDS Sentinel Terminal, with packaging metadata, a GUI, release automation, cached outputs, and user-facing commands for traffic inspection, hunting, file hashing, local network review, and IOC management. That shift matters because productization changes the evaluation standard. Once a codebase claims to be a tool, the question is not only whether an algorithm works on a benchmark, but also whether a user can install it, understand its outputs, and operate it safely across real environments.

A quick structural count highlights that transition. The working tree presently contains 1,272 files overall, including 63 Python files and 29 CSV files. The central orchestrator, ``ids_app/product_terminal.py``, spans more than two thousand lines and consolidates analysis, caching, IOC operations, lightweight shell behavior, local host triage, parser construction, and entryptpoint handling. The GUI, by contrast, is compact and native: a single Tkinter module translates the command surface into a dark-mode analyst console. The code therefore exhibits a recognizable product pattern, but it does so through a monolithic coordination layer rather than through sharply separated services.

This report treats the repository as a combined software engineering and applied security artifact. It asks whether the product's current behaviors are coherent with accepted intrusion detection doctrine, whether the repository's benchmark choices remain defensible, and whether the packaging and release machinery are sufficiently truthful about runtime expectations. The goal is not to dismiss the code for being incomplete. The goal is to establish precisely where it is already solid, where it is only locally convincing, and where it still depends on assumptions that would fail in a stricter operational setting.

Methodologically, the report uses four evidence streams. First, it analyzes the repository itself, including module structure, parser design, packaged assets, historical run summaries, and release workflows. Second, it executes the live product commands that are available in the current Python environment, recording status, attacks, dataset catalog, ports, and GUI output. Third, it validates failure paths, such as the legacy training console, in order to distinguish implemented capabilities from merely intended ones. Fourth, it situates these observations against standards and literature from NIST, MITRE, and representative intrusion-detection research.

A useful way to read the remainder of the document is to keep three layers in mind. The first layer is operational: the user experience of commands, caches, reports, and screens. The second layer is analytic: how the product scores rows, learns a lightweight model, and summarizes traffic. The third layer is scientific: whether the benchmark logic and evaluation assumptions still hold when compared with post-KDD datasets and cross-dataset generalization research. A credible product in this space has to satisfy all three layers at once.

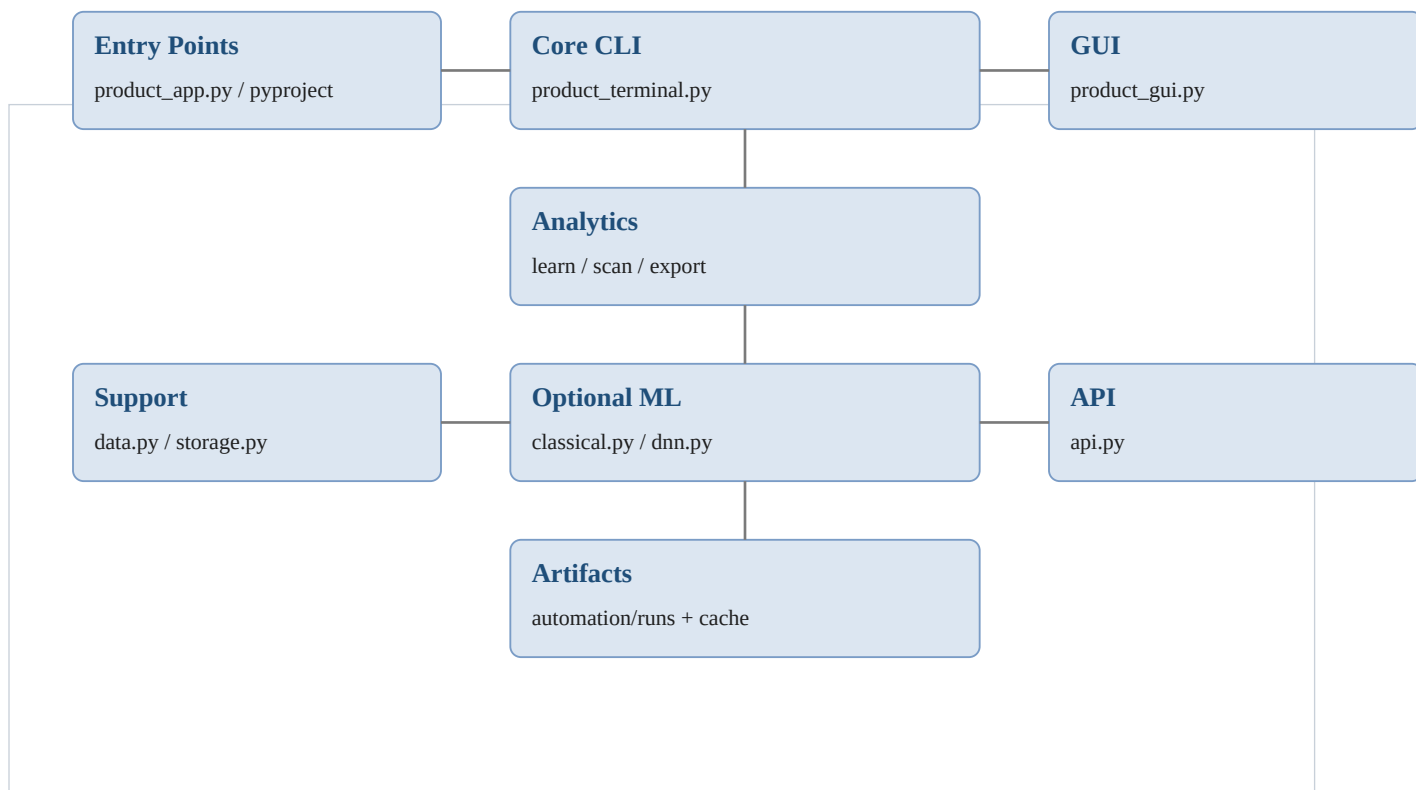


Figure 1. High-level repository architecture reconstructed from the codebase.

2. Standards and Literature Foundation

The conceptual ancestry of this repository is easy to locate. Denning's 1987 model framed intrusion detection around monitored profiles and deviations from normal behavior, establishing the durable idea that security violations can often be surfaced statistically rather than only through hard-coded signatures. A decade later, Lee, Stolfo, and Mok pushed the field toward explicit data-mining workflows for building intrusion detection models from audit data. The repository inherits both instincts. It exposes a lightweight learned profile, but it also retains the spirit of a benchmark-oriented model evaluation workflow through its classical and DNN training subsystems.

NIST's SP 800-94 remains a useful operational anchor because it reminds us that IDS technology is not merely a classifier. The guidance treats understanding, designing, configuring, securing, monitoring, and maintaining IDPS as a lifecycle concern that spans network-based, wireless, behavior-analysis, and host-based perspectives. That framing is important here. IDS Sentinel Terminal is not an inline IPS. It is better understood as a defensive analyst console that borrows from network IDS thinking while also offering host-side adjuncts such as process listing, file hashing, IOC tracking, and port explanation. In NIST terms, it is a complementary defensive technology rather than a complete prevention appliance.

SP 800-61 and SP 800-83 broaden that interpretation. Incident handling guidance emphasizes that effective response depends on preparation, analysis, and repeatable operational procedures. Malware handling guidance adds that response readiness is inseparable from preventive controls and rapid triage. Those documents help explain why the repository's non-model features matter. The CSV analyzer alone would be too narrow. The ability to hash files, hunt text across generated reports, inspect exposed ports, and organize indicators into a local IOC store gives the tool a more incident-centric shape, even if each feature remains deliberately lightweight.

MITRE ATT&CK adds a contemporary vocabulary for talking about what the product can and cannot currently support. The landing page accessed during this study advertised ATT&CK v19 and describes ATT&CK as a globally

accessible knowledge base of adversary tactics and techniques based on real-world observations. That matters because analysts increasingly expect detection and hunt tooling to map findings to tactic and technique language. IDS Sentinel Terminal does not yet produce ATT&CK mappings directly, but several of its affordances align naturally with ATT&CK-style reasoning: brute-force indicators touch credential access, scan behavior touches discovery, file triage supports malware analysis, and IOC workflows support collection and correlation.

Modern survey work also sharpens the evaluation standard. The 2019 survey by Liu and Lang shows how broad the ML and DL IDS literature has become, but it also reiterates recurring goals such as improved detection accuracy, reduced false alarms, and the ability to detect unknown attacks. More recent work, particularly Cantone et al. in 2024, undercuts the comfort of same-dataset evaluation by showing that cross-dataset generalization can collapse toward chance levels even when in-dataset scores look excellent. This single observation should shape how the repository's existing run artifacts are interpreted: good benchmark accuracy is evidence of local fit, not proof of field readiness.

Dataset	Origin	Rows	Features/Columns	Attack coverage note
Bundled KDD train	Local repo asset	494,021	42	Binary normal/attack labels from the KDD99 derivative used
Bundled KDD test	Local repo asset	311,029	42	Held-out companion split with a similarly high attack share.
CIC-IDS2017	UNB CIC official page	5 days of PCAP/CSV flow captures	80+ flow features	FTP/SSH brute force, DoS, DDoS, Heartbleed, web attacks
UNSW-NB15	UNSW official page	2,540,044 total records; 195 sub-records	182,032 features	Fuzzers, Reconnaissance, Backdoors, DoS, Exploits, Generic, Recon

Table 1. Local benchmark assets versus two modern external datasets cited by the product.

3. Dataset Baseline and Threat Realism

The repository's built-in operating baseline is firmly centered on a KDD-derived connection dataset. The local training asset contains 494,021 rows with 42 columns and an attack share of 80.31%. The bundled test asset contains 311,029 rows with the same column count and an attack share of 80.52%. Those proportions matter. A benchmark in which attacks dominate more than eighty percent of rows creates a very different learning environment from an operational network where benign traffic overwhelmingly dominates and analyst attention is consumed by rare but meaningful anomalies.

The feature schema is also historically revealing. KDD-style fields such as `count`, `srv\_count`, `error\_rate`, and host-centric error rates embody an era in which connection-level aggregates were a pragmatic and powerful way to distinguish hostile behavior. The repository's own learned model confirms that this structure still separates the local binary labels strongly. Yet the same strength is also a warning sign: when the most discriminative dimensions are tightly coupled to the quirks of a benchmark, models can become excellent at recognizing the benchmark instead of the adversary.

The external dataset catalog hard-coded in the product is therefore a sign of architectural maturity even before any download occurs. CIC-IDS2017 contributes modern attack categories, richer flow features, five days of scenario-specific traffic, and a cleaner bridge between PCAP and flow CSVs. UNSW-NB15 contributes a different feature design, a large labeled corpus, and explicit attack categories such as Fuzzers, Reconnaissance, Generic, and Shellcode. Their presence in the catalog signals that the repository's maintainers already understand the need to look beyond KDD-style assets.

At the same time, the product has not yet operationalized that catalog into an end-to-end evaluation harness. It can import or download CSV material, but the core reportable training evidence in `automation/runs` is still tied to the

bundled benchmark family. This is where the report's scientific critique becomes concrete rather than abstract. The repository is not wrong to start from KDD-derived data. It is incomplete if it treats that starting point as sufficient evidence of modern generalization.

A second realism issue concerns labeling granularity. The bundled assets are binary normal-versus-attack datasets. The product's malware and behavior outputs therefore infer families such as ``dos_flood``, ``probe_scan``, or ``credential_abuse`` heuristically from numeric patterns rather than from native family labels. This is a reasonable product decision, but it should be described honestly. The tool is not learning named malware families from the bundled CSVs; it is translating benchmark-style traffic signals into analyst-friendly behavior categories.

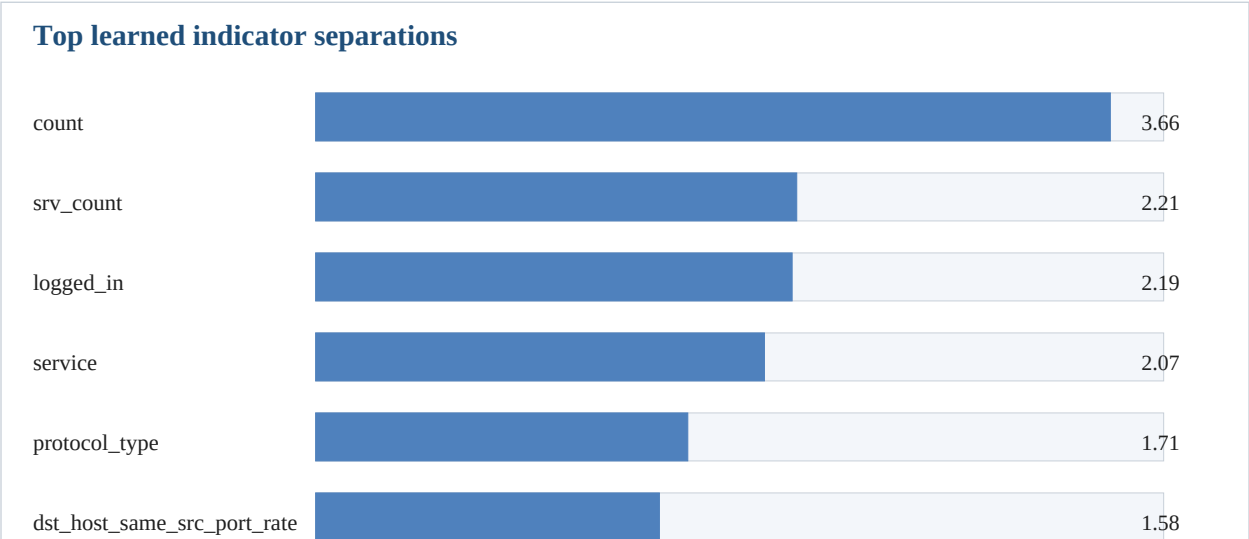


Figure 2. The local Gaussian profile finds the strongest class separation in count-like and host-oriented connection statistics.



4. Repository Architecture and Product Shape

From a software engineering perspective, the clearest structural fact is that ``ids_app/product_terminal.py`` is the system's gravitational center. It owns runtime bootstrap logic, data directory management, CSV parsing, model learning, scoring, reporting, IOC operations, a shell-like subsystem, network triage helpers, file triage helpers, command parsing, and the final entrypoint. Such concentration is not unusual in fast-moving internal tools, but it carries consequences. Change velocity is initially high because there is only one place to wire new commands. Over time, however, testability, onboarding, and change isolation all become harder because unrelated concerns share a single large surface.

The surrounding modules partly offset that concentration. ``product_gui.py`` translates the command layer into a themed Tk application rather than duplicating business logic. ``data.py``, ``storage.py``, and ``metrics.py`` separate lower-level concerns. ``classical.py`` and ``dnn.py`` isolate heavier training logic. ``api.py`` exposes a web-service shape for run metadata. ``scripts/build_python_package.py`` and the release workflow complete the product story by making packaging an explicit part of the codebase rather than an afterthought. In other words, the repository is not architecturally flat. It is architecturally asymmetric: one large coordinator surrounded by smaller, better-scoped helpers.

This asymmetry also explains why the product is pleasant to use in its successful paths. Because the GUI delegates into the CLI core, both surfaces share a consistent command vocabulary and output model. Because exports, runs,

and caches live under stable automation directories, the product can present historical evidence instead of only ephemeral terminal prints. Because the packaged assets are copied into a writable runtime home when installed, the same user-facing commands can behave sensibly both inside the source tree and from a wheel installed elsewhere.

However, the asymmetry creates maintenance pressure. Adding a new command often means editing parser logic, the dispatch routine, output formatting, caching behavior, and sometimes GUI wiring. The product remains coherent because one authorial style dominates the file, but it would become harder for multiple contributors to evolve safely without stronger modular boundaries. The report therefore interprets the monolith not as an immediate defect, but as the main architectural pressure point that future refactoring should address.

Module	Lines	Functions	Classes	Dominant Role
ids_app/__init__.py	3	0	0	Support module
ids_app/api.py	106	10	2	FastAPI interface for runs and jobs
ids_app/classical.py	113	2	0	Classical supervised model training suite
ids_app/config.py	96	0	0	Support module
ids_app/data.py	123	7	0	Dataset loading, sampling, and summary helpers
ids_app/dnn.py	145	4	0	TensorFlow-based deep learning suite
ids_app/jobs.py	87	0	1	Support module
ids_app/legacy.py	90	3	0	Support module
ids_app/metrics.py	33	2	0	Support module
ids_app/product_app.py	18	1	0	Support module
ids_app/product_gui.py	348	1	1	Dark-mode Tk GUI for the product console
ids_app/product_terminal.py	2,350	102	1	Primary CLI, analysis core, caching, IOC, network triage, parser
ids_app/storage.py	50	7	0	Run and job persistence helpers
ids_app/terminal.py	646	26	0	Legacy command console and training workflow
scripts/build_distributions.py	198	9	0	Support module
scripts/build_python_package.py	81	6	0	Support module

Table 2. Module inventory for the executable Python surfaces in the repository.

5. Detection and Analytics Pipeline

The pure-Python learned profile is one of the repository's most pragmatic design choices. According to the local status output, it is stored as a ``streaming_gaussian_profile`` model and was created from 505,221 rows spanning bundled training data and generated analysis exports. In practice this means the product can deliver a useful minimum viable scoring path even when libraries such as numpy, scikit-learn, and TensorFlow are absent. This design decision is not just convenient; it materially improves the odds that a lightweight install will still be capable of analysis rather than failing immediately at import time.

The learning process itself is intentionally transparent. The code builds per-label running statistics across numeric features, derives Gaussian-like parameters, computes class priors, and then uses those parameters at scoring time to estimate how well an incoming row fits the normal and attack profiles. A subsequent classification layer converts these numeric distinctions into categories that are easier for a human operator to act on. This is where the product becomes more than a benchmark wrapper. It does not merely emit a probability. It emits a risk narrative.

That narrative layer is especially visible in ``classify_behavior``. The function uses feature combinations to produce terms such as ``dos_flood``, ``probe_scan``, ``credential_abuse``, ``payload_or_exfiltration``, ``privilege_escalation``, and ``malware_like_activity``. A purist might object that such labels are heuristic and not statistically learned end-to-end. That objection is fair, but incomplete. In analyst tooling, heuristics are not a weakness by default. They become a weakness only when they are hidden, overclaimed, or impossible to inspect. In this repository they are inspectable and therefore auditable.

The export path further strengthens the operational value of the pipeline. The scan routine produces CSV and JSON artifacts, caches summaries, and makes those exports discoverable through subsequent commands. This means that the product already supports a modest but meaningful analytic loop: ingest, score, export, revisit, and hunt across prior outputs. In a small defensive team or classroom environment, that loop is considerably more useful than a raw notebook cell that prints a confusion matrix and exits.

The key caveat is that the pipeline's semantic richness exceeds the semantic richness of the source labels. The product speaks in attack behaviors, but the local benchmark remains binary. Accordingly, any operator or researcher using the tool should treat behavior names as structured hypotheses derived from connection patterns, not as irrefutable ground-truth families.

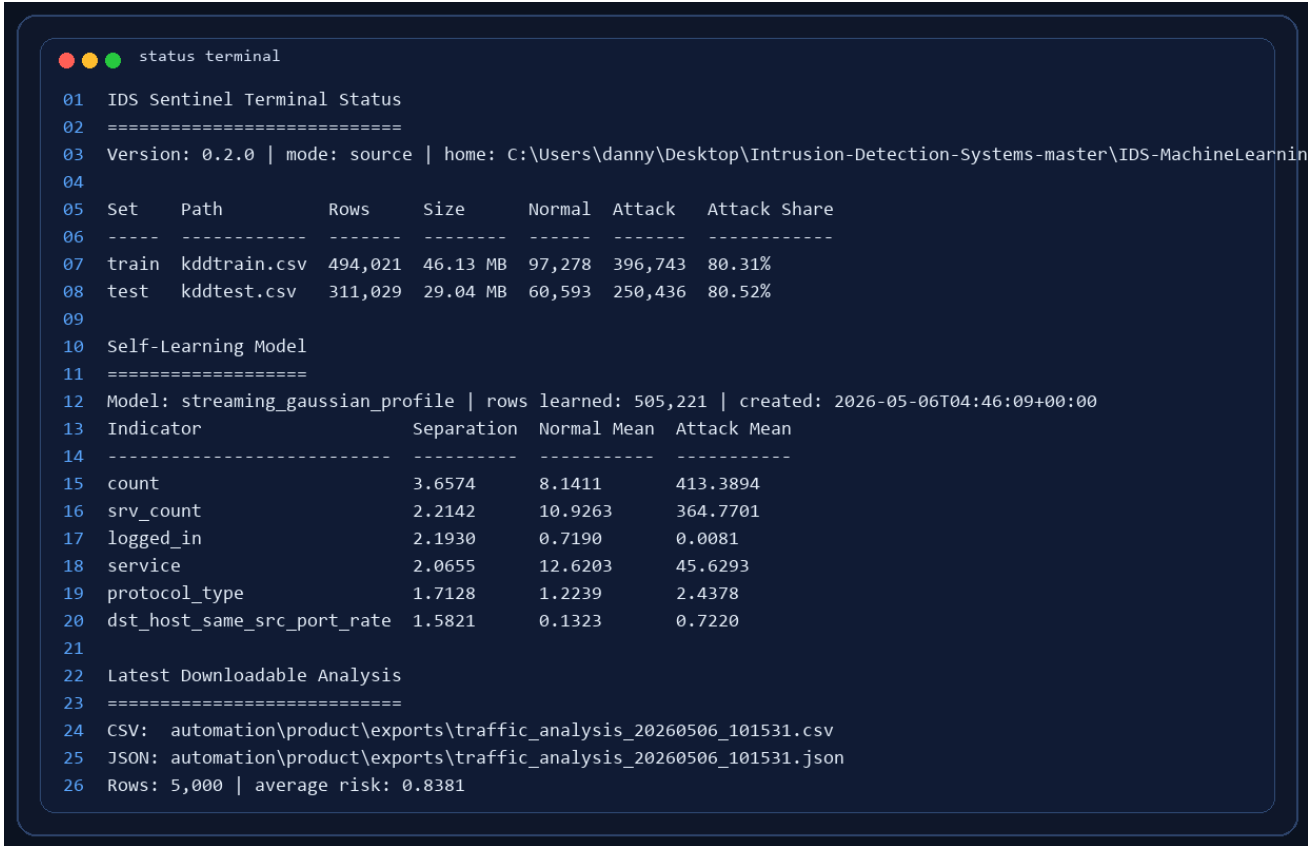


Figure 3. Terminal-styled rendering of the live ``status`` command used as primary local evidence.

6. Supervised Training Subsystems and Historical Results

Although the primary packaged product surface now emphasizes the lightweight analyzer, the repository still carries a more conventional machine-learning subsystem. The classical suite assembles logistic regression, Gaussian Naive

Bayes, decision tree, AdaBoost, random forest, and K-nearest neighbors under a consistent training-and-summary interface. The DNN suite does something similar for TensorFlow-backed architectures. Historical evidence in `automation/runs` confirms that these paths were executed previously and that structured summaries were persisted.

The run artifacts are locally persuasive but scientifically limited. Several classical runs report best accuracies in the low 0.92 to 0.93 range, with random forest or AdaBoost usually emerging as the best classical model. The strongest recorded DNN run in the local artifacts reaches approximately 0.9215 accuracy for a three-layer network over a larger train/test sample pair. Those are competent benchmark numbers. They are also exactly the kind of same-dataset numbers that the broader literature warns can be misleading when generalized beyond a single data family.

The engineering story is even more interesting than the metric story. In the current Python environment used for this report, the legacy terminal training path fails before it can even present help because `joblib` is missing. The API module imports FastAPI and Pydantic directly, and those packages are also absent. The DNN code requires TensorFlow and numpy, which are absent as well. This means that the repo contains valuable historical training evidence, but the installable product surface is only partially honest about the runtime stack required to reproduce that evidence from scratch.

There is a deeper lesson here. By keeping the pure-Python analyzer separate from the heavier training paths, the repository has accidentally discovered a sensible product pattern: the everyday operational console can remain light, while the research-oriented training surfaces can become explicit extras. The problem is not that optional dependencies exist. The problem is that the current package metadata does not clearly express them.

Run ID	Kind	Train rows	Test rows	Best model	Accuracy
dnn-1772763347b9	dnn_train	5,000	2,000	dnn_1_layer	0.9200
classical-d7b00f74d926	classical_train	5,000	2,000	adaboost	0.9210
dnn-dcefbda59cdb	dnn_train	3,000	1,200	dnn_1_layer	0.9008
classical-45fc41fedadd	classical_train	3,000	1,200	random_forest	0.9275
classical-892cd7c2f6a0	classical_train	5,000	2,000	adaboost	0.9210
dnn-29fd1f5f240b	dnn_train	5,000	2,000	dnn_1_layer	0.9160
dnn-9612cd3f9981	dnn_train	50,000	15,000	dnn_3_layer	0.9215
classical-4676bd9735c7	classical_train	40,000	15,000	random_forest	0.9232

Table 3. Historical run summaries recorded under `automation/runs`.

7. Operational Tool Surface

A notable strength of IDS Sentinel Terminal is that it does not confine itself to one analytic gesture. The command surface spans status dashboards, CSV analysis, dataset catalog inspection, IOC management, keyword hunting, hashing, file scanning, DNS resolution, local process listing, port explanations, port probing, and network connection parsing. That breadth is precisely what makes the tool feel closer to a compact analyst workbench than to a single-purpose benchmark harness.

The GUI reinforces that interpretation. The live window captured for this report demonstrates a dark-mode, analyst-oriented design with a persistent sidebar, tabbed control plane, and a scrollable output console. Crucially, it does not invent a second logic layer. It calls into the same command routines as the terminal interface and color-tags the resulting output for readability. This is the correct kind of GUI for a tool at this maturity level: it adds operational

affordance without forking the behavior model.

The command set is also disciplined. Where many internal tools would expose raw shell execution or broad remote scanning, this product prefers bounded operations. The port probe is a TCP connect probe with a capped port list. The shell-like layer favors read-oriented operations such as `ls`, `cat`, `head`, `tail`, `find`, and `grep` style behaviors inside a controlled environment. The report interprets this as an implicit safety posture. Even where the code is informal, the operator affordances are not recklessly broad.

The main operational weakness in this layer is platform realism. The netstat parsing routine clearly assumes Windows `netstat -ano` formatting and loads service names from the Windows services file path. The GUI itself is cross-platform in principle because Tkinter is standard, but the host-triage commands are not yet abstracted by platform. The current product should therefore be described as packaging-portable before it is described as operationally identical across macOS, Linux, and Windows.

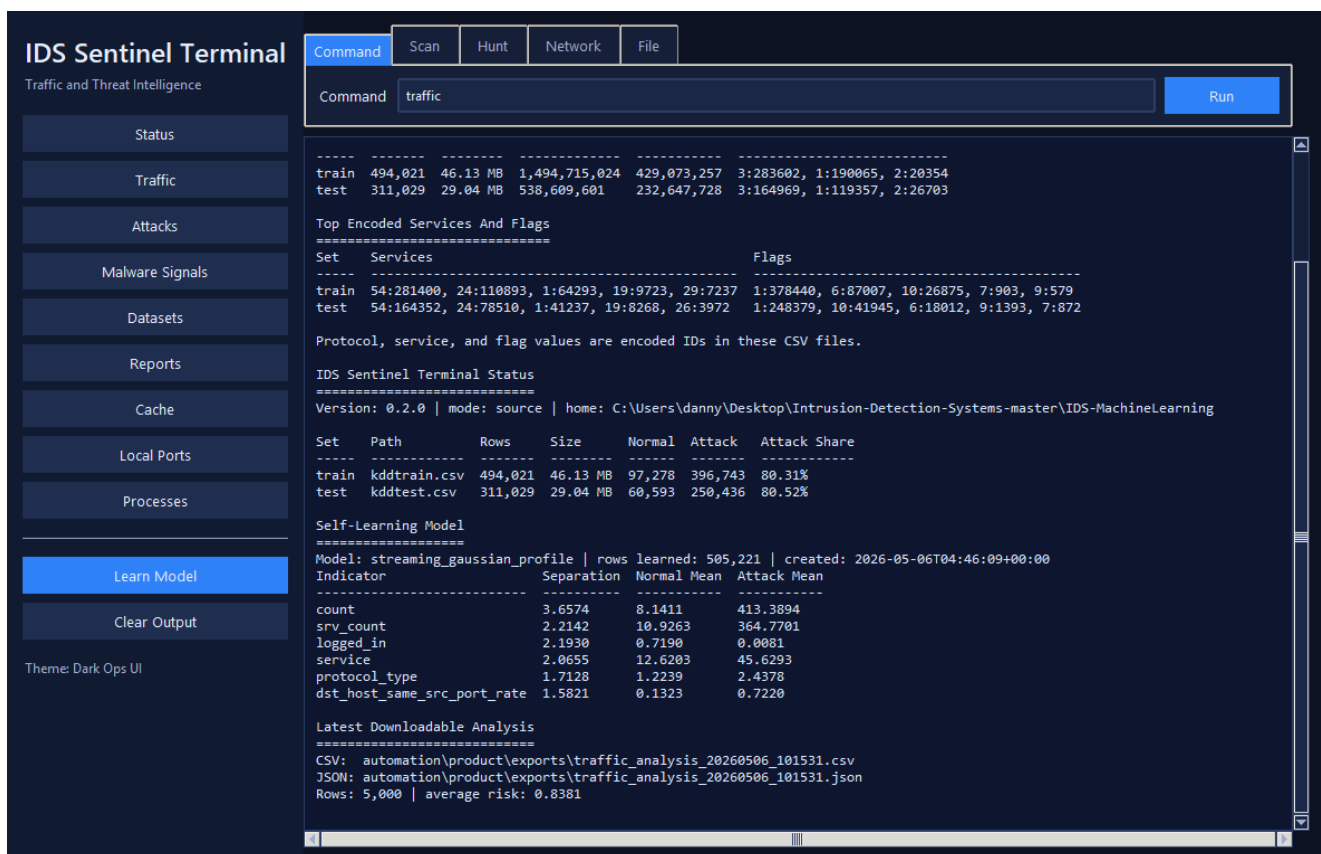


Figure 4. Live window capture of the Tk-based GUI running the traffic workflow.

```

01 Network Connections
02 =====
03 Proto Local Service Remote State PID
04 -----
05 UDP 0.0.0.0:53 domain *: * UDP 7524
06 TCP 0.0.0.0:135 msrpc 0.0.0.0:0 LISTENING 1752
07 TCP [::]:135 msrpc [::]:0 LISTENING 1752
08 UDP 172.25.144.1:137 netbios-ns *: * UDP 4
09 UDP 192.168.1.201:137 netbios-ns *: * UDP 4
10 UDP 172.25.144.1:138 netbios-dgm *: * UDP 4
11 UDP 192.168.1.201:138 netbios-dgm *: * UDP 4
12 TCP 172.25.144.1:139 netbios-ssn 0.0.0.0:0 LISTENING 4
13 TCP 192.168.1.201:139 netbios-ssn 0.0.0.0:0 LISTENING 4
14 TCP 0.0.0.0:445 microsoft-ds 0.0.0.0:0 LISTENING 4
15 TCP [::]:445 microsoft-ds [::]:0 LISTENING 4
16 TCP 127.0.0.1:1042 0.0.0.0:0 LISTENING 17384
17 TCP 127.0.0.1:1043 0.0.0.0:0 LISTENING 17384
18 UDP 0.0.0.0:1900 ssdp *: * UDP 33920
19 UDP 0.0.0.0:1900 ssdp *: * UDP 33920
20 UDP 127.0.0.1:1900 ssdp *: * UDP 9328
21 UDP 172.25.144.1:1900 ssdp *: * UDP 9328
22 UDP 192.168.1.201:1900 ssdp *: * UDP 9328
23 UDP [::1]:1900 ssdp *: * UDP 9328
24 UDP [fe80::22d6:a544:1365:aaf%45]:1900 ssdp *: * UDP 9328

```

Figure 5. Terminal-styled rendering of the local port inspection output.

8. Distribution Engineering and Release Posture

Packaging is where this repository most clearly departs from the academic-project stereotype. The `pyproject.toml` file defines installable console scripts, a GUI script, package data for bundled assets, version metadata, and cross-platform classifiers. The release workflow builds wheel and source distributions, smoke-tests the built wheel, uploads artifacts, attaches them to GitHub releases, and then publishes to PyPI using trusted publishing through GitHub OIDC. In other words, the project already has the skeleton of a legitimate downloadable toolchain.

The supporting documentation consulted for this report aligns well with the implementation direction. The Python `zipapp` documentation explains how self-contained Python applications can be distributed once dependencies are bundled. The tkinter documentation confirms the standard-library nature of the GUI foundation across Windows, macOS, and Unix-like systems. GitHub and PyPI documentation describe the exact OIDC-driven trusted publishing flow already encoded in the workflow file. On paper, therefore, the distribution strategy is technically coherent.

The weakness is again one of truthfulness rather than ambition. The package metadata currently exposes a CLI and GUI but does not declare the broader dependency landscape required by the API module, the legacy terminal, or the training suites. A wheel built from this metadata is a working product shell, not a full scientific environment. That is still valuable, but it should be named explicitly. The package today is best described as a lightweight operational console with optional research subsystems, not as a single install that enables every path in the repository.

This distinction matters even more because the product now aspires to `pipx`-style consumption. Users who install a CLI via `pipx` reasonably expect the declared metadata to be the source of truth. If critical dependencies are intentionally optional, they should be organized as extras and documented as such. If they are actually required for

certain public commands, they belong in the dependency metadata.

Component	Observed state
Python runtime	Python 3.14.3
reportlab	available
Pillow	available
tkinter	available
numpy	missing
joblib	missing
fastapi	missing
pydantic	missing
tensorflow	missing

Table 4. Dependency observations from the exact Python environment used to produce this report.

9. Critical Findings, Bugs, and Optimization Priorities

The report's bug audit is intentionally conservative: only issues that were directly supported by local execution, code inspection, or authoritative documentation are listed as findings.

- Undeclared dependencies are the most important verified product issue. The installable metadata does not list joblib, numpy, fastapi, pydantic, scikit-learn, or tensorflow even though repository modules import them directly.
- The legacy terminal fails immediately in the observed environment with `ModuleNotFoundError: No module named 'joblib'`, proving that at least one public code path is not reproducible from the declared package surface alone.
- The API module imports FastAPI and Pydantic at module import time. Because these packages are absent, the API surface is also not reproducible from the package metadata.
- Network inspection behavior is Windows-centric. The code parses `netstat -ano` output and consults the Windows `services` file path, so the current implementation does not justify a claim of identical operational behavior on Linux and macOS.
- The core orchestration file is large and multi-concern. This is not an immediate runtime defect, but it is the principal maintainability risk and the most likely source of future regressions.
- There is still a naming mismatch between the repo's historical local folder name and the newer `IDS-Sentinel` branding. This is an operational/documentation mismatch more than a code defect, but it increases confusion for users following installation or publication instructions.
- The scientific evaluation story remains bound to KDD-style data for local evidence. Without integrated cross-dataset evaluation, the product cannot yet claim robust contemporary generalization.
- The release workflow's smoke test is modest. It proves wheel installability and basic CLI health but does not verify GUI launch, optional dependencies, or training reproducibility.

Observed failure evidence

Traceback (most recent call last): File "<frozen runpy>", line 198, in `_run_module_as_main` File "<frozen runpy>", line 88

Priority optimization roadmap

- Refactor ``product_terminal.py`` into command modules grouped by concern: data, analytics, IOC, host triage, output, and parser wiring.
- Declare an explicit base dependency set for the operational console and separate extras such as ``[api]``, ``[ml]``, and ``[dnn]`` for optional surfaces.
- Add a platform abstraction for local host inspection instead of hard-coding Windows netstat assumptions.
- Build an integrated evaluation harness for CIC-IDS2017 and UNSW-NB15 so that the external catalog becomes actionable research infrastructure.
- Add contract tests for ``status``, ``scan``, export generation, cache listing, and parser behavior, plus smoke tests for GUI launch in CI.

10. Conclusion

IDS Sentinel Terminal has crossed an important threshold. It is no longer just a collection of intrusion-detection experiments. It is a working defensive console with a coherent brand, a packageable CLI, a native GUI, reproducible export artifacts, and a lightweight scoring model that can operate in a constrained Python environment. Those are not trivial achievements, especially when compared with many security-analytics repositories that never move beyond notebooks and screenshots.

Yet the report's central conclusion is deliberately balanced. The product is operationally ahead of its metadata and scientifically ahead of its benchmark story. Operationally, several public surfaces still depend on libraries that the package does not declare, and some host-oriented commands remain Windows-shaped. Scientifically, the bundled KDD-derived data supports local demonstrations but cannot carry a modern generalization claim on its own. These are solvable issues, but they are foundational issues rather than cosmetic ones.

If the repository continues along its current trajectory, the most promising direction is not simply to add more commands. It is to stabilize the product contract. That means separating base and optional dependencies, modularizing the command core, and turning the external dataset catalog into a formal evaluation pipeline. With those changes, the codebase could legitimately position itself as a portable analyst tool that is anchored in both usability and research discipline. Without them, it remains a strong prototype whose best qualities are clarity of intent, pragmatic packaging work, and a surprisingly usable local analysis workflow.

Appendix A. Feature Glossary and Analytical Interpretation

duration

Connection duration in seconds. Short, repetitive sessions often characterize scanning and flooding, while longer sessions can signal interactive or application-level abuse. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

protocol_type

Encoded transport protocol. Even in encoded form, the field captures major shifts among TCP, UDP, and ICMP-like behavior classes. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

service

Encoded service indicator. It condenses port-and-protocol usage into an application-oriented signal that often separates benign browsing from attack staging. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

flag

Encoded TCP flag state. This is especially informative for incomplete handshakes, resets, and other failure-heavy patterns common in hostile traffic. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

src_bytes

Bytes sent from source to destination. Useful for distinguishing sparse probes from bulk transfer or flooding behavior. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_bytes

Bytes sent from destination to source. Combined with source bytes, this exposes asymmetric conversations and failed requests. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

land

Boolean marker for source and destination equivalence. Historically useful for detecting malformed or deliberately abusive packet patterns. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

wrong_fragment

Count of malformed or misplaced fragments. Elevated values are uncommon in healthy traffic and may point to evasion or corruption. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

urgent

Urgent packet count. Rare in ordinary enterprise activity, so spikes deserve explanation. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

hot

Count of suspicious content indicators derived from the original dataset schema. It acts as a coarse application-layer risk surrogate. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

num_failed_logins

Failed authentication attempts. Directly relevant to brute-force and credential abuse patterns. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

logged_in

Binary success marker for authenticated sessions. The repository's learned model treats this as highly separating for benign versus hostile behavior. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

num_compromised

Number of compromised conditions reported in the source schema. High values are strong compromise indicators. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

root_shell

Binary marker for root shell acquisition. This is a direct privilege escalation cue. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

su_attempted

Whether privilege switching was attempted. Even low frequency events matter because they align with post-exploitation workflows. In this repository, the feature participates in a binary normal-versus-attack setting and

may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

num_root

Count of root-level operations or accesses. A useful escalation strength signal. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

num_file_creations

File creation count within the session. Helpful when reasoning about payload dropper or exfiltration staging behavior. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

num_shells

Shell spawns associated with the session. Repeated shell creation is highly suspicious in most network contexts. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

num_access_files

Sensitive file access count. Supports coarse inference about reconnaissance or data collection. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

num_outbound_cmds

Outbound command count. The field is almost always zero in the bundled data, which itself is a reminder that some legacy features have little discriminative value today. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

is_host_login

Binary flag for privileged local host login. Rare and context-sensitive. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

is_guest_login

Binary flag for guest access. Can signal low-trust account activity or misconfiguration. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

count

Connections to the same host in a short window. This is one of the strongest attack separators in the repository's learned model. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

srv_count

Connections to the same service in a short window. Useful for burst-style probes and service-specific flooding. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

error_rate

SYN error rate over the short window. High values often reveal handshake-heavy denial or probing traffic. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

srv_error_rate

Service-scoped SYN error rate. A tighter lens on service-targeted failures. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

rerror_rate

Reset error rate over the window. Helpful for failed access attempts and aggressive scanning. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

srv_rerror_rate

Service-scoped reset error rate. Useful when a single application endpoint is under stress. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

same_srv_rate

Fraction of recent connections hitting the same service. Stable benign workflows and attack loops both affect this field, but in different combinations with error rates. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

diff_srv_rate

Fraction of recent connections hitting different services. Elevated values can indicate horizontal service enumeration. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level

behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

srv_diff_host_rate

How widely a service is spread across hosts in the recent window. Strong for spotting fan-out behavior. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_count

Historical count of connections to the destination host. In this repository it is one of the most separating host-centric indicators. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_srv_count

Historical count of connections to the destination host and service pair. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_same_srv_rate

Fraction of destination-host connections using the same service. A useful stability versus concentration measure. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_diff_srv_rate

Fraction of destination-host connections using different services. Supports service sweep detection. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_same_src_port_rate

Fraction of connections to the destination host sharing the same source port. The learned model ranks this unusually high, suggesting the dataset encodes repeatability patterns strongly here. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_srv_diff_host_rate

Fraction of a destination service's traffic arriving from different hosts. Useful for distinguishing one-to-one sessions from distributed activity. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_serror_rate

Host-level SYN error rate. One of the most class-separating host-centric fields in the bundled model. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_srv_serror_rate

Host-and-service SYN error rate. Important for service-targeted denial or misconfiguration patterns. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_rerror_rate

Host-level reset error rate. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

dst_host_srv_rerror_rate

Host-and-service reset error rate. This often complements the SYN error fields by describing how targets refuse or terminate connections. In this repository, the feature participates in a binary normal-versus-attack setting and may also feed higher-level behavior inference. Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's narrative scoring pipeline.

Appendix B. Command Reference

shell

Starts the interactive IDS shell that exposes the product's own command grammar rather than the host operating system shell.

Observed parser usage: Starts the interactive IDS shell that exposes the product's own command grammar rather than the host operating system shell.

gui

Opens the Tkinter-based graphical console for status review, scans, hunting, network probing, and file triage.

Observed parser usage: Opens the Tkinter-based graphical console for status review, scans, hunting, network probing, and file triage.

status

Summarizes installation mode, dataset volumes, learned model indicators, cached outputs, and historical runs.

Observed parser usage: Summarizes installation mode, dataset volumes, learned model indicators, cached outputs, and historical runs.

traffic

Reports encoded traffic distribution from the bundled train and test CSV files.

Observed parser usage: Reports encoded traffic distribution from the bundled train and test CSV files.

attacks

Shows attack share statistics together with the most separating learned indicators.

Observed parser usage: Shows attack share statistics together with the most separating learned indicators.

datasets

Lists local sources and the hard-coded external dataset catalog.

Observed parser usage: Lists local sources and the hard-coded external dataset catalog.

malware

Infers malware-like behavior categories from CSV-derived features rather than from malware family labels.

Observed parser usage: Infers malware-like behavior categories from CSV-derived features rather than from malware family labels.

learn

Builds or refreshes the repository's pure-Python Gaussian profile using bundled and generated CSV rows.

Observed parser usage: Builds or refreshes the repository's pure-Python Gaussian profile using bundled and generated CSV rows.

scan

Analyzes a CSV file row-by-row, scores risk, classifies behavior, and exports results by default.

Observed parser usage: Analyzes a CSV file row-by-row, scores risk, classifies behavior, and exports results by default.

export

Convenience alias for full-result CSV and JSON export production.

Observed parser usage: Convenience alias for full-result CSV and JSON export production.

import

Copies an external CSV into the product import area so it can be indexed and analyzed consistently.

Observed parser usage: Copies an external CSV into the product import area so it can be indexed and analyzed consistently.

download

Fetches a public URL into the import area; useful for bringing in external benchmark material.

Observed parser usage: Fetches a public URL into the import area; useful for bringing in external benchmark material.

index

Inspects schema-like properties of a CSV file, including column counts and sample rows.

Observed parser usage: Inspects schema-like properties of a CSV file, including column counts and sample rows.

hunt

Searches datasets, imports, and reports for a textual pattern.

Observed parser usage: Searches datasets, imports, and reports for a textual pattern.

ioc

Manages and hunts indicators of compromise stored in the product IOC file.

Observed parser usage: Manages and hunts indicators of compromise stored in the product IOC file.

netstat

Parses local network connections from the host's `netstat -ano` output.

Observed parser usage: Parses local network connections from the host's `netstat -ano` output.

ports

Shows local listening ports and UDP endpoints.

Observed parser usage: Shows local listening ports and UDP endpoints.

port

Explains a single port using the built-in service and risk knowledge base.

Observed parser usage: Explains a single port using the built-in service and risk knowledge base.

probe

Performs an authorized TCP connect probe against a host and bounded set of ports.

Observed parser usage: Performs an authorized TCP connect probe against a host and bounded set of ports.

dns

Resolves a hostname and attempts reverse lookups where possible.

Observed parser usage: Resolves a hostname and attempts reverse lookups where possible.

ps

Lists local processes for lightweight host triage.

Observed parser usage: Lists local processes for lightweight host triage.

hash

Calculates file hashes for later comparison and response workflows.

Observed parser usage: Calculates file hashes for later comparison and response workflows.

filescan

Hashes a file and applies simple suspicious-pattern triage logic.

Observed parser usage: Hashes a file and applies simple suspicious-pattern triage logic.

reports

Lists generated downloadable analysis exports.

Observed parser usage: Lists generated downloadable analysis exports.

runs

Lists historical classical and DNN training runs from `automation/runs`.

Observed parser usage: Lists historical classical and DNN training runs from `automation/runs`.

cache

Enumerates cached command artifacts stored by the product.

Observed parser usage: Enumerates cached command artifacts stored by the product.

Appendix C. Module-by-Module Symbol Inventory

ids_app/__init__.py

This module contains 3 lines, 0 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
--------	------	-------	-----	----------------------------

ids_app/api.py

This module contains 106 lines, 10 top-level function(s), and 2 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
ClassicalJobRequest	class	20	25	Encapsulates a focused behavior cluster within the module.
DnnJobRequest	class	28	35	Encapsulates a focused behavior cluster within the module.
index	function	44	45	Support routine in the module's local workflow.
health	function	49	50	Support routine in the module's local workflow.
overview	function	54	64	Support routine in the module's local workflow.
list_jobs	function	68	69	Enumerates available artifacts or resources.
get_job	function	73	77	Support routine in the module's local workflow.
list_runs	function	81	82	Enumerates available artifacts or resources.
get_run	function	86	90	Support routine in the module's local workflow.
launch_legacy_evaluation	function	94	95	Support routine in the module's local workflow.
launch_classical	function	99	100	Support routine in the module's local workflow.
launch_dnn	function	104	105	Support routine in the module's local workflow.

ids_app/classical.py

This module contains 113 lines, 2 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
resolve_classical_config	function	32	47	Normalizes config or paths into concrete objects.

Symbol	Kind	Start	End	Approximate responsibility
train_classical_suite	function	50	113	Support routine in the module's local workflow.

ids_app/config.py

This module contains 96 lines, 0 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
--------	------	-------	-----	----------------------------

ids_app/data.py

This module contains 123 lines, 7 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
_sample_frame	function	16	20	Support routine in the module's local workflow.
load_dataset_split	function	23	53	Loads data or configuration from storage or the environment.
load_classical_split	function	56	57	Loads data or configuration from storage or the environment.
load_dnn_split	function	60	61	Loads data or configuration from storage or the environment.
_dataset_file_summary	function	64	92	Support routine in the module's local workflow.
_relative_path	function	95	99	Support routine in the module's local workflow.
dataset_summary	function	103	123	Support routine in the module's local workflow.

ids_app/dnn.py

This module contains 145 lines, 4 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
_tensorflow	function	16	21	Support routine in the module's local workflow.
resolve_dnn_config	function	24	39	Normalizes config or paths into concrete objects.
_build_model	function	42	53	Support routine in the module's local workflow.
train_dnn_suite	function	56	145	Support routine in the module's local workflow.

ids_app/jobs.py

This module contains 87 lines, 0 top-level function(s), and 1 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
JobManager	class	15	83	Encapsulates a focused behavior cluster within the module.

ids_app/legacy.py

This module contains 90 lines, 3 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
_load_labels	function	13	14	Support routine in the module's local workflow.
_load_history	function	17	32	Support routine in the module's local workflow.
evaluate_legacy_predictions	function	36	90	Support routine in the module's local workflow.

ids_app/metrics.py

This module contains 33 lines, 2 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
binary_metrics	function	7	21	Support routine in the module's local workflow.
probability_summary	function	24	32	Support routine in the module's local workflow.

ids_app/product_app.py

This module contains 18 lines, 1 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
main	function	8	14	Support routine in the module's local workflow.

ids_app/product_gui.py

This module contains 348 lines, 1 top-level function(s), and 1 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
IDSProductGUI	class	14	336	Encapsulates a focused behavior cluster within the module.
main	function	339	344	Support routine in the module's local workflow.

ids_app/product_terminal.py

This module contains 2,350 lines, 102 top-level function(s), and 1 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
RunningStat	class	433	457	Encapsulates a focused behavior cluster within the module.
_copy_bundled_asset	function	44	48	Support routine in the module's local workflow.
bootstrap_runtime_home	function	51	59	Support routine in the module's local workflow.
ensure_product_dirs	function	218	225	Support routine in the module's local workflow.
utc_now	function	228	229	Support routine in the module's local workflow.
compact_timestamp	function	232	233	Support routine in the module's local workflow.
relative_path	function	236	240	Support routine in the module's local workflow.
format_number	function	243	250	Support routine in the module's local workflow.
percent	function	253	256	Support routine in the module's local workflow.

Symbol	Kind	Start	End	Approximate responsibility
table	function	259	268	Support routine in the module's local workflow.
section	function	271	273	Support routine in the module's local workflow.
print_json	function	276	277	Support routine in the module's local workflow.
resolve_repo_path	function	280	291	Normalizes config or paths into concrete objects.
read_json	function	294	298	Reads a persisted artifact.
write_json	function	301	304	Writes a persisted artifact.
cache_artifact	function	307	313	Support routine in the module's local workflow.
prune_cache_artifacts	function	316	319	Support routine in the module's local workflow.
path_cache_key	function	322	326	Support routine in the module's local workflow.
file_signature	function	329	331	Support routine in the module's local workflow.
cached_json_path	function	334	335	Support routine in the module's local workflow.
is_cache_current	function	338	341	Support routine in the module's local workflow.
likely_header	function	344	354	Support routine in the module's local workflow.
all_csv_sources	function	357	363	Support routine in the module's local workflow.
resolve_any_product_path	function	366	378	Normalizes config or paths into concrete objects.
resolve_readable_path	function	381	393	Normalizes config or paths into concrete objects.
safe_float	function	396	400	Support routine in the module's local workflow.
iter_kdd_rows	function	403	414	Support routine in the module's local workflow.
iter_generated_rows	function	417	429	Support routine in the module's local workflow.
empty_label_stats	function	460	461	Support routine in the module's local workflow.
update_model_stats	function	464	469	Support routine in the module's local workflow.
generated_export_paths	function	472	475	Support routine in the module's local workflow.
learn_model	function	478	540	Builds or refreshes a learned profile.
rank_indicators	function	543	561	Support routine in the module's local workflow.
load_or_learn_model	function	564	570	Loads data or configuration from storage or the environment.
gaussian_log_probability	function	573	584	Support routine in the module's local workflow.
score_row	function	587	604	Support routine in the module's local workflow.
feature_map	function	607	608	Support routine in the module's local workflow.
risk_level	function	611	618	Support routine in the module's local workflow.
classify_behavior	function	621	660	Converts signals into a risk or behavior label.
summarize_dataset	function	663	703	Support routine in the module's local workflow.
summarize_dataset_cached	function	706	721	Support routine in the module's local workflow.
summarize_all_datasets	function	724	725	Support routine in the module's local workflow.
inspect_csv	function	728	798	Support routine in the module's local workflow.
analyze_csv	function	801	939	Support routine in the module's local workflow.
latest_export_summary	function	942	948	Support routine in the module's local workflow.
list_reports	function	951	968	Enumerates available artifacts or resources.
list_cache_artifacts	function	971	985	Enumerates available artifacts or resources.
show_cache	function	988	1008	Formats and presents operator-facing output.
list_run_summaries	function	1011	1022	Enumerates available artifacts or resources.

Symbol	Kind	Start	End	Approximate responsibility
show_dataset_catalog	function	1025	1041	Formats and presents operator-facing output.
import_csv	function	1044	1058	Support routine in the module's local workflow.
download_url	function	1061	1081	Support routine in the module's local workflow.
show_import	function	1084	1091	Formats and presents operator-facing output.
show_index	function	1094	1111	Formats and presents operator-facing output.
load_services	function	1114	1155	Loads data or configuration from storage or the environment.
parse_ports	function	1158	1181	Interprets text or user input into structured values.
split_host_port	function	1184	1192	Support routine in the module's local workflow.
parse_netstat	function	1195	1237	Interprets text or user input into structured values.
show_netstat	function	1240	1265	Formats and presents operator-facing output.
show_port	function	1268	1287	Formats and presents operator-facing output.
probe_ports	function	1290	1303	Actively tests a local or remote endpoint.
show_probe	function	1306	1317	Formats and presents operator-facing output.
show_dns	function	1320	1344	Formats and presents operator-facing output.
show_processes	function	1347	1367	Formats and presents operator-facing output.
hash_file	function	1370	1385	Computes a file digest for triage.
show_hash	function	1388	1399	Formats and presents operator-facing output.
scan_file	function	1402	1421	Performs analysis over a file or dataset.
show_file_scan	function	1424	1438	Formats and presents operator-facing output.
read_iocs	function	1441	1443	Reads a persisted artifact.
write_iocs	function	1446	1447	Writes a persisted artifact.
classify_ioc	function	1450	1465	Converts signals into a risk or behavior label.
add_ioc	function	1468	1480	Support routine in the module's local workflow.
remove_ioc	function	1483	1489	Support routine in the module's local workflow.
search_text_files	function	1492	1514	Support routine in the module's local workflow.
show_hunt	function	1517	1528	Formats and presents operator-facing output.
show_ioc	function	1531	1584	Formats and presents operator-facing output.
shell_path	function	1587	1598	Support routine in the module's local workflow.
shell_cd	function	1601	1606	Support routine in the module's local workflow.
shell_ls	function	1609	1623	Support routine in the module's local workflow.
shell_cat	function	1626	1633	Support routine in the module's local workflow.
shell_head	function	1636	1642	Support routine in the module's local workflow.
shell_tail	function	1645	1654	Support routine in the module's local workflow.
shell_grep	function	1657	1664	Support routine in the module's local workflow.
shell_find	function	1667	1676	Support routine in the module's local workflow.
shell_wc	function	1679	1687	Support routine in the module's local workflow.
shell_du	function	1690	1696	Support routine in the module's local workflow.
shell_stat	function	1699	1710	Support routine in the module's local workflow.
show_status	function	1713	1770	Formats and presents operator-facing output.
show_traffic	function	1773	1806	Formats and presents operator-facing output.

Symbol	Kind	Start	End	Approximate responsibility
show_attacks	function	1809	1833	Formats and presents operator-facing output.
show_malware	function	1836	1859	Formats and presents operator-facing output.
show_learn	function	1862	1878	Formats and presents operator-facing output.
show_scan	function	1881	1909	Formats and presents operator-facing output.
show_reports	function	1912	1924	Formats and presents operator-facing output.
show_runs	function	1927	1950	Formats and presents operator-facing output.
print_shell_help	function	1953	1985	Support routine in the module's local workflow.
parse_limit	function	1988	1996	Interprets text or user input into structured values.
command_shell	function	1999	2014	Support routine in the module's local workflow.
split_shell_command	function	2017	2020	Support routine in the module's local workflow.
run_shell_command	function	2023	2164	Executes a workflow, command, or worker path.
build_parser	function	2167	2254	Constructs a parser, model, or object graph.
normalize_global_args	function	2257	2264	Support routine in the module's local workflow.
main	function	2267	2346	Support routine in the module's local workflow.

ids_app/storage.py

This module contains 50 lines, 7 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
ensure_directories	function	10	12	Support routine in the module's local workflow.
read_json	function	15	19	Reads a persisted artifact.
write_json	function	22	25	Writes a persisted artifact.
job_path	function	28	29	Support routine in the module's local workflow.
run_dir	function	32	33	Executes a workflow, command, or worker path.
run_summary_path	function	36	37	Executes a workflow, command, or worker path.
list_run_summaries	function	40	49	Enumerates available artifacts or resources.

ids_app/terminal.py

This module contains 646 lines, 26 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
_format_number	function	88	93	Support routine in the module's local workflow.
_percent	function	96	97	Support routine in the module's local workflow.
_table	function	100	109	Support routine in the module's local workflow.
_section	function	112	114	Support routine in the module's local workflow.
_print_json	function	117	118	Support routine in the module's local workflow.
show_datasets	function	121	136	Formats and presents operator-facing output.
show_attacks	function	139	169	Formats and presents operator-facing output.
show_features	function	172	179	Formats and presents operator-facing output.

Symbol	Kind	Start	End	Approximate responsibility
show_legacy	function	182	213	Formats and presents operator-facing output.
show_runs	function	216	242	Formats and presents operator-facing output.
show_reports	function	245	266	Formats and presents operator-facing output.
show_best	function	269	299	Formats and presents operator-facing output.
show_head	function	302	315	Formats and presents operator-facing output.
show_overview	function	318	332	Formats and presents operator-facing output.
train_classical	function	335	346	Support routine in the module's local workflow.
train_dnn	function	349	366	Support routine in the module's local workflow.
auto_train	function	369	385	Support routine in the module's local workflow.
_print_train_summary	function	388	407	Support routine in the module's local workflow.
command_shell	function	410	427	Support routine in the module's local workflow.
run_shell_command	function	430	472	Executes a workflow, command, or worker path.
print_shell_help	function	475	495	Support routine in the module's local workflow.
run_train_command	function	498	536	Executes a workflow, command, or worker path.
list_path	function	539	551	Enumerates available artifacts or resources.
_resolve_repo_path	function	554	563	Support routine in the module's local workflow.
build_parser	function	566	602	Constructs a parser, model, or object graph.
main	function	605	642	Support routine in the module's local workflow.

scripts/build_distributions.py

This module contains 198 lines, 9 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
copy_tree	function	24	31	Support routine in the module's local workflow.
write_text	function	34	38	Writes a persisted artifact.
prepare_stage	function	41	67	Support routine in the module's local workflow.
write_launchers	function	70	136	Writes a persisted artifact.
make_zip	function	139	144	Support routine in the module's local workflow.
make_tar	function	147	151	Support routine in the module's local workflow.
build_archives	function	154	167	Constructs a parser, model, or object graph.
build_python_package	function	170	180	Constructs a parser, model, or object graph.
main	function	183	194	Support routine in the module's local workflow.

scripts/build_python_package.py

This module contains 81 lines, 6 top-level function(s), and 0 top-level class(es). The table below is generated directly from the Python AST and therefore reflects the actual structure of the checked-out code.

Symbol	Kind	Start	End	Approximate responsibility
is_python_package_artifact	function	14	20	Support routine in the module's local workflow.
clean_previous_outputs	function	23	33	Support routine in the module's local workflow.

Symbol	Kind	Start	End	Approximate responsibility
stage_sources	function	36	48	Support routine in the module's local workflow.
ensure_build_tools	function	51	56	Support routine in the module's local workflow.
run_setup	function	59	64	Executes a workflow, command, or worker path.
main	function	67	77	Support routine in the module's local workflow.

Appendix D. Annotated Code Listings

Listing 1. Runtime bootstrap and packaged asset staging

The opening block shows how the tool distinguishes a source checkout from an installed package and how it bootstraps bundled CSVs, IOC seeds, and the self-learning model into a writable runtime home.

```

1: from __future__ import annotations
2:
3: import argparse
4: import csv
5: import fnmatch
6: import hashlib
7: import ipaddress
8: import importlib.resources
9: import json
10: import math
11: import os
12: import re
13: import shlex
14: import shutil
15: import socket
16: import subprocess
17: import sys
18: import time
19: import urllib.request
20: from collections import Counter
21: from dataclasses import dataclass
22: from datetime import datetime, timezone
23: from pathlib import Path
24: from typing import Any, Iterable
25: from uuid import uuid4
26:
27: from . import __version__
28:
29: PACKAGE_DIR = Path(__file__).resolve().parent
30: SOURCE_ROOT = PACKAGE_DIR.parent.resolve()
31: DEFAULT_HOME_DIR = Path.home() / ".ids-sentinel-terminal"
32: IS_SOURCE_CHECKOUT = (SOURCE_ROOT / ".git").exists() and (SOURCE_ROOT / "kddtrain.csv").exists() and (SOURCE_ROOT / "kddt
33: ENV_ROOT = os.environ.get("IDS_PRODUCT_HOME")
34: ROOT_DIR = Path(ENV_ROOT).expanduser().resolve() if ENV_ROOT else (SOURCE_ROOT if IS_SOURCE_CHECKOUT else DEFAULT_HOME_DI
35: RUNTIME_MODE = "override" if ENV_ROOT else ("source" if IS_SOURCE_CHECKOUT else "installed")
36: BUNDLED_SEED_FILES = {
37:     "kddtrain.csv": "kddtrain.csv",
38:     "kddtest.csv": "kddtest.csv",
39:     "automation/product/self_learning_model.json": "self_learning_model.json",
40:     "automation/product/iocs.json": "iocs.json",
41: }
42:
43:
44: def _copy_bundled_asset(asset_name: str, destination: Path) -> None:
45:     resource = importlib.resources.files("ids_app").joinpath("assets", asset_name)
46:     destination.parent.mkdir(parents=True, exist_ok=True)
47:     with importlib.resources.as_file(resource) as source_path:
48:         shutil.copy2(source_path, destination)
49:
50:
51: def bootstrap_runtime_home() -> None:
52:     if RUNTIME_MODE == "source":
53:         return
54:     ROOT_DIR.mkdir(parents=True, exist_ok=True)

```

```

55:     for relative_target, asset_name in BUNDLED_SEED_FILES.items():
56:         destination = ROOT_DIR / relative_target
57:         if destination.exists():
58:             continue
59:         _copy_bundled_asset(asset_name, destination)
60:
61:
62: bootstrap_runtime_home()
63:
64: TRAIN_CSV = ROOT_DIR / "kddtrain.csv"
65: TEST_CSV = ROOT_DIR / "kddtest.csv"
66: PRODUCT_DIR = ROOT_DIR / "automation" / "product"
67: EXPORTS_DIR = PRODUCT_DIR / "exports"
68: IMPORTS_DIR = PRODUCT_DIR / "imports"
69: CACHE_DIR = PRODUCT_DIR / "cache"
70: INDEX_DIR = CACHE_DIR / "indexes"
71: COMMAND_CACHE_DIR = CACHE_DIR / "commands"
72: LEGACY_INDEX_DIR = PRODUCT_DIR / "indexes"
73: MODEL_PATH = PRODUCT_DIR / "self_learning_model.json"
74: IOC_PATH = PRODUCT_DIR / "iocs.json"
75:
76: SHELL_STATE = {"cwd": ROOT_DIR, "history": []}
77:
78: EXTERNAL_DATASETS = [
79:     {
80:         "id": "cicids2017",
81:         "name": "CIC-IDS2017",
82:         "source": "Canadian Institute for Cybersecurity, University of New Brunswick",
83:         "url": "https://www.unb.ca/cic/datasets/ids-2017.html",
84:         "format": "PCAP and CICFlowMeter CSV flow data",
85:         "notes": "Contains benign traffic plus FTP/SSH brute force, DoS, Heartbleed, web attacks, infiltration, botnet, D
86:     },
87:     {
88:         "id": "unsw-nb15",
89:         "name": "UNSW-NB15",
90:         "source": "UNSW Canberra Cyber Range Lab",
91:         "url": "https://research.unsw.edu.au/projects/unsw-nb15-dataset",
92:         "format": "CSV network-flow features with binary and attack-category labels",
93:         "notes": "Contains normal traffic and Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shell
94:     },
95:     {
96:         "id": "unsw-nb15-hf",
97:         "name": "UNSW-NB15 Hugging Face mirror",
98:         "source": "Hugging Face community dataset mirror",
99:         "url": "https://huggingface.co/datasets/lacg030175/UNSW-NB15",
100:         "format": "Dataset hub train/test splits",
101:         "notes": "Useful when the Python datasets package is installed. This terminal can still import downloaded CSV fil
102:     },
103: ]
104:
105: COMMON_PORT_RISKS = {
106:     20: "FTP data channel. Plaintext file transfer; watch for data exfiltration.",
107:     21: "FTP control. Plaintext credentials; common brute-force target.",
108:     22: "SSH. Remote admin service; watch for brute-force and unusual geos.",
109:     23: "Telnet. Plaintext remote shell; should usually be disabled.",
110:     25: "SMTP. Mail relay; watch for spam abuse and open relay exposure.",
111:     53: "DNS. Watch for tunneling, amplification, and suspicious resolver exposure.",
112:     80: "HTTP. Web service; inspect for web attacks and exposed admin panels.",
113:     110: "POP3. Plaintext mail retrieval unless wrapped; legacy exposure risk.",
114:     135: "MS RPC. Windows lateral-movement surface; restrict to trusted networks.",
115:     139: "NetBIOS. Legacy Windows file-sharing surface.",
116:     143: "IMAP. Mail retrieval; watch authentication abuse.",
117:     443: "HTTPS. Web service; inspect certificates and web attack logs.",
118:     445: "SMB. High-value Windows file-sharing target; restrict heavily.",
119:     1433: "Microsoft SQL Server. Database exposure risk.",
120:     1521: "Oracle database listener. Database exposure risk.",
121:     2049: "NFS. File-sharing service; restrict to trusted hosts.",
122:     2375: "Docker API without TLS. Critical exposure if reachable.",
123:     3306: "MySQL/MariaDB. Database exposure risk.",
124:     3389: "RDP. Common brute-force and ransomware entry point.",
125:     5432: "PostgreSQL. Database exposure risk.",
126:     5900: "VNC. Remote desktop; often weakly protected.",
127:     6379: "Redis. Critical if unauthenticated or internet-exposed.",
128:     8080: "Alternate HTTP/proxy/admin service.",
129:     9200: "Elasticsearch. Data exposure risk if unauthenticated.",
130:     11211: "Memcached. Amplification and data exposure risk.",

```

Listing 2. Streaming row ingestion and pure-Python model learning

This excerpt is central to the report: it demonstrates that the self-learning path is intentionally dependency-light and can operate even when the heavier scientific stack is absent.

```
403: def iter_kdd_rows(path: Path, limit: int | None = None) -> Iterable[tuple[int, str, list[float]]]:
404:     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
405:         reader = csv.reader(handle)
406:         emitted = 0
407:         for row_number, row in enumerate(reader, start=1):
408:             if len(row) < len(FEATURE_NAMES) + 1:
409:                 continue
410:             features = [safe_float(value) for value in row[1 : len(FEATURE_NAMES) + 1]]
411:             yield row_number, row[0].strip(), features
412:             emitted += 1
413:             if limit is not None and emitted >= limit:
414:                 return
415:
416:
417: def iter_generated_rows(path: Path, limit: int | None = None) -> Iterable[tuple[int, str, list[float]]]:
418:     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
419:         reader = csv.DictReader(handle)
420:         emitted = 0
421:         for row_number, row in enumerate(reader, start=2):
422:             label = str(row.get("actual_label") or row.get("label") or "").strip()
423:             if label not in BINARY_LABELS:
424:                 continue
425:             features = [safe_float(row.get(f"{CSV_FEATURE_PREFIX}{name}", "0")) for name in FEATURE_NAMES]
426:             yield row_number, label, features
427:             emitted += 1
428:             if limit is not None and emitted >= limit:
429:                 return
430:
431:
432: @dataclass
433: class RunningStat:
434:     count: int = 0
435:     mean: float = 0.0
436:     m2: float = 0.0
437:     min_value: float = math.inf
438:     max_value: float = -math.inf
439:
440:     def update(self, value: float) -> None:
441:         self.count += 1
442:         delta = value - self.mean
443:         self.mean += delta / self.count
444:         delta2 = value - self.mean
445:         self.m2 += delta * delta2
446:         self.min_value = min(self.min_value, value)
447:         self.max_value = max(self.max_value, value)
448:
449:     def to_json(self) -> dict[str, float | int]:
450:         variance = self.m2 / max(self.count - 1, 1)
451:         return {
452:             "count": self.count,
453:             "mean": round(self.mean, 8),
454:             "variance": round(max(variance, 1e-9), 8),
455:             "min": round(self.min_value if self.count else 0.0, 8),
456:             "max": round(self.max_value if self.count else 0.0, 8),
457:         }
458:
459:
460: def empty_label_stats() -> dict[str, list[RunningStat]]:
461:     return {label: [RunningStat() for _ in FEATURE_NAMES] for label in BINARY_LABELS}
462:
463:
464: def update_model_stats(stats: dict[str, list[RunningStat]], labels: Counter[str], label: str, features: list[float]) -> None:
465:     if label not in stats:
466:         return
467:     labels[label] += 1
468:     for index, value in enumerate(features):
469:         stats[label][index].update(value)
470:
```

```

471:
472: def generated_export_paths() -> list[Path]:
473:     if not EXPORTS_DIR.exists():
474:         return []
475:     return sorted(EXPORTS_DIR.glob("traffic_analysis_*.csv"))
476:
477:
478: def learn_model(
479:     *,
480:     limit: int | None = None,
481:     include_generated: bool = True,
482:     include_test: bool = False,
483: ) -> dict[str, Any]:
484:     ensure_product_dirs()
485:     stats = empty_label_stats()
486:     labels: Counter[str] = Counter()
487:     sources: list[dict[str, Any]] = []
488:
489:     source_paths = [TRAIN_CSV]
490:     if include_test:
491:         source_paths.append(TEST_CSV)
492:
493:     for path in source_paths:
494:         rows_used = 0
495:         source_labels: Counter[str] = Counter()
496:         for _, label, features in iter_kdd_rows(path, limit):
497:             update_model_stats(stats, labels, label, features)
498:             source_labels[label] += 1
499:             rows_used += 1
500:         sources.append({"path": relative_path(path), "rows_used": rows_used, "label_counts": dict(source_labels)})
501:
502:     if include_generated:
503:         for path in generated_export_paths():
504:             rows_used = 0
505:             source_labels = Counter()
506:             for _, label, features in iter_generated_rows(path, limit):
507:                 update_model_stats(stats, labels, label, features)
508:                 source_labels[label] += 1
509:                 rows_used += 1
510:             if rows_used:
511:                 sources.append({"path": relative_path(path), "rows_used": rows_used, "label_counts": dict(source_labels)})
512:
513:     total_rows = sum(labels.values())
514:     if total_rows == 0 or labels["0"] == 0 or labels["1"] == 0:
515:         raise RuntimeError("not enough labeled normal and attack rows to build a model")
516:
517:     label_payload: dict[str, Any] = {}
518:     for label in BINARY_LABELS:
519:         label_payload[label] = {
520:             "name": BINARY_LABELS[label],
521:             "count": labels[label],
522:             "prior": labels[label] / total_rows,
523:             "features": [item.to_json() for item in stats[label]],
524:         }
525:
526:     top_indicators = rank_indicators(label_payload)
527:     model = {
528:         "version": 1,
529:         "created_at": utc_now(),
530:         "model_type": "streaming_gaussian_profile",
531:         "description": "Pure-Python self-learning profile built from labeled IDS CSV rows and terminal-generated analysis",
532:         "features": FEATURE_NAMES,
533:         "labels": label_payload,
534:         "total_rows": total_rows,
535:         "sources": sources,
536:         "top_indicators": top_indicators[:12],
537:     }
538:     write_json(MODEL_PATH, model)
539:     cache_artifact("learn", {"model_path": relative_path(MODEL_PATH), "rows_learned": total_rows, "sources": sources})
540:     return model
541:
542:
543: def rank_indicators(label_payload: dict[str, Any]) -> list[dict[str, Any]]:
544:     rows = []
545:     normal_stats = label_payload["0"]["features"]
546:     attack_stats = label_payload["1"]["features"]

```

```

547:     for index, name in enumerate(FEATURE_NAMES):
548:         normal = normal_stats[index]
549:         attack = attack_stats[index]
550:         pooled_std = math.sqrt((float(normal["variance"]) + float(attack["variance"])) / 2.0)
551:         score = abs(float(attack["mean"]) - float(normal["mean"])) / max(pooled_std, 1e-6)
552:         rows.append(
553:             {
554:                 "feature": name,
555:                 "separation": round(score, 6),
556:                 "normal_mean": round(float(normal["mean"]), 6),
557:                 "attack_mean": round(float(attack["mean"]), 6),
558:             }
559:         )
560:     rows.sort(key=lambda item: item["separation"], reverse=True)
561:     return rows
562:
563:

```

Listing 3. Probabilistic scoring and behavioral classification

These functions transform raw numeric feature values into log-probability scores, risk levels, and narrative behavior classes such as `dos\_flood` or `probe\_scan`.

```

573: def gaussian_log_probability(features: list[float], label_model: dict[str, Any], indicator_names: set[str] | None = None)
574:     logp = math.log(max(float(label_model["prior"]), 1e-12))
575:     for index, value in enumerate(features):
576:         name = FEATURE_NAMES[index]
577:         if indicator_names is not None and name not in indicator_names:
578:             continue
579:         stat = label_model["features"][index]
580:         mean = float(stat["mean"])
581:         variance = max(float(stat["variance"]), 1e-6)
582:         logp += -0.5 * math.log(2.0 * math.pi * variance)
583:         logp += -((value - mean) ** 2) / (2.0 * variance)
584:     return logp
585:
586:
587: def score_row(model: dict[str, Any], features: list[float]) -> dict[str, Any]:
588:     indicator_names = {item["feature"] for item in model.get("top_indicators", [])[:16]} or None
589:     normal_log = gaussian_log_probability(features, model["labels"]["0"], indicator_names)
590:     attack_log = gaussian_log_probability(features, model["labels"]["1"], indicator_names)
591:     delta = max(min(attack_log - normal_log, 60.0), -60.0)
592:     attack_probability = 1.0 / (1.0 + math.exp(-delta))
593:     predicted = "1" if attack_probability >= 0.5 else "0"
594:     confidence = abs(attack_probability - 0.5) * 2.0
595:     family, reasons = classify_behavior(features, attack_probability, model)
596:     return {
597:         "predicted_label": predicted,
598:         "predicted_name": BINARY_LABELS[predicted],
599:         "risk_score": round(attack_probability, 6),
600:         "confidence": round(confidence, 6),
601:         "risk_level": risk_level(attack_probability),
602:         "family": family,
603:         "reasons": reasons,
604:     }
605:
606:
607: def feature_map(features: list[float]) -> dict[str, float]:
608:     return dict(zip(FEATURE_NAMES, features))
609:
610:
611: def risk_level(score: float) -> str:
612:     if score >= 0.9:
613:         return "critical"
614:     if score >= 0.75:
615:         return "high"
616:     if score >= 0.55:
617:         return "medium"
618:     return "low"
619:
620:
621: def classify_behavior(features: list[float], risk_score: float, model: dict[str, Any]) -> tuple[str, str]:
622:     values = feature_map(features)
623:     families: list[str] = []

```

```

624:     reasons: list[str] = []
625:
626:     if values["count"] >= 80 or values["srv_count"] >= 80 or values["error_rate"] >= 0.5 or values["srv_error_rate"] >= 0.5:
627:         families.append("dos_flood")
628:         reasons.append("high connection or service-error rate")
629:     if values["diff_srv_rate"] >= 0.35 or values["srv_diff_host_rate"] >= 0.35 or values["dst_host_srv_diff_host_rate"] >= 0.35:
630:         families.append("probe_scan")
631:         reasons.append("high service or host diversity")
632:     if values["num_failed_logins"] > 0 or values["is_guest_login"] > 0 or values["logged_in"] == 0 and values["hot"] >= 2:
633:         families.append("credential_abuse")
634:         reasons.append("login or credential anomaly")
635:     if values["root_shell"] > 0 or values["su_attempted"] > 0 or values["num_compromised"] > 0 or values["num_root"] > 0:
636:         families.append("privilege_escalation")
637:         reasons.append("compromise or privilege signal")
638:     if values["num_file_creations"] > 0 or values["num_shells"] > 0 or values["num_access_files"] > 0:
639:         families.append("malware_like_activity")
640:         reasons.append("file, shell, or access-file behavior")
641:     if values["wrong_fragment"] > 0 or values["urgent"] > 0 or values["src_bytes"] > 100000 or values["dst_bytes"] > 100000:
642:         families.append("payload_or_exfiltration")
643:         reasons.append("fragment, urgent, or high byte volume")
644:
645:     if risk_score < 0.55 and not families:
646:         return "normal", "close to learned normal profile"
647:
648:     if not families:
649:         for item in model.get("top_indicators", []):
650:             name = item["feature"]
651:             index = FEATURE_NAMES.index(name)
652:             value = features[index]
653:             normal_mean = float(item["normal_mean"])
654:             attack_mean = float(item["attack_mean"])
655:             if abs(value - attack_mean) < abs(value - normal_mean):
656:                 direction = "high" if attack_mean > normal_mean else "low"
657:                 reasons.append(f"{name} is {direction} versus normal profile")
658:             families.append("network_attack")
659:
660:     return families[0], "; ".join(reasons[:4]) if reasons else "matches learned attack profile"
661:
662:

```

Listing 4. CSV analysis and export pipeline

The analysis routine ties together row parsing, scoring, export generation, and cached metadata creation. It is the core of the operational terminal workflow.

```

801: def analyze_csv(
802:     source: Path,
803:     *,
804:     limit: int | None = 5000,
805:     export: bool = True,
806:     model: dict[str, Any] | None = None,
807: ) -> dict[str, Any]:
808:     ensure_product_dirs()
809:     model = model or load_or_learn_model()
810:     analysis_id = f"scan-{compact_timestamp()}"
811:     export_csv_path = EXPORTS_DIR / f"traffic_analysis_{compact_timestamp()}.csv"
812:     export_json_path = export_csv_path.with_suffix(".json")
813:
814:     total = 0
815:     malformed = 0
816:     actual_counts: Counter[str] = Counter()
817:     predicted_counts: Counter[str] = Counter()
818:     risk_counts: Counter[str] = Counter()
819:     family_counts: Counter[str] = Counter()
820:     protocol_counts: Counter[str] = Counter()
821:     service_counts: Counter[str] = Counter()
822:     flag_counts: Counter[str] = Counter()
823:     metrics = Counter()
824:     risk_sum = 0.0
825:
826:     fieldnames = [
827:         "analysis_id",
828:         "analyzed_at",
829:         "source_file",

```

```

830:         "row_number",
831:         "actual_label",
832:         "actual_name",
833:         "predicted_label",
834:         "predicted_name",
835:         "risk_score",
836:         "confidence",
837:         "risk_level",
838:         "family",
839:         "reasons",
840:         *[f"{CSV_FEATURE_PREFIX}{name}" for name in FEATURE_NAMES],
841:     ]
842:
843:     writer = None
844:     export_handle = None
845:     try:
846:         if export:
847:             export_handle = export_csv_path.open("w", encoding="utf-8", newline="")
848:             writer = csv.DictWriter(export_handle, fieldnames=fieldnames)
849:             writer.writeheader()
850:
851:             with source.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
852:                 reader = csv.reader(handle)
853:                 for row_number, row in enumerate(reader, start=1):
854:                     if len(row) < len(FEATURE_NAMES) + 1:
855:                         malformed += 1
856:                         continue
857:                     label = row[0].strip()
858:                     features = [safe_float(value) for value in row[1 : len(FEATURE_NAMES) + 1]]
859:                     result = score_row(model, features)
860:                     total += 1
861:                     actual_counts[label] += 1
862:                     predicted_counts[result["predicted_label"]] += 1
863:                     risk_counts[result["risk_level"]] += 1
864:                     family_counts[result["family"]] += 1
865:                     protocol_counts[row[2].strip()] += 1
866:                     service_counts[row[3].strip()] += 1
867:                     flag_counts[row[4].strip()] += 1
868:                     risk_sum += float(result["risk_score"])
869:
870:                     if label in BINARY_LABELS:
871:                         actual_attack = label == "1"
872:                         predicted_attack = result["predicted_label"] == "1"
873:                         if actual_attack and predicted_attack:
874:                             metrics["tp"] += 1
875:                         elif actual_attack and not predicted_attack:
876:                             metrics["fn"] += 1
877:                         elif not actual_attack and predicted_attack:
878:                             metrics["fp"] += 1
879:                         else:
880:                             metrics["tn"] += 1
881:
882:                     if writer:
883:                         output_row = {
884:                             "analysis_id": analysis_id,
885:                             "analyzed_at": utc_now(),
886:                             "source_file": relative_path(source),
887:                             "row_number": row_number,
888:                             "actual_label": label,
889:                             "actual_name": BINARY_LABELS.get(label, "unknown"),
890:                             **result,
891:                         }
892:                         output_row["reasons"] = result["reasons"]
893:                         for index, name in enumerate(FEATURE_NAMES):
894:                             output_row[f"{CSV_FEATURE_PREFIX}{name}"] = features[index]
895:                         writer.writerow(output_row)
896:
897:                     if limit is not None and total >= limit:
898:                         break
899:     finally:
900:         if export_handle:
901:             export_handle.close()
902:
903:     precision = metrics["tp"] / max(metrics["tp"] + metrics["fp"], 1)
904:     recall = metrics["tp"] / max(metrics["tp"] + metrics["fn"], 1)
905:     accuracy = (metrics["tp"] + metrics["tn"]) / max(sum(metrics.values()), 1)

```

```

906:     f1 = (2 * precision * recall) / max(precision + recall, 1e-12)
907:     summary = {
908:         "analysis_id": analysis_id,
909:         "created_at": utc_now(),
910:         "source_file": relative_path(source),
911:         "rows_analyzed": total,
912:         "malformed_rows": malformed,
913:         "limit": limit,
914:         "average_risk_score": round(risk_sum / total, 6) if total else 0,
915:         "actual_counts": dict(actual_counts),
916:         "predicted_counts": dict(predicted_counts),
917:         "risk_counts": dict(risk_counts),
918:         "family_counts": dict(family_counts),
919:         "top_protocols": protocol_counts.most_common(8),
920:         "top_services": service_counts.most_common(8),
921:         "top_flags": flag_counts.most_common(8),
922:         "metrics": {
923:             "accuracy": round(accuracy, 6),
924:             "precision": round(precision, 6),
925:             "recall": round(recall, 6),
926:             "f1": round(f1, 6),
927:             "tp": metrics["tp"],
928:             "tn": metrics["tn"],
929:             "fp": metrics["fp"],
930:             "fn": metrics["fn"],
931:         },
932:         "model_created_at": model.get("created_at"),
933:         "export_csv": relative_path(export_csv_path) if export else None,
934:         "export_json": relative_path(export_json_path) if export else None,
935:     }
936:     if export:
937:         write_json(export_json_path, summary)
938:     cache_artifact("scan", summary)
939:     return summary
940:
941:

```

Listing 5. Local service knowledge, netstat parsing, and active probing

This block explains why the network triage surface is useful but also why the report classifies it as Windows-shaped rather than fully portable.

```

1114: def load_services() -> dict[int, str]:
1115:     services: dict[int, str] = {
1116:         20: "ftp-data",
1117:         21: "ftp",
1118:         22: "ssh",
1119:         23: "telnet",
1120:         25: "smtp",
1121:         53: "domain",
1122:         80: "http",
1123:         110: "pop3",
1124:         135: "msrpc",
1125:         139: "netbios-ssn",
1126:         143: "imap",
1127:         443: "https",
1128:         445: "microsoft-ds",
1129:         1433: "ms-sql-s",
1130:         3306: "mysql",
1131:         3389: "ms-wbt-server",
1132:         5432: "postgresql",
1133:         6379: "redis",
1134:         8080: "http-alt",
1135:         9200: "elasticsearch",
1136:         27017: "mongodb",
1137:     }
1138:     services_file = Path(os.environ.get("SystemRoot", "C:\\Windows")) / "System32" / "drivers" / "etc" / "services"
1139:     if services_file.exists():
1140:         with services_file.open("r", encoding="utf-8", errors="ignore") as handle:
1141:             for line in handle:
1142:                 line = line.strip()
1143:                 if not line or line.startswith("#"):
1144:                     continue
1145:                 parts = line.split()

```

```

1146:         if len(parts) < 2 or "/" not in parts[1]:
1147:             continue
1148:         port_text, proto = parts[1].split("/", 1)
1149:         if proto.lower() not in {"tcp", "udp"}:
1150:             continue
1151:         try:
1152:             services.setdefault(int(port_text), parts[0])
1153:         except ValueError:
1154:             continue
1155:     return services
1156:
1157:
1158: def parse_ports(text: str) -> list[int]:
1159:     if text.lower() in {"common", "top"}:
1160:         return COMMON_PROBE_PORTS
1161:     ports: set[int] = set()
1162:     for part in text.split(","):
1163:         part = part.strip()
1164:         if not part:
1165:             continue
1166:         if "-" in part:
1167:             start_text, end_text = part.split("-", 1)
1168:             start = int(start_text)
1169:             end = int(end_text)
1170:             if end < start:
1171:                 start, end = end, start
1172:             for port in range(start, min(end, start + 127) + 1):
1173:                 if 1 <= port <= 65535:
1174:                     ports.add(port)
1175:         else:
1176:             port = int(part)
1177:             if 1 <= port <= 65535:
1178:                 ports.add(port)
1179:     if len(ports) > 128:
1180:         raise ValueError("port list is capped at 128 ports per probe")
1181:     return sorted(ports)
1182:
1183:
1184: def split_host_port(value: str) -> tuple[str, int | None]:
1185:     value = value.strip()
1186:     if value.startswith("[") and "]" in value:
1187:         host, port_text = value.rsplit(":", 1)
1188:         return host.strip("[]"), int(port_text) if port_text.isdigit() else None
1189:     if ":" in value and value.count(":") == 1:
1190:         host, port_text = value.rsplit(":", 1)
1191:         return host, int(port_text) if port_text.isdigit() else None
1192:     return value, None
1193:
1194:
1195: def parse_netstat() -> list[dict[str, Any]]:
1196:     try:
1197:         completed = subprocess.run(["netstat", "-ano"], capture_output=True, text=True, timeout=15, check=False)
1198:     except FileNotFoundError:
1199:         return []
1200:     rows = []
1201:     for line in completed.stdout.splitlines():
1202:         parts = line.split()
1203:         if not parts or parts[0] not in {"TCP", "UDP"}:
1204:             continue
1205:         proto = parts[0]
1206:         if proto == "TCP" and len(parts) >= 5:
1207:             local_host, local_port = split_host_port(parts[1])
1208:             remote_host, remote_port = split_host_port(parts[2])
1209:             rows.append(
1210:                 {
1211:                     "proto": proto,
1212:                     "local": parts[1],
1213:                     "local_host": local_host,
1214:                     "local_port": local_port,
1215:                     "remote": parts[2],
1216:                     "remote_host": remote_host,
1217:                     "remote_port": remote_port,
1218:                     "state": parts[3],
1219:                     "pid": parts[4],
1220:                 }
1221:             )

```

```

1222:         elif proto == "UDP" and len(parts) >= 4:
1223:             local_host, local_port = split_host_port(parts[1])
1224:             rows.append(
1225:                 {
1226:                     "proto": proto,
1227:                     "local": parts[1],
1228:                     "local_host": local_host,
1229:                     "local_port": local_port,
1230:                     "remote": parts[2],
1231:                     "remote_host": "*",
1232:                     "remote_port": None,
1233:                     "state": "UDP",
1234:                     "pid": parts[3],
1235:                 }
1236:             )
1237:         return rows
1238:
1239:
1240: def show_netstat(json_output: bool = False, only_listening: bool = False, limit: int = 40) -> None:
1241:     services = load_services()
1242:     rows = parse_netstat()
1243:     if only_listening:
1244:         rows = [row for row in rows if row["state"] in {"LISTENING", "UDP"}]
1245:     rows = sorted(rows, key=lambda item: (item.get("local_port") or 0, item["proto"], item["pid"]))
1246:     payload = rows[:limit]
1247:     cache_artifact("ports" if only_listening else "netstat", payload)
1248:     if json_output:
1249:         print_json(payload)
1250:         return
1251:     section("Network Connections")
1252:     if not payload:
1253:         print("No netstat rows found.")
1254:         return
1255:     print(table(["Proto", "Local", "Service", "Remote", "State", "PID"], [
1256:         [
1257:             row["proto"],
1258:             row["local"],
1259:             services.get(row.get("local_port") or -1, ""),
1260:             row["remote"],
1261:             row["state"],
1262:             row["pid"],
1263:         ]
1264:         for row in payload
1265:     ]))
1266:
1267:
1268: def show_port(port: int, json_output: bool = False) -> None:
1269:     services = load_services()
1270:     payload = {
1271:         "port": port,
1272:         "service": services.get(port, "unknown"),
1273:         "risk": COMMON_PORT_RISKS.get(port, "No specific built-in note. Validate whether this service should be exposed."),
1274:         "local_matches": [row for row in parse_netstat() if row.get("local_port") == port],
1275:     }
1276:     cache_artifact("port", payload)
1277:     if json_output:
1278:         print_json(payload)
1279:         return
1280:     section(f"Port {port}")
1281:     print(f"Service: {payload['service']}")
1282:     print(f"Risk: {payload['risk']}")
1283:     if payload["local_matches"]:
1284:         print()
1285:         print(table(["Proto", "Local", "Remote", "State", "PID"], [
1286:             [row["proto"], row["local"], row["remote"], row["state"], row["pid"]] for row in payload["local_matches"]
1287:         ]))
1288:
1289:
1290: def probe_ports(host: str, ports: list[int], timeout_seconds: float = 0.2) -> list[dict[str, Any]]:
1291:     results = []
1292:     services = load_services()
1293:     for port in ports:
1294:         started = time.perf_counter()
1295:         status = "closed"
1296:         try:
1297:             with socket.create_connection((host, port), timeout=timeout_seconds):

```

```

1298:         status = "open"
1299:     except (TimeoutError, OSError):
1300:         status = "closed"
1301:     elapsed_ms = round((time.perf_counter() - started) * 1000, 2)
1302:     results.append({"host": host, "port": port, "service": services.get(port, ""), "status": status, "elapsed_ms": elapsed_ms})
1303:     return results
1304:
1305:
1306: def show_probe(host: str, ports_text: str, json_output: bool = False) -> None:
1307:     ports = parse_ports(ports_text)
1308:     payload = probe_ports(host, ports)
1309:     cache_artifact("probe", payload)

```

Listing 6. File triage, IOC handling, and text hunting

The repository combines lightweight file hashing with IOC management and keyword search to approximate analyst-grade quick triage.

```

1402: def scan_file(path: Path) -> dict[str, Any]:
1403:     payload = hash_file(path)
1404:     max_bytes = 5 * 1024 * 1024
1405:     with path.open("rb") as handle:
1406:         data = handle.read(max_bytes)
1407:         lower = data.lower()
1408:         ascii_text = lower.decode("latin-1", errors="ignore")
1409:         findings = []
1410:         if data.startswith(b"MZ"):
1411:             findings.append("windows_pe_executable")
1412:         if b"\x7fELF" in data[:4]:
1413:             findings.append("linux_elf_executable")
1414:         if b"PK\x03\x04" in data[:4]:
1415:             findings.append("zip_or_office_container")
1416:         for pattern in SUSPICIOUS_FILE_PATTERNS:
1417:             if pattern in ascii_text:
1418:                 findings.append(f"suspicious_string:{pattern}")
1419:     payload["findings"] = findings
1420:     payload["triage"] = "suspicious" if findings else "no_built_in_findings"
1421:     return payload
1422:
1423:
1424: def show_file_scan(path: Path, json_output: bool = False) -> None:
1425:     payload = scan_file(path)
1426:     cache_artifact("filescan", payload)
1427:     if json_output:
1428:         print_json(payload)
1429:     return
1430:     section("File Triage")
1431:     print(f"Path: {payload['path']}")
1432:     print(f"Size: {payload['size']:,}")
1433:     print(f"SHA256: {payload['sha256']}")
1434:     print(f"SHA1: {payload['sha1']}")
1435:     print(f"MD5: {payload['md5']}")
1436:     print(f"Triage: {payload['triage']}")
1437:     if payload["findings"]:
1438:         print(table(["Finding"], [[item] for item in payload["findings"]]))
1439:
1440:
1441: def read_iocs() -> list[dict[str, Any]]:
1442:     payload = read_json(IOC_PATH, default={"iocs": []})
1443:     return payload.get("iocs", [])
1444:
1445:
1446: def write_iocs(iocs: list[dict[str, Any]]) -> None:
1447:     write_json(IOC_PATH, {"updated_at": utc_now(), "iocs": iocs})
1448:
1449:
1450: def classify_ioc(value: str, explicit_type: str | None = None) -> str:
1451:     if explicit_type:
1452:         return explicit_type
1453:     try:
1454:         ipaddress.ip_address(value)
1455:         return "ip"
1456:     except ValueError:
1457:         pass

```

```

1458:     if value.isdigit() and 1 <= int(value) <= 65535:
1459:         return "port"
1460:     lowered = value.lower()
1461:     if re.fullmatch(r"[a-f0-9]{32}|[a-f0-9]{40}|[a-f0-9]{64}", lowered):
1462:         return "hash"
1463:     if "." in value and not any(char.isspace() for char in value):
1464:         return "domain"
1465:     return "string"
1466:
1467:
1468: def add_ioc(value: str, ioc_type: str | None = None, note: str = "") -> dict[str, Any]:
1469:     iocs = read_iocs()
1470:     entry = {
1471:         "id": hashlib.sha1(f"{value}|{utc_now()}".encode("utf-8")).hexdigest()[:10],
1472:         "type": classify_ioc(value, ioc_type),
1473:         "value": value,
1474:         "note": note,
1475:         "created_at": utc_now(),
1476:     }
1477:     iocs.append(entry)
1478:     write_iocs(iocs)
1479:     cache_artifact("ioc_add", entry)
1480:     return entry
1481:
1482:
1483: def remove_ioc(ioc_id: str) -> bool:
1484:     iocs = read_iocs()
1485:     kept = [item for item in iocs if item.get("id") != ioc_id]
1486:     write_iocs(kept)
1487:     removed = len(kept) != len(iocs)
1488:     cache_artifact("ioc_remove", {"ioc_id": ioc_id, "removed": removed})
1489:     return removed
1490:
1491:
1492: def search_text_files(pattern: str, paths: list[Path], limit: int = 50) -> list[dict[str, Any]]:
1493:     results = []
1494:     lowered = pattern.lower()
1495:     for path in paths:
1496:         if not path.exists() or not path.is_file():
1497:             continue
1498:         try:
1499:             if path.suffix.lower() in TEXT_SEARCH_SKIP_SUFFIXES:
1500:                 continue
1501:             if path.stat().st_size > TEXT_SEARCH_MAX_FILE_BYTES:
1502:                 continue
1503:         except OSError:
1504:             continue
1505:         try:
1506:             with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
1507:                 for line_number, line in enumerate(handle, start=1):
1508:                     if lowered in line.lower():
1509:                         results.append({"path": relative_path(path), "line": line_number, "text": line.strip()[:240]})
1510:                         if len(results) >= limit:
1511:                             return results
1512:         except OSError:
1513:             continue
1514:     return results
1515:
1516:
1517: def show_hunt(pattern: str, path: Path | None = None, json_output: bool = False, limit: int = 50) -> None:
1518:     paths = [path] if path else all_csv_sources(include_exports=True)
1519:     payload = {"pattern": pattern, "matches": search_text_files(pattern, paths, limit=limit)}
1520:     cache_artifact("hunt", payload)
1521:     if json_output:
1522:         print_json(payload)
1523:         return
1524:     section("Hunt")
1525:     if not payload["matches"]:
1526:         print("No matches.")
1527:         return
1528:     print(table(["Path", "Line", "Text"], [[item["path"], item["line"], item["text"]] for item in payload["matches"]]))
1529:
1530:
1531: def show_ioc(args: list[str], json_output: bool = False) -> None:
1532:     action = args[0].lower() if args else "list"
1533:     if action == "list":

```

```

1534:     payload = read_iocs()
1535:     cache_artifact("ioc_list", payload)
1536:     if json_output:
1537:         print_json(payload)
1538:         return
1539:     section("IOCs")
1540:     if not payload:
1541:         print("No IOCs stored.")
1542:         return
1543:     print(table(["ID", "Type", "Value", "Note"], [[item["id"], item["type"], item["value"], item.get("note", "")] for item in payload]))
1544:     return
1545: if action == "add":
1546:     if len(args) < 2:
1547:         raise ValueError("usage: ioc add <value> [type] [note]")
1548:     value = args[1]
1549:     ioc_type = args[2] if len(args) >= 3 and args[2] in {"ip", "domain", "hash", "port", "string", "malware"} else None
1550:     note_start = 3 if ioc_type else 2
1551:     payload = add_ioc(value, ioc_type, " ".join(args[note_start:]))
1552:     if json_output:
1553:         print_json(payload)
1554:     else:
1555:         print(f"Added IOC {payload['id']} ({payload['type']}): {payload['value']}")
1556:         return
1557: if action in {"rm", "remove", "delete"}:
1558:     if len(args) < 2:
1559:         raise ValueError("usage: ioc remove <id>")
1560:     removed = remove_ioc(args[1])
1561:     print("removed" if removed else "not found")
1562:     return
1563: if action == "hunt":
1564:     iocs = read_iocs()
1565:     matches = []
1566:     for item in iocs:
1567:         found = search_text_files(str(item["value"]), all_csv_sources(include_exports=True), limit=20)
1568:         if found:
1569:             matches.append({"ioc": item, "matches": found})
1570:     cache_artifact("ioc_hunt", matches)
1571:     if json_output:
1572:         print_json(matches)
1573:         return
1574:     section("IOC Hunt")
1575:     if not matches:
1576:         print("No IOC matches.")
1577:         return
1578:     rows = []

```

Listing 7. Embedded shell command layer

Instead of handing the operator directly to the host shell, the product offers bounded, read-oriented shell-like commands. That design reduces destructive risk and keeps the UX coherent.

```

1587: def shell_path(path_text: str | None = None) -> Path:
1588:     if not path_text:
1589:         return Path(SHELL_STATE.get("cwd", ROOT_DIR))
1590:     path = Path(path_text)
1591:     if not path.is_absolute():
1592:         path = Path(SHELL_STATE.get("cwd", ROOT_DIR)) / path
1593:     resolved = path.resolve()
1594:     try:
1595:         resolved.relative_to(ROOT_DIR)
1596:     except ValueError:
1597:         raise ValueError("path must stay inside the IDS Sentinel home directory")
1598:     return resolved
1599:
1600:
1601: def shell_cd(path_text: str | None) -> None:
1602:     path = shell_path(path_text or ".")
1603:     if not path.exists() or not path.is_dir():
1604:         raise ValueError(f"not a directory: {path_text}")
1605:     SHELL_STATE["cwd"] = path
1606:     print(relative_path(path) or ".")
1607:
1608:
1609: def shell_ls(path_text: str | None = None, all_files: bool = False) -> None:

```

```

1610:     path = shell_path(path_text or ".")
1611:     if path.is_file():
1612:         print(relative_path(path))
1613:         return
1614:     rows = []
1615:     for child in sorted(path.iterdir(), key=lambda item: (item.is_file(), item.name.lower())):
1616:         if not all_files and child.name.startswith("."):
1617:             continue
1618:         rows.append([
1619:             child.name + ("/" if child.is_dir() else ""),
1620:             "dir" if child.is_dir() else child.stat().st_size,
1621:             datetime.fromtimestamp(child.stat().st_mtime).isoformat(timespec="seconds"),
1622:         ])
1623:     print(table(["Name", "Size", "Modified"], rows))
1624:
1625:
1626: def shell_cat(path_text: str, limit_bytes: int = 1024 * 1024) -> None:
1627:     path = shell_path(path_text)
1628:     if not path.is_file():
1629:         raise ValueError("cat requires a file")
1630:     if path.stat().st_size > limit_bytes:
1631:         raise ValueError("file is too large for cat; use head, tail, or grep")
1632:     with path.open("r", encoding="utf-8", errors="ignore") as handle:
1633:         print(handle.read())
1634:
1635:
1636: def shell_head(path_text: str, lines: int = 20) -> None:
1637:     path = shell_path(path_text)
1638:     with path.open("r", encoding="utf-8", errors="ignore") as handle:
1639:         for index, line in enumerate(handle):
1640:             if index >= lines:
1641:                 break
1642:             print(line.rstrip())
1643:
1644:
1645: def shell_tail(path_text: str, lines: int = 20) -> None:
1646:     path = shell_path(path_text)
1647:     buffer: list[str] = []
1648:     with path.open("r", encoding="utf-8", errors="ignore") as handle:
1649:         for line in handle:
1650:             buffer.append(line.rstrip())
1651:             if len(buffer) > lines:
1652:                 buffer.pop(0)
1653:     for line in buffer:
1654:         print(line)
1655:
1656:
1657: def shell_grep(pattern: str, path_text: str | None = None, limit: int = 50) -> None:
1658:     path = shell_path(path_text or ".")
1659:     paths = [path] if path.is_file() else [item for item in path.rglob("*") if item.is_file()]
1660:     matches = search_text_files(pattern, paths, limit=limit)
1661:     if not matches:
1662:         print("No matches.")
1663:         return
1664:     print(table(["Path", "Line", "Text"], [[item["path"], item["line"], item["text"]] for item in matches]))
1665:
1666:
1667: def shell_find(pattern: str = "*", path_text: str | None = None, limit: int = 200) -> None:
1668:     root = shell_path(path_text or ".")
1669:     rows = []
1670:     iterator = root.rglob(pattern) if root.is_dir() else [root]
1671:     for item in iterator:
1672:         if fnmatch.fnmatch(item.name.lower(), pattern.lower()):
1673:             rows.append([relative_path(item), "dir" if item.is_dir() else item.stat().st_size])
1674:             if len(rows) >= limit:
1675:                 break
1676:     print(table(["Path", "Size"], rows))
1677:
1678:
1679: def shell_wc(path_text: str) -> None:
1680:     path = shell_path(path_text)
1681:     lines = words = chars = 0
1682:     with path.open("r", encoding="utf-8", errors="ignore") as handle:
1683:         for line in handle:
1684:             lines += 1
1685:             words += len(line.split())

```

```

1686:         chars += len(line)
1687:     print(table(["Lines", "Words", "Chars", "Path"], [[lines, words, chars, relative_path(path)]]))
1688:
1689:
1690: def shell_du(path_text: str | None = None) -> None:
1691:     path = shell_path(path_text or ".")
1692:     if path.is_file():
1693:         size = path.stat().st_size
1694:     else:
1695:         size = sum(item.stat().st_size for item in path.rglob("*") if item.is_file())
1696:     print(table(["Path", "Bytes", "MB"], [[relative_path(path) or ".", size, round(size / (1024 * 1024), 2)]]))
1697:
1698:
1699: def shell_stat(path_text: str) -> None:
1700:     path = shell_path(path_text)
1701:     stat = path.stat()
1702:     print_json(
1703:         {
1704:             "path": relative_path(path),
1705:             "type": "directory" if path.is_dir() else "file",
1706:             "bytes": stat.st_size,
1707:             "modified": datetime.fromtimestamp(stat.st_mtime).isoformat(timespec="seconds"),
1708:             "created": datetime.fromtimestamp(stat.st_ctime).isoformat(timespec="seconds"),
1709:         }

```

Listing 8. Status, traffic, attacks, malware, and run dashboards

This range contains the presentation functions that convert cached and learned state into human-readable dashboards.

```

1713: def show_status(json_output: bool = False) -> None:
1714:     payload = {
1715:         "installation": {
1716:             "version": __version__,
1717:             "runtime_mode": RUNTIME_MODE,
1718:             "home_dir": str(ROOT_DIR),
1719:             "env_override": bool(ENV_ROOT),
1720:         },
1721:         "datasets": summarize_all_datasets(),
1722:         "model": read_json(MODEL_PATH),
1723:         "latest_export": latest_export_summary(),
1724:         "recent_runs": list_run_summaries(),
1725:     }
1726:     cache_artifact("status", payload)
1727:     if json_output:
1728:         print_json(payload)
1729:         return
1730:
1731:     section("IDS Sentinel Terminal Status")
1732:     print(
1733:         f"Version: {payload['installation']['version']} | mode: {payload['installation']['runtime_mode']} | "
1734:         f"home: {payload['installation']['home_dir']}"
1735:     )
1736:     print()
1737:     dataset_rows = []
1738:     for name, item in payload["datasets"].items():
1739:         dataset_rows.append(
1740:             [
1741:                 name,
1742:                 item["path"],
1743:                 item["rows"],
1744:                 f"{item['size_mb']} MB",
1745:                 item["label_counts"].get("0", 0),
1746:                 item["label_counts"].get("1", 0),
1747:                 percent(item["label_counts"].get("1", 0), item["rows"]),
1748:             ]
1749:         )
1750:     print(table(["Set", "Path", "Rows", "Size", "Normal", "Attack", "Attack Share"], dataset_rows))
1751:
1752:     section("Self-Learning Model")
1753:     model = payload["model"]
1754:     if not model:
1755:         print("No model yet. Run: learn")
1756:     else:
1757:         print(f"Model: {model['model_type']} | rows learned: {model['total_rows']:,} | created: {model['created_at']}")

```

```

1758:         print(table(["Indicator", "Separation", "Normal Mean", "Attack Mean"], [
1759:             [item["feature"], item["separation"], item["normal_mean"], item["attack_mean"]]
1760:             for item in model.get("top_indicators", [])[:6]
1761:         ]))
1762:
1763:     latest = payload["latest_export"]
1764:     section("Latest Downloadable Analysis")
1765:     if latest:
1766:         print(f"CSV: {latest['export_csv']}")
1767:         print(f"JSON: {latest['export_json']}")
1768:         print(f"Rows: {latest['rows_analyzed']:,} | average risk: {latest['average_risk_score']:.4f}")
1769:     else:
1770:         print("No analysis export yet. Run: scan")
1771:
1772:
1773: def show_traffic(json_output: bool = False) -> None:
1774:     payload = summarize_all_datasets()
1775:     cache_artifact("traffic", payload)
1776:     if json_output:
1777:         print_json(payload)
1778:         return
1779:
1780:     section("Traffic Data")
1781:     rows = []
1782:     for name, item in payload.items():
1783:         rows.append(
1784:             [
1785:                 name,
1786:                 item["rows"],
1787:                 f"{item['size_mb']} MB",
1788:                 item["total_src_bytes"],
1789:                 item["total_dst_bytes"],
1790:                 ", ".join(f"{value}:{count}" for value, count in item["top_protocols"][:3]),
1791:             ]
1792:         )
1793:     print(table(["Set", "Rows", "Size", "Source Bytes", "Dest Bytes", "Top Encoded Protocols"], rows))
1794:
1795:     section("Top Encoded Services And Flags")
1796:     rows = []
1797:     for name, item in payload.items():
1798:         rows.append(
1799:             [
1800:                 name,
1801:                 ", ".join(f"{value}:{count}" for value, count in item["top_services"][:5]),
1802:                 ", ".join(f"{value}:{count}" for value, count in item["top_flags"][:5]),
1803:             ]
1804:         )
1805:     print(table(["Set", "Services", "Flags"], rows))
1806:     print("\nProtocol, service, and flag values are encoded IDs in these CSV files.")
1807:
1808:
1809: def show_attacks(json_output: bool = False) -> None:
1810:     datasets = summarize_all_datasets()
1811:     model = read_json(MODEL_PATH)
1812:     payload = {"datasets": datasets, "model_indicators": model.get("top_indicators", []) if model else []}
1813:     cache_artifact("attacks", payload)
1814:     if json_output:
1815:         print_json(payload)
1816:         return
1817:
1818:     section("Attack Distribution")
1819:     rows = []
1820:     for name, item in datasets.items():
1821:         total = item["rows"]
1822:         rows.append([name, "normal", item["label_counts"].get("0", 0), percent(item["label_counts"].get("0", 0), total)])
1823:         rows.append([name, "attack", item["label_counts"].get("1", 0), percent(item["label_counts"].get("1", 0), total)])
1824:     print(table(["Set", "Label", "Rows", "Share"], rows))
1825:
1826:     section("Learned Attack Indicators")
1827:     if not model:
1828:         print("No learned model yet. Run: learn")
1829:         return
1830:     print(table(["Feature", "Separation", "Normal Mean", "Attack Mean"], [
1831:         [item["feature"], item["separation"], item["normal_mean"], item["attack_mean"]]
1832:         for item in model.get("top_indicators", [])[:10]
1833:     ]))

```

```

1834:
1835:
1836: def show_malware(json_output: bool = False, limit: int = 5000) -> None:
1837:     model = load_or_learn_model()
1838:     summary = analyze_csv(TEST_CSV, limit=limit, export=False, model=model)
1839:     malware_like = summary["family_counts"].get("malware_like_activity", 0)
1840:     privilege = summary["family_counts"].get("privilege_escalation", 0)
1841:     payload = {
1842:         "note": "The bundled CSVs have binary normal/attack labels, not named malware-family labels. These are behavior",
1843:         "rows_analyzed": summary["rows_analyzed"],
1844:         "malware_like_activity": malware_like,
1845:         "privilege_escalation": privilege,
1846:         "family_counts": summary["family_counts"],
1847:     }
1848:     cache_artifact("malware", payload)
1849:     if json_output:
1850:         print_json(payload)
1851:         return
1852:
1853:     section("Malware-Like Behavior")
1854:     print(payload["note"])
1855:     print(table(["Indicator", "Rows", "Share"], [
1856:         ["malware_like_activity", malware_like, percent(malware_like, summary["rows_analyzed"])],
1857:         ["privilege_escalation", privilege, percent(privilege, summary["rows_analyzed"])],
1858:     ]))
1859:     print(table(["Family", "Rows"], [[name, count] for name, count in sorted(summary["family_counts"].items())]))
1860:
1861:
1862: def show_learn(model: dict[str, Any], json_output: bool = False) -> None:
1863:     if json_output:
1864:         print_json(model)
1865:         return
1866:     section("Self-Learning Complete")
1867:     print(f"Model: {relative_path(MODEL_PATH)}")
1868:     print(f"Rows learned: {model['total_rows']:,}")
1869:     print(f"Created: {model['created_at']}")
1870:     print(table(["Source", "Rows", "Labels"], [
1871:         [item["path"], item["rows_used"], ", ".join(f"{label}:{count}" for label, count in item["label_counts"].items())]
1872:         for item in model["sources"]
1873:     ]))
1874:     print()
1875:     print(table(["Top Indicator", "Separation", "Normal Mean", "Attack Mean"], [
1876:         [item["feature"], item["separation"], item["normal_mean"], item["attack_mean"]]
1877:         for item in model["top_indicators"][:8]
1878:     ]))
1879:
1880:
1881: def show_scan(summary: dict[str, Any], json_output: bool = False) -> None:
1882:     if json_output:
1883:         print_json(summary)
1884:         return
1885:     section("Traffic Analysis")
1886:     print(f"Source: {summary['source_file']}")
1887:     print(f"Rows analyzed: {summary['rows_analyzed']:,}")
1888:     print(f"Average risk: {summary['average_risk_score']:.4f}")
1889:     print(table(["Prediction", "Rows", "Share"], [
1890:         [BINARY_LABELS.get(label, label), count, percent(count, summary["rows_analyzed"])]
1891:         for label, count in sorted(summary["predicted_counts"].items())
1892:     ]))
1893:     print(table(["Risk", "Rows"], [[name, count] for name, count in sorted(summary["risk_counts"].items())]))
1894:     print(table(["Family", "Rows"], [[name, count] for name, count in sorted(summary["family_counts"].items())]))
1895:     print()
1896:     print(table(["Accuracy", "Precision", "Recall", "F1", "TP", "TN", "FP", "FN"], [[
1897:         summary["metrics"]["accuracy"],
1898:         summary["metrics"]["precision"],
1899:         summary["metrics"]["recall"],
1900:         summary["metrics"]["f1"],
1901:         summary["metrics"]["tp"],
1902:         summary["metrics"]["tn"],
1903:         summary["metrics"]["fp"],
1904:         summary["metrics"]["fn"],
1905:     ]]))
1906:     if summary.get("export_csv"):
1907:         print()
1908:         print(f"Downloadable CSV: {summary['export_csv']}")
1909:         print(f"Summary JSON: {summary['export_json']}")

```

```

1910:
1911:
1912: def show_reports(json_output: bool = False, limit: int | None = 20) -> None:
1913:     reports = list_reports(limit)
1914:     cache_artifact("reports", reports)
1915:     if json_output:
1916:         print_json(reports)
1917:         return
1918:     section("Downloadable Reports")
1919:     if not reports:
1920:         print("No product reports yet. Run: scan")
1921:         return
1922:     print(table(["Name", "Path", "Size KB", "Modified"], [
1923:         [item["name"], item["path"], item["size_kb"], item["modified"]] for item in reports
1924:     ]))
1925:
1926:
1927: def show_runs(json_output: bool = False, limit: int | None = 10) -> None:
1928:     runs = list_run_summaries(limit)

```

Listing 9. Command dispatch, parser construction, and entrypoint handling

The closing block reflects the product's current architectural trade-off: fast feature growth through a large monolithic dispatcher.

```

2023: def run_shell_command(raw: str) -> bool:
2024:     SHELL_STATE["history"].append(raw)
2025:     parts = split_shell_command(raw)
2026:     if not parts:
2027:         return False
2028:     command, *args = parts
2029:     command = command.lower()
2030:
2031:     if command in {"exit", "quit", "q"}:
2032:         return True
2033:     if command == "help":
2034:         print_shell_help()
2035:     elif command == "clear":
2036:         os.system("cls" if os.name == "nt" else "clear")
2037:     elif command == "history":
2038:         print(table(["#", "Command"], [[index + 1, value] for index, value in enumerate(SHELL_STATE["history"][-50:])]))
2039:     elif command == "pwd":
2040:         print(relative_path(Path(SHELL_STATE.get("cwd", ROOT_DIR))) or ".")
2041:     elif command == "cd":
2042:         shell_cd(args[0] if args else ".")
2043:     elif command == "ls":
2044:         all_files = "-a" in args
2045:         path_args = [arg for arg in args if arg != "-a"]
2046:         shell_ls(path_args[0] if path_args else ".", all_files=all_files)
2047:     elif command == "cat":
2048:         if not args:
2049:             print("usage: cat <file>")
2050:         else:
2051:             shell_cat(args[0])
2052:     elif command == "head":
2053:         if not args:
2054:             print("usage: head <file> [lines]")
2055:         else:
2056:             shell_head(args[0], int(args[1]) if len(args) > 1 else 20)
2057:     elif command == "tail":
2058:         if not args:
2059:             print("usage: tail <file> [lines]")
2060:         else:
2061:             shell_tail(args[0], int(args[1]) if len(args) > 1 else 20)
2062:     elif command == "grep":
2063:         if not args:
2064:             print("usage: grep <pattern> [path] [limit]")
2065:         else:
2066:             shell_grep(args[0], args[1] if len(args) > 1 else ".", int(args[2]) if len(args) > 2 else 50)
2067:     elif command == "find":
2068:         shell_find(args[0] if args else "*", args[1] if len(args) > 1 else ".", int(args[2]) if len(args) > 2 else 200)
2069:     elif command == "wc":
2070:         if not args:
2071:             print("usage: wc <file>")
2072:         else:

```

```

2073:         shell_wc(args[0])
2074:     elif command == "du":
2075:         shell_du(args[0] if args else ".")
2076:     elif command == "stat":
2077:         if not args:
2078:             print("usage: stat <path>")
2079:         else:
2080:             shell_stat(args[0])
2081:     elif command in {"status", "overview", "dashboard"}:
2082:         show_status()
2083:     elif command in {"traffic", "data"}:
2084:         show_traffic()
2085:     elif command in {"attack", "attacks"}:
2086:         show_attacks()
2087:     elif command in {"malware", "malwares"}:
2088:         show_malware(limit=parse_limit(args[0], 5000) if args else 5000)
2089:     elif command == "learn":
2090:         mode = args[0].lower() if args else "full"
2091:         limit = 20000 if mode == "quick" else None
2092:         show_learn(learn_model(limit=limit, include_generated=True))
2093:     elif command in {"scan", "analyze"}:
2094:         path = resolve_readable_path(args[0] if args else None, default=TEST_CSV, base=Path(SHELL_STATE.get("cwd", ROOT_
2095:         limit = parse_limit(args[1], 5000) if len(args) > 1 else 5000
2096:         show_scan(analyze_csv(path, limit=limit, export=True))
2097:     elif command == "export":
2098:         path = resolve_readable_path(args[0] if args else None, default=TEST_CSV, base=Path(SHELL_STATE.get("cwd", ROOT_
2099:         limit = parse_limit(args[1], None) if len(args) > 1 else None
2100:         show_scan(analyze_csv(path, limit=limit, export=True))
2101:     elif command in {"datasets", "catalog"}:
2102:         show_dataset_catalog()
2103:     elif command == "import":
2104:         if not args:
2105:             print("usage: import <csv-path> [name]")
2106:         else:
2107:             show_import(import_csv(resolve_readable_path(args[0], base=Path(SHELL_STATE.get("cwd", ROOT_DIR))), args[1]
2108:     elif command == "download":
2109:         if not args:
2110:             print("usage: download <url> [name]")
2111:         else:
2112:             downloaded = download_url(args[0], args[1] if len(args) > 1 else None)
2113:             print(f"Downloaded: {relative_path(downloaded)}")
2114:     elif command == "index":
2115:         path = resolve_readable_path(args[0] if args else None, default=TEST_CSV, base=Path(SHELL_STATE.get("cwd", ROOT_
2116:         limit = parse_limit(args[1], 50000) if len(args) > 1 else 50000
2117:         show_index(path, limit=limit)
2118:     elif command == "hunt":
2119:         if not args:
2120:             print("usage: hunt <term> [path] [limit]")
2121:         else:
2122:             show_hunt(args[0], resolve_readable_path(args[1], base=Path(SHELL_STATE.get("cwd", ROOT_DIR))) if len(args)
2123:     elif command == "ioc":
2124:         show_ioc(args)
2125:     elif command in {"ports", "listeners"}:
2126:         show_netstat(only_listening=True, limit=parse_limit(args[0], 40) if args else 40)
2127:     elif command in {"netstat", "connections"}:
2128:         show_netstat(only_listening=False, limit=parse_limit(args[0], 40) if args else 40)
2129:     elif command == "port":
2130:         if not args:
2131:             print("usage: port <number>")
2132:         else:
2133:             show_port(int(args[0]))
2134:     elif command == "probe":
2135:         if len(args) < 2:
2136:             print("usage: probe <host> <ports>")
2137:         else:
2138:             show_probe(args[0], args[1])
2139:     elif command == "dns":
2140:         if not args:
2141:             print("usage: dns <host>")
2142:         else:
2143:             show_dns(args[0])
2144:     elif command == "ps":
2145:         show_processes(limit=parse_limit(args[0], 40) if args else 40)
2146:     elif command == "hash":
2147:         if not args:
2148:             print("usage: hash <file>")

```

```

2149:         else:
2150:             show_hash(resolve_readable_path(args[0], base=Path(SHELL_STATE.get("cwd", ROOT_DIR))), json_output=False)
2151:     elif command in {"filescan", "scanfile"}:
2152:         if not args:
2153:             print("usage: filescan <file>")
2154:         else:
2155:             show_file_scan(resolve_readable_path(args[0], base=Path(SHELL_STATE.get("cwd", ROOT_DIR))))
2156:     elif command in {"reports", "downloads"}:
2157:         show_reports(limit=parse_limit(args[0], 20) if args else 20)
2158:     elif command == "cache":
2159:         show_cache(limit=parse_limit(args[0], 40) if args else 40)
2160:     elif command == "runs":
2161:         show_runs(limit=parse_limit(args[0], 10) if args else 10)
2162:     else:
2163:         print(f"Unknown command: {command}. Type 'help'.")
2164:     return False
2165:
2166:
2167: def build_parser() -> argparse.ArgumentParser:
2168:     parser = argparse.ArgumentParser(description="IDS Sentinel Terminal for defensive CSV traffic analysis and local triage")
2169:     parser.add_argument("--json", action="store_true", help="Print JSON for commands that support it.")
2170:     parser.add_argument("--version", action="version", version=f"IDS Sentinel Terminal {__version__}")
2171:     subparsers = parser.add_subparsers(dest="command")
2172:
2173:     subparsers.add_parser("shell", help="Open IDS Sentinel Terminal interactive mode.")
2174:     subparsers.add_parser("gui", help="Open the graphical product console.")
2175:     subparsers.add_parser("status", help="Show product status.")
2176:     subparsers.add_parser("traffic", help="Show traffic data.")
2177:     subparsers.add_parser("attacks", help="Show attacks and learned indicators.")
2178:     subparsers.add_parser("datasets", help="Show local and external IDS dataset catalog.")
2179:
2180:     malware_parser = subparsers.add_parser("malware", help="Show malware-like behavior indicators.")
2181:     malware_parser.add_argument("--limit", type=int, default=5000)
2182:
2183:     learn_parser = subparsers.add_parser("learn", help="Build/update the self-learning model.")
2184:     learn_parser.add_argument("--quick", action="store_true", help="Use a 20,000-row sample instead of all rows.")
2185:     learn_parser.add_argument("--full", action="store_true", help="Use all source rows. This is the default.")
2186:     learn_parser.add_argument("--include-test", action="store_true", help="Also learn from kddtest.csv labels.")
2187:     learn_parser.add_argument("--skip-generated", action="store_true", help="Do not learn from terminal-generated CSV exports.")
2188:
2189:     scan_parser = subparsers.add_parser("scan", help="Analyze a CSV and export CSV/JSON results.")
2190:     scan_parser.add_argument("path", nargs="?", default=None)
2191:     scan_parser.add_argument("--limit", type=int, default=5000)
2192:     scan_parser.add_argument("--all", action="store_true", help="Analyze all rows.")
2193:     scan_parser.add_argument("--no-export", action="store_true", help="Only print summary; do not write downloadable file.")
2194:
2195:     export_parser = subparsers.add_parser("export", help="Analyze and export all rows by default.")
2196:     export_parser.add_argument("path", nargs="?", default=None)
2197:     export_parser.add_argument("--limit", type=int)
2198:
2199:     import_parser = subparsers.add_parser("import", help="Copy a CSV into product imports and index it.")
2200:     import_parser.add_argument("path")
2201:     import_parser.add_argument("--name")
2202:
2203:     download_parser = subparsers.add_parser("download", help="Download a public URL into product imports.")
2204:     download_parser.add_argument("url")
2205:     download_parser.add_argument("--name")
2206:
2207:     index_parser = subparsers.add_parser("index", help="Inspect a CSV file.")
2208:     index_parser.add_argument("path", nargs="?", default=None)
2209:     index_parser.add_argument("--limit", type=int, default=5000)
2210:     index_parser.add_argument("--all", action="store_true")
2211:
2212:     hunt_parser = subparsers.add_parser("hunt", help="Search datasets, imports, and reports for text.")
2213:     hunt_parser.add_argument("pattern")
2214:     hunt_parser.add_argument("--path")
2215:     hunt_parser.add_argument("--limit", type=int, default=50)
2216:
2217:     ioc_parser = subparsers.add_parser("ioc", help="Manage and hunt indicators of compromise.")
2218:     ioc_parser.add_argument("ioc_args", nargs="*")
2219:
2220:     netstat_parser = subparsers.add_parser("netstat", help="Show local network connections.")
2221:     netstat_parser.add_argument("--limit", type=int, default=40)
2222:     netstat_parser.add_argument("--listening", action="store_true")
2223:
2224:     ports_parser = subparsers.add_parser("ports", help="Show local listening ports.")

```

```

2225:     ports_parser.add_argument("--limit", type=int, default=40)
2226:
2227:     port_parser = subparsers.add_parser("port", help="Explain a port and show local matches.")
2228:     port_parser.add_argument("number", type=int)
2229:
2230:     probe_parser = subparsers.add_parser("probe", help="Authorized TCP connect probe.")
2231:     probe_parser.add_argument("host")
2232:     probe_parser.add_argument("ports")
2233:
2234:     dns_parser = subparsers.add_parser("dns", help="Resolve a host.")
2235:     dns_parser.add_argument("host")
2236:
2237:     ps_parser = subparsers.add_parser("ps", help="List local processes.")
2238:     ps_parser.add_argument("--limit", type=int, default=40)
2239:
2240:     hash_parser = subparsers.add_parser("hash", help="Hash a file.")
2241:     hash_parser.add_argument("path")
2242:
2243:     filescan_parser = subparsers.add_parser("filescan", help="Hash and triage a file.")
2244:     filescan_parser.add_argument("path")
2245:
2246:     reports_parser = subparsers.add_parser("reports", help="List generated downloadable reports.")
2247:     reports_parser.add_argument("--limit", type=int, default=20)
2248:
2249:     runs_parser = subparsers.add_parser("runs", help="List existing ML training runs.")
2250:     runs_parser.add_argument("--limit", type=int, default=10)
2251:
2252:     cache_parser = subparsers.add_parser("cache", help="List cached command artifacts.")
2253:     cache_parser.add_argument("--limit", type=int, default=40)
2254:     return parser
2255:
2256:
2257: def normalize_global_args(argv: list[str] | None) -> list[str] | None:
2258:     if argv is None:
2259:         return None
2260:     normalized = list(argv)
2261:     if "--json" in normalized:
2262:         normalized = [value for value in normalized if value != "--json"]
2263:         normalized.insert(0, "--json")
2264:     return normalized
2265:
2266:
2267: def main(argv: list[str] | None = None) -> int:
2268:     ensure_product_dirs()
2269:     parser = build_parser()
2270:     args = parser.parse_args(normalize_global_args(argv))
2271:
2272:     try:
2273:         if args.command is None or args.command == "shell":
2274:             command_shell()
2275:         elif args.command == "gui":
2276:             from .product_gui import main as gui_main
2277:
2278:             return gui_main([])
2279:         elif args.command == "status":
2280:             show_status(args.json)
2281:         elif args.command == "traffic":
2282:             show_traffic(args.json)
2283:         elif args.command == "attacks":
2284:             show_attacks(args.json)
2285:         elif args.command == "datasets":
2286:             show_dataset_catalog(args.json)
2287:         elif args.command == "malware":
2288:             show_malware(args.json, args.limit)
2289:         elif args.command == "learn":
2290:             model = learn_model(
2291:                 limit=20000 if args.quick else None,
2292:                 include_generated=not args.skip_generated,
2293:                 include_test=args.include_test,
2294:             )
2295:             show_learn(model, args.json)
2296:         elif args.command == "scan":
2297:             source = resolve_readable_path(args.path, default=TEST_CSV)
2298:             summary = analyze_csv(source, limit=None if args.all else args.limit, export=not args.no_export)
2299:             show_scan(summary, args.json)
2300:         elif args.command == "export":

```

```

2301:         source = resolve_readable_path(args.path, default=TEST_CSV)
2302:         summary = analyze_csv(source, limit=args.limit, export=True)
2303:         show_scan(summary, args.json)
2304:     elif args.command == "import":
2305:         show_import(import_csv(resolve_readable_path(args.path), args.name), args.json)
2306:     elif args.command == "download":
2307:         downloaded = download_url(args.url, args.name)
2308:         payload = {"downloaded": relative_path(downloaded)}
2309:         print_json(payload) if args.json else print(f"Downloaded: {payload['downloaded']}")
2310:     elif args.command == "index":
2311:         show_index(resolve_readable_path(args.path, default=TEST_CSV), args.json, limit=None if args.all else args.limit)
2312:     elif args.command == "hunt":
2313:         show_hunt(args.pattern, resolve_readable_path(args.path) if args.path else None, args.json, args.limit)
2314:     elif args.command == "ioc":
2315:         show_ioc(args.ioc_args, args.json)
2316:     elif args.command == "netstat":
2317:         show_netstat(args.json, only_listening=args.listening, limit=args.limit)
2318:     elif args.command == "ports":
2319:         show_netstat(args.json, only_listening=True, limit=args.limit)
2320:     elif args.command == "port":
2321:         show_port(args.number, args.json)
2322:     elif args.command == "probe":
2323:         show_probe(args.host, args.ports, args.json)
2324:     elif args.command == "dns":
2325:         show_dns(args.host, args.json)
2326:     elif args.command == "ps":
2327:         show_processes(args.json, args.limit)
2328:     elif args.command == "hash":
2329:         show_hash(resolve_readable_path(args.path), args.json)
2330:     elif args.command == "filescan":
2331:         show_file_scan(resolve_readable_path(args.path), args.json)
2332:     elif args.command == "reports":
2333:         show_reports(args.json, args.limit)
2334:     elif args.command == "runs":
2335:         show_runs(args.json, args.limit)
2336:     elif args.command == "cache":
2337:         show_cache(args.json, args.limit)
2338:     else:
2339:         parser.error(f"Unknown command: {args.command}")
2340: except KeyboardInterrupt:
2341:     print("\nInterrupted.")
2342:     return 130
2343: except Exception as exc:
2344:     print(f"error: {exc}", file=sys.stderr)
2345:     return 1
2346: return 0
2347:
2348:
2349: if __name__ == "__main__":
2350:     raise SystemExit(main(sys.argv[1:]))

```

Listing 10. GUI theme, layout, and dark-mode composition

The GUI is not a wrapper around a browser; it is a native Tk interface with a custom palette, pane layout, themed notebook tabs, and a command-driven workflow.

```

1: from __future__ import annotations
2:
3: import contextlib
4: import io
5: import queue
6: import threading
7: import tkinter as tk
8: from pathlib import Path
9: from tkinter import filedialog, messagebox, ttk
10:
11: from . import product_terminal
12:
13:
14: class IDSProductGUI:
15:     def __init__(self, root: tk.Tk) -> None:
16:         self.root = root
17:         self.root.title("IDS Sentinel Terminal")
18:         self.root.geometry("1180x760")

```

```

19:         self.root.minsize(920, 620)
20:         self.output_queue: queue.Queue[str] = queue.Queue()
21:         self.palette = {
22:             "bg": "#0c1222",
23:             "sidebar": "#101a31",
24:             "panel": "#151f38",
25:             "panel_alt": "#1a2747",
26:             "text": "#e6edf9",
27:             "muted": "#9eaeed",
28:             "accent": "#2f81f7",
29:             "accent_hover": "#4d95ff",
30:             "accent_pressed": "#1e6fe3",
31:             "button": "#1f2d4f",
32:             "button_hover": "#2b3d67",
33:             "button_pressed": "#1a2948",
34:             "border": "#2d3f66",
35:             "output_bg": "#0d162e",
36:             "output_text": "#dde7fd",
37:             "success": "#56d39f",
38:             "warning": "#f2c26b",
39:             "danger": "#ff7b72",
40:         }
41:
42:         self.command_var = tk.StringVar(value="status")
43:         self.scan_path_var = tk.StringVar(value="kddtest.csv")
44:         self.scan_limit_var = tk.StringVar(value="5000")
45:         self.hunt_var = tk.StringVar(value="dos_flood")
46:         self.host_var = tk.StringVar(value="127.0.0.1")
47:         self.ports_var = tk.StringVar(value="common")
48:         self.file_path_var = tk.StringVar(value="automation/product/self_learning_model.json")
49:
50:         self._configure_theme()
51:         self._build_layout()
52:         self.root.after(100, self._drain_output)
53:         self.run_command(["status"])
54:
55:     def _configure_theme(self) -> None:
56:         style = ttk.Style(self.root)
57:         with contextlib.suppress(tk.TclError):
58:             style.theme_use("clam")
59:
60:         p = self.palette
61:         self.root.configure(bg=p["bg"])
62:         style.configure(".", background=p["bg"], foreground=p["text"], font=("Segoe UI", 10))
63:         style.configure("Sidebar.TFrame", background=p["sidebar"])
64:         style.configure("Main.TFrame", background=p["bg"])
65:         style.configure("Tab.TFrame", background=p["panel"])
66:
67:         style.configure("SidebarTitle.TLabel", background=p["sidebar"], foreground=p["text"], font=("Segoe UI Semibold",
68:         style.configure("SidebarSub.TLabel", background=p["sidebar"], foreground=p["muted"], font=("Segoe UI", 9))
69:         style.configure("Sidebar.TLabel", background=p["sidebar"], foreground=p["text"])
70:         style.configure("Panel.TLabel", background=p["panel"], foreground=p["text"])
71:         style.configure("TSeparator", background=p["border"])
72:
73:         style.configure("Sidebar.TButton", padding=(12, 7), background=p["button"], foreground=p["text"], borderwidth=0,
74:         style.map(
75:             "Sidebar.TButton",
76:             background=[("pressed", p["button_pressed"]), ("active", p["button_hover"])],
77:             foreground=[("disabled", p["muted"])],
78:         )
79:
80:         style.configure("Primary.TButton", padding=(12, 7), background=p["accent"], foreground=p["text"], borderwidth=0,
81:         style.map(
82:             "Primary.TButton",
83:             background=[("pressed", p["accent_pressed"]), ("active", p["accent_hover"])],
84:             foreground=[("disabled", p["muted"])],
85:         )
86:
87:         style.configure("Tool.TButton", padding=(10, 6), background=p["button"], foreground=p["text"], borderwidth=0, rel
88:         style.map(
89:             "Tool.TButton",
90:             background=[("pressed", p["button_pressed"]), ("active", p["button_hover"])],
91:             foreground=[("disabled", p["muted"])],
92:         )
93:
94:         style.configure(

```

```

95:         "TEntry",
96:         fieldbackground=p["panel_alt"],
97:         foreground=p["text"],
98:         bordercolor=p["border"],
99:         lightcolor=p["border"],
100:        darkcolor=p["border"],
101:        insertcolor=p["text"],
102:        padding=6,
103:    )
104:    style.map("TEntry", fieldbackground=[("readonly", p["panel_alt"])]))
105:
106:    style.configure("App.TNotebook", background=p["bg"], borderwidth=0, tabmargins=(0, 0, 0, 0))
107:    style.configure("TNotebook.Tab", background=p["panel"], foreground=p["muted"], padding=(14, 7), borderwidth=0)
108:    style.map(
109:        "TNotebook.Tab",
110:        background=[("selected", p["accent"]), ("active", p["panel_alt"])],
111:        foreground=[("selected", p["text"]), ("active", p["text"])],
112:    )
113:
114:    style.configure(
115:        "Vertical.TScrollbar",
116:        background=p["panel_alt"],
117:        troughcolor=p["panel"],
118:        bordercolor=p["border"],
119:        arrowcolor=p["muted"],
120:    )
121:    style.configure(
122:        "Horizontal.TScrollbar",
123:        background=p["panel_alt"],
124:        troughcolor=p["panel"],
125:        bordercolor=p["border"],
126:        arrowcolor=p["muted"],
127:    )
128:
129:    def _build_layout(self) -> None:
130:        self.root.columnconfigure(0, weight=0)
131:        self.root.columnconfigure(1, weight=1)
132:        self.root.rowconfigure(0, weight=1)
133:
134:        sidebar = ttk.Frame(self.root, padding=14, style="Sidebar.TFrame")
135:        sidebar.grid(row=0, column=0, sticky="ns")
136:        sidebar.configure(width=248)
137:
138:        main = ttk.Frame(self.root, padding=(0, 12, 12, 12), style="Main.TFrame")
139:        main.grid(row=0, column=1, sticky="nsew")
140:        main.columnconfigure(0, weight=1)
141:        main.rowconfigure(1, weight=1)
142:
143:        ttk.Label(sidebar, text="IDS Sentinel Terminal", style="SidebarTitle.TLabel").pack(anchor="w", pady=(4, 2))
144:        ttk.Label(sidebar, text="Traffic and Threat Intelligence", style="SidebarSub.TLabel").pack(anchor="w", pady=(0, 1))
145:        for label, command in [
146:            ("Status", ["status"]),
147:            ("Traffic", ["traffic"]),
148:            ("Attacks", ["attacks"]),
149:            ("Malware Signals", ["malware", "--limit", "5000"]),
150:            ("Datasets", ["datasets"]),
151:            ("Reports", ["reports", "--limit", "20"]),
152:            ("Cache", ["cache", "--limit", "20"]),
153:            ("Local Ports", ["ports", "--limit", "25"]),
154:            ("Processes", ["ps", "--limit", "25"]),
155:        ]:
156:            ttk.Button(sidebar, text=label, style="Sidebar.TButton", command=lambda cmd=command: self.run_command(cmd)).pack(
157:                side="top", fill="both", expand=True,
158:            )
159:            ttk.Separator(sidebar).pack(fill="x", pady=12)
160:            ttk.Button(sidebar, text="Learn Model", style="Primary.TButton", command=lambda: self.run_command(["learn"])).pack(
161:                side="top", fill="both", expand=True,
162:            )
163:            ttk.Button(sidebar, text="Clear Output", style="Sidebar.TButton", command=self.clear_output).pack(fill="x", pady=5)
164:            ttk.Label(sidebar, text="Theme: Dark Ops UI", style="SidebarSub.TLabel").pack(anchor="w", pady=(12, 0))
165:
166:        controls = ttk.Notebook(main, style="App.TNotebook")
167:        controls.grid(row=0, column=0, sticky="ew", pady=(0, 8))
168:
169:        self._build_command_tab(controls)
170:        self._build_scan_tab(controls)
171:        self._build_hunt_tab(controls)
172:        self._build_network_tab(controls)
173:        self._build_file_tab(controls)

```

```

171:
172:     self.output = tk.Text(
173:         main,
174:         wrap="none",
175:         font=("Consolas", 10),
176:         undo=False,
177:         bg=self.palette["output_bg"],
178:         fg=self.palette["output_text"],
179:         insertbackground=self.palette["text"],
180:         selectbackground=self.palette["accent"],
181:         selectforeground=self.palette["text"],
182:         relief="flat",
183:         borderwidth=0,
184:         highlightthickness=1,
185:         highlightbackground=self.palette["border"],
186:         highlightcolor=self.palette["accent"],
187:         padx=10,
188:         pady=10,
189:     )
190:     self.output.grid(row=1, column=0, sticky="nsew")
191:     self.output.tag_configure("command", foreground=self.palette["accent"])
192:     self.output.tag_configure("success", foreground=self.palette["success"])
193:     self.output.tag_configure("warning", foreground=self.palette["warning"])
194:     self.output.tag_configure("danger", foreground=self.palette["danger"])
195:
196:     y_scroll = ttk.Scrollbar(main, orient="vertical", command=self.output.yview, style="Vertical.TScrollbar")
197:     y_scroll.grid(row=1, column=1, sticky="ns")
198:     self.output.configure(yscrollcommand=y_scroll.set)
199:
200:     x_scroll = ttk.Scrollbar(main, orient="horizontal", command=self.output.xview, style="Horizontal.TScrollbar")
201:     x_scroll.grid(row=2, column=0, sticky="ew")
202:     self.output.configure(xscrollcommand=x_scroll.set)
203:
204:     def _build_command_tab(self, notebook: ttk.Notebook) -> None:
205:         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")

```

Listing 11. GUI command execution and output coloring

The second half of the GUI shows how background threads execute terminal commands and stream tagged output back into the text console.

```

206:         tab.columnconfigure(1, weight=1)
207:         notebook.add(tab, text="Command")
208:         ttk.Label(tab, text="Command", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
209:         ttk.Entry(tab, textvariable=self.command_var).grid(row=0, column=1, sticky="ew")
210:         ttk.Button(tab, text="Run", style="Primary.TButton", command=self.run_freeform_command).grid(row=0, column=2, padx=(8, 0))
211:
212:     def _build_scan_tab(self, notebook: ttk.Notebook) -> None:
213:         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
214:         tab.columnconfigure(1, weight=1)
215:         notebook.add(tab, text="Scan")
216:         ttk.Label(tab, text="CSV", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
217:         ttk.Entry(tab, textvariable=self.scan_path_var).grid(row=0, column=1, sticky="ew")
218:         ttk.Button(tab, text="Browse", style="Tool.TButton", command=self.choose_scan_file).grid(row=0, column=2, padx=(8, 0))
219:         ttk.Label(tab, text="Limit", style="Panel.TLabel").grid(row=0, column=3, sticky="w", padx=(12, 8))
220:         ttk.Entry(tab, width=10, textvariable=self.scan_limit_var).grid(row=0, column=4, sticky="w")
221:         ttk.Button(tab, text="Scan", style="Primary.TButton", command=self.run_scan).grid(row=0, column=5, padx=(8, 0))
222:
223:     def _build_hunt_tab(self, notebook: ttk.Notebook) -> None:
224:         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
225:         tab.columnconfigure(1, weight=1)
226:         notebook.add(tab, text="Hunt")
227:         ttk.Label(tab, text="Term", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
228:         ttk.Entry(tab, textvariable=self.hunt_var).grid(row=0, column=1, sticky="ew")
229:         ttk.Button(
230:             tab,
231:             text="Hunt",
232:             style="Primary.TButton",
233:             command=lambda: self.run_command(["hunt", self.hunt_var.get(), "--limit", "20"]),
234:         ).grid(row=0, column=2, padx=(8, 0))
235:
236:     def _build_network_tab(self, notebook: ttk.Notebook) -> None:
237:         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
238:         tab.columnconfigure(1, weight=1)

```

```

239:     notebook.add(tab, text="Network")
240:     ttk.Label(tab, text="Host", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
241:     ttk.Entry(tab, textvariable=self.host_var).grid(row=0, column=1, sticky="ew")
242:     ttk.Label(tab, text="Ports", style="Panel.TLabel").grid(row=0, column=2, sticky="w", padx=(12, 8))
243:     ttk.Entry(tab, width=18, textvariable=self.ports_var).grid(row=0, column=3, sticky="w")
244:     ttk.Button(
245:         tab,
246:         text="Probe",
247:         style="Primary.TButton",
248:         command=lambda: self.run_command(["probe", self.host_var.get(), self.ports_var.get()]),
249:     ).grid(row=0, column=4, padx=(8, 0))
250:     ttk.Button(tab, text="DNS", style="Tool.TButton", command=lambda: self.run_command(["dns", self.host_var.get()]))
251:
252: def _build_file_tab(self, notebook: ttk.Notebook) -> None:
253:     tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
254:     tab.columnconfigure(1, weight=1)
255:     notebook.add(tab, text="File")
256:     ttk.Label(tab, text="File", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
257:     ttk.Entry(tab, textvariable=self.file_path_var).grid(row=0, column=1, sticky="ew")
258:     ttk.Button(tab, text="Browse", style="Tool.TButton", command=self.choose_file).grid(row=0, column=2, padx=(8, 0))
259:     ttk.Button(tab, text="Hash", style="Tool.TButton", command=lambda: self.run_command(["hash", self.file_path_var.get()]))
260:     ttk.Button(tab, text="Scan", style="Primary.TButton", command=lambda: self.run_command(["filescan", self.file_path_var.get()]))
261:
262: def choose_scan_file(self) -> None:
263:     path = filedialog.askopenfilename(title="Choose traffic CSV", filetypes=[("CSV files", "*.csv"), ("All files", "*")])
264:     if path:
265:         self.scan_path_var.set(self._display_path(path))
266:
267: def choose_file(self) -> None:
268:     path = filedialog.askopenfilename(title="Choose file")
269:     if path:
270:         self.file_path_var.set(self._display_path(path))
271:
272: def _display_path(self, path: str) -> str:
273:     try:
274:         return str(Path(path).resolve().relative_to(product_terminal.ROOT_DIR))
275:     except ValueError:
276:         return path
277:
278: def run_freeform_command(self) -> None:
279:     try:
280:         args = product_terminal.split_shell_command(self.command_var.get())
281:     except ValueError as exc:
282:         messagebox.showerror("Command parse error", str(exc))
283:         return
284:     self.run_command(args)
285:
286: def run_scan(self) -> None:
287:     args = ["scan", self.scan_path_var.get()]
288:     limit = self.scan_limit_var.get().strip()
289:     if limit.lower() == "all":
290:         args.append("--all")
291:     elif limit:
292:         args.extend(["--limit", limit])
293:     self.run_command(args)
294:
295: def run_command(self, args: list[str]) -> None:
296:     self._append(f"\n$ ids-sentinel-terminal {' '.join(args)}\n")
297:     thread = threading.Thread(target=self._run_command_worker, args=(args,), daemon=True)
298:     thread.start()
299:
300: def _run_command_worker(self, args: list[str]) -> None:
301:     stream = io.StringIO()
302:     with contextlib.redirect_stdout(stream), contextlib.redirect_stderr(stream):
303:         code = product_terminal.main(args)
304:         text = stream.getvalue()
305:         if code:
306:             text += f"\nCommand exited with code {code}\n"
307:         self.output_queue.put(text)
308:
309: def _drain_output(self) -> None:
310:     try:
311:         while True:
312:             self._append(self.output_queue.get_nowait())
313:     except queue.Empty:
314:         pass

```

```

315:         self.root.after(100, self._drain_output)
316:
317:     def _append(self, text: str) -> None:
318:         for line in text.splitlines(keepends=True):
319:             tag = None
320:             normalized = line.lower()
321:             if line.lstrip().startswith("$ ids-sentinel-terminal"):
322:                 tag = "command"
323:             elif any(token in normalized for token in ("error", "exception", "traceback", "not recognized", "failed", "ex
324:                 tag = "danger"
325:             elif any(token in normalized for token in ("critical", "high", "warning", "suspicious")):
326:                 tag = "warning"
327:             elif any(token in normalized for token in ("healthy", "no threats", "completed", "success")):
328:                 tag = "success"
329:             if tag:
330:                 self.output.insert("end", line, tag)
331:             else:
332:                 self.output.insert("end", line)
333:             self.output.see("end")
334:
335:     def clear_output(self) -> None:
336:         self.output.delete("1.0", "end")
337:
338:
339: def main(argv: list[str] | None = None) -> int:
340:     del argv
341:     root = tk.Tk()
342:     IDSPProductGUI(root)
343:     root.mainloop()
344:     return 0
345:
346:
347: if __name__ == "__main__":
348:     raise SystemExit(main())

```

Listing 12. Classical training suite

This file packages a compact benchmark set of supervised classifiers and records structured results into `automation/runs`.

```

1: from __future__ import annotations
2:
3: import time
4: from datetime import datetime, timezone
5: from pathlib import Path
6: from uuid import uuid4
7:
8: import joblib
9: import numpy as np
10: from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier
11: from sklearn.linear_model import LogisticRegression
12: from sklearn.naive_bayes import GaussianNB
13: from sklearn.neighbors import KNeighborsClassifier
14: from sklearn.tree import DecisionTreeClassifier
15:
16: from .config import CLASSICAL_MODEL_LABELS, CLASSICAL_PROFILES
17: from .data import load_classical_split
18: from .metrics import binary_metrics, probability_summary
19: from .storage import run_dir, run_summary_path, write_json
20:
21:
22: MODEL_BUILDERS = {
23:     "logistic_regression": lambda: LogisticRegression(max_iter=400, solver="lbfgs"),
24:     "naive_bayes": lambda: GaussianNB(),
25:     "decision_tree": lambda: DecisionTreeClassifier(random_state=42),
26:     "adaboost": lambda: AdaBoostClassifier(n_estimators=100, random_state=42, algorithm="SAMME"),
27:     "random_forest": lambda: RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42),
28:     "knn": lambda: KNeighborsClassifier(n_neighbors=5, weights="distance", n_jobs=-1),
29: }
30:
31:
32: def resolve_classical_config(config: dict[str, object] | None) -> dict[str, object]:
33:     payload = dict(config or {})
34:     profile = str(payload.get("profile", "fast"))
35:     if profile not in CLASSICAL_PROFILES:

```

```

36:     profile = "fast"
37:     merged = dict(CLASSICAL_PROFILES[profile])
38:     merged["profile"] = profile
39:     if payload.get("models"):
40:         selected = [name for name in payload["models"] if name in MODEL_BUILDERS]
41:         if selected:
42:             merged["models"] = selected
43:     for key in ("train_sample", "test_sample", "random_state"):
44:         if key in payload:
45:             merged[key] = payload.get(key)
46:     merged.setdefault("random_state", 42)
47:     return merged
48:
49:
50: def train_classical_suite(job_id: str, config: dict[str, object] | None = None) -> dict[str, object]:
51:     resolved = resolve_classical_config(config)
52:     split = load_classical_split(
53:         train_sample=resolved.get("train_sample"),
54:         test_sample=resolved.get("test_sample"),
55:         random_state=int(resolved.get("random_state", 42)),
56:     )
57:
58:     generated_run_id = f"classical-{uuid4().hex[:12]}"
59:     target_dir = run_dir(generated_run_id)
60:     models_dir = target_dir / "models"
61:     predictions_dir = target_dir / "predictions"
62:     models_dir.mkdir(parents=True, exist_ok=True)
63:     predictions_dir.mkdir(parents=True, exist_ok=True)
64:
65:     y_test = split["y_test"]
66:     results = []
67:
68:     for model_name in resolved["models"]:
69:         builder = MODEL_BUILDERS[model_name]
70:         estimator = builder()
71:         started = time.perf_counter()
72:         estimator.fit(split["X_train"], split["y_train"])
73:         training_seconds = round(time.perf_counter() - started, 3)
74:
75:         predicted = estimator.predict(split["X_test"]).astype(int)
76:         probabilities = None
77:         if hasattr(estimator, "predict_proba"):
78:             probabilities = estimator.predict_proba(split["X_test"])[ :, 1]
79:
80:         metrics = binary_metrics(y_test, predicted)
81:         joblib.dump(estimator, models_dir / f"{model_name}.joblib")
82:         np.savetxt(predictions_dir / f"{model_name}_labels.txt", predicted, fmt="%01d")
83:         if probabilities is not None:
84:             np.savetxt(predictions_dir / f"{model_name}_probabilities.txt", probabilities)
85:
86:         results.append(
87:             {
88:                 "id": model_name,
89:                 "label": CLASSICAL_MODEL_LABELS[model_name],
90:                 "training_seconds": training_seconds,
91:                 "metrics": metrics,
92:                 "probability_summary": probability_summary(probabilities),
93:                 "model_path": str((models_dir / f"{model_name}.joblib").relative_to(target_dir.parent.parent)),
94:             }
95:         )
96:
97:     results.sort(key=lambda item: item["metrics"]["f1"], reverse=True)
98:     summary = {
99:         "run_id": generated_run_id,
100:         "job_id": job_id,
101:         "kind": "classical_train",
102:         "created_at": datetime.now(timezone.utc).isoformat(),
103:         "config": resolved,
104:         "dataset": {
105:             "train_rows": split["train_rows"],
106:             "test_rows": split["test_rows"],
107:             "feature_count": split["feature_count"],
108:         },
109:         "results": results,
110:         "best_model": results[0]["id"] if results else None,
111:     }

```

```

112:     write_json(run_summary_path(generated_run_id), summary)
113:     return summary

```

Listing 13. DNN training suite

The DNN path is deliberately thinner than the classical suite, deferring the heavy dependency load to TensorFlow while preserving a consistent summary schema.

```

1: from __future__ import annotations
2:
3: import os
4: import time
5: from datetime import datetime, timezone
6: from uuid import uuid4
7:
8: import numpy as np
9:
10: from .config import DNN_LAYER_SPECS, DNN_PROFILES
11: from .data import load_dnn_split
12: from .metrics import binary_metrics, probability_summary
13: from .storage import run_dir, run_summary_path, write_json
14:
15:
16: def _tensorflow():
17:     os.environ.setdefault("CUDA_VISIBLE_DEVICES", "-1")
18:     os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "3")
19:     import tensorflow as tf
20:
21:     return tf
22:
23:
24: def resolve_dnn_config(config: dict[str, object] | None) -> dict[str, object]:
25:     payload = dict(config or {})
26:     profile = str(payload.get("profile", "fast"))
27:     if profile not in DNN_PROFILES:
28:         profile = "fast"
29:     merged = dict(DNN_PROFILES[profile])
30:     merged["profile"] = profile
31:     if payload.get("architectures"):
32:         selected = [int(value) for value in payload["architectures"] if int(value) in DNN_LAYER_SPECS]
33:         if selected:
34:             merged["architectures"] = selected
35:     for key in ("train_sample", "test_sample", "epochs", "batch_size", "random_state"):
36:         if key in payload:
37:             merged[key] = payload.get(key)
38:     merged.setdefault("random_state", 42)
39:     return merged
40:
41:
42: def _build_model(layer_count: int, dropout_rate: float = 0.01):
43:     tf = _tensorflow()
44:
45:     tf.keras.backend.clear_session()
46:     model = tf.keras.Sequential()
47:     for index, units in enumerate(DNN_LAYER_SPECS[layer_count]):
48:         kwargs = {"input_dim": 41} if index == 0 else {}
49:         model.add(tf.keras.layers.Dense(units, activation="relu", **kwargs))
50:         model.add(tf.keras.layers.Dropout(dropout_rate))
51:     model.add(tf.keras.layers.Dense(1, activation="sigmoid"))
52:     model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
53:     return model
54:
55:
56: def train_dnn_suite(job_id: str, config: dict[str, object] | None = None) -> dict[str, object]:
57:     tf = _tensorflow()
58:
59:     resolved = resolve_dnn_config(config)
60:     tf.random.set_seed(int(resolved.get("random_state", 42)))
61:     np.random.seed(int(resolved.get("random_state", 42)))
62:
63:     split = load_dnn_split(
64:         train_sample=resolved.get("train_sample"),
65:         test_sample=resolved.get("test_sample"),
66:         random_state=int(resolved.get("random_state", 42)),

```

```

67:     )
68:
69:     generated_run_id = f"dnn-{uuid4().hex[:12]}"
70:     target_dir = run_dir(generated_run_id)
71:     models_dir = target_dir / "models"
72:     history_dir = target_dir / "history"
73:     predictions_dir = target_dir / "predictions"
74:     models_dir.mkdir(parents=True, exist_ok=True)
75:     history_dir.mkdir(parents=True, exist_ok=True)
76:     predictions_dir.mkdir(parents=True, exist_ok=True)
77:
78:     y_test = split["y_test"]
79:     results = []
80:
81:     for layer_count in resolved["architectures"]:
82:         model = _build_model(layer_count)
83:         checkpoint_path = models_dir / f"dnn{layer_count}_best.keras"
84:         csv_logger_path = history_dir / f"dnn{layer_count}_history.csv"
85:
86:         callbacks = [
87:             tf.keras.callbacks.ModelCheckpoint(filepath=str(checkpoint_path), save_best_only=True, monitor="loss", verbose=0),
88:             tf.keras.callbacks.CSVLogger(str(csv_logger_path), separator=";", append=False),
89:         ]
90:
91:         started = time.perf_counter()
92:         history = model.fit(
93:             split["X_train"],
94:             split["y_train"],
95:             batch_size=int(resolved["batch_size"]),
96:             epochs=int(resolved["epochs"]),
97:             verbose=0,
98:             callbacks=callbacks,
99:         )
100:         training_seconds = round(time.perf_counter() - started, 3)
101:         best_model = tf.keras.models.load_model(checkpoint_path, compile=False)
102:         probabilities = best_model.predict(split["X_test"], verbose=0).reshape(-1)
103:         predicted = (probabilities >= 0.5).astype(int)
104:         metrics = binary_metrics(y_test, predicted)
105:
106:         model_path = models_dir / f"dnn{layer_count}.keras"
107:         best_model.save(model_path)
108:         np.savetxt(predictions_dir / f"dnn{layer_count}_labels.txt", predicted, fmt="%01d")
109:         np.savetxt(predictions_dir / f"dnn{layer_count}_probabilities.txt", probabilities)
110:
111:         results.append(
112:             {
113:                 "id": f"dnn_{layer_count}_layer",
114:                 "label": f"DNN {layer_count} Layer",
115:                 "training_seconds": training_seconds,
116:                 "metrics": metrics,
117:                 "probability_summary": probability_summary(probabilities),
118:                 "history": {
119:                     "epochs": int(len(history.history["loss"])),
120:                     "best_accuracy": round(float(max(history.history["accuracy"])), 6),
121:                     "best_loss": round(float(min(history.history["loss"])), 6),
122:                     "final_accuracy": round(float(history.history["accuracy"][-1]), 6),
123:                     "final_loss": round(float(history.history["loss"][-1]), 6),
124:                 },
125:                 "model_path": str(model_path.relative_to(target_dir.parent.parent)),
126:             }
127:         )
128:
129:     results.sort(key=lambda item: item["metrics"]["f1"], reverse=True)
130:     summary = {
131:         "run_id": generated_run_id,
132:         "job_id": job_id,
133:         "kind": "dnn_train",
134:         "created_at": datetime.now(timezone.utc).isoformat(),
135:         "config": resolved,
136:         "dataset": {
137:             "train_rows": split["train_rows"],
138:             "test_rows": split["test_rows"],
139:             "feature_count": split["feature_count"],
140:         },
141:         "results": results,
142:         "best_model": results[0]["id"] if results else None,

```

```

143:     }
144:     write_json(run_summary_path(generated_run_id), summary)
145:     return summary

```

Listing 14. Dataset loading and split management

All higher-level workflows depend on these dataset loaders for sampling, summaries, and feature extraction.

```

1: from __future__ import annotations
2:
3: import csv
4: import hashlib
5: from functools import lru_cache
6: from pathlib import Path
7:
8: import numpy as np
9: import pandas as pd
10: from sklearn.model_selection import train_test_split
11: from sklearn.preprocessing import Normalizer
12:
13: from .config import CLASSICAL_TEST_DATA, CLASSICAL_TRAIN_DATA, DNN_TEST_DATA, DNN_TRAIN_DATA, ROOT_DIR
14:
15:
16: def _sample_frame(frame: pd.DataFrame, sample_size: int | None, random_state: int) -> pd.DataFrame:
17:     if sample_size is None or sample_size >= len(frame):
18:         return frame
19:     sampled, _ = train_test_split(frame, train_size=sample_size, stratify=frame.iloc[:, 0], random_state=random_state)
20:     return sampled.reset_index(drop=True)
21:
22:
23: def load_dataset_split(
24:     train_path: Path,
25:     test_path: Path,
26:     train_sample: int | None = None,
27:     test_sample: int | None = None,
28:     random_state: int = 42,
29: ) -> dict[str, np.ndarray | int | str]:
30:     train_frame = pd.read_csv(train_path, header=None)
31:     test_frame = pd.read_csv(test_path, header=None)
32:
33:     train_frame = _sample_frame(train_frame, train_sample, random_state)
34:     test_frame = _sample_frame(test_frame, test_sample, random_state)
35:
36:     X_train = train_frame.iloc[:, 1:42].to_numpy(dtype=np.float32)
37:     y_train = train_frame.iloc[:, 0].to_numpy(dtype=np.int32)
38:     X_test = test_frame.iloc[:, 1:42].to_numpy(dtype=np.float32)
39:     y_test = test_frame.iloc[:, 0].to_numpy(dtype=np.int32)
40:
41:     scaler = Normalizer()
42:     X_train = scaler.fit_transform(X_train)
43:     X_test = scaler.transform(X_test)
44:
45:     return {
46:         "X_train": X_train,
47:         "y_train": y_train,
48:         "X_test": X_test,
49:         "y_test": y_test,
50:         "train_rows": int(X_train.shape[0]),
51:         "test_rows": int(X_test.shape[0]),
52:         "feature_count": int(X_train.shape[1]),
53:     }
54:
55:
56: def load_classical_split(train_sample: int | None = None, test_sample: int | None = None, random_state: int = 42) -> dict:
57:     return load_dataset_split(CLASSICAL_TRAIN_DATA, CLASSICAL_TEST_DATA, train_sample, test_sample, random_state)
58:
59:
60: def load_dnn_split(train_sample: int | None = None, test_sample: int | None = None, random_state: int = 42) -> dict[str, np.ndarray | int | str]:
61:     return load_dataset_split(DNN_TRAIN_DATA, DNN_TEST_DATA, train_sample, test_sample, random_state)
62:
63:
64: def _dataset_file_summary(path: Path) -> dict[str, object]:
65:     sha256 = hashlib.sha256()
66:     row_count = 0

```

```

67:     columns = 0
68:     label_counts: dict[str, int] = {}
69:
70:     with path.open("rb") as raw_handle:
71:         for chunk in iter(lambda: raw_handle.read(1024 * 1024), b''):
72:             sha256.update(chunk)
73:
74:     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
75:         reader = csv.reader(handle)
76:         for row in reader:
77:             if not row:
78:                 continue
79:             row_count += 1
80:             if columns == 0:
81:                 columns = len(row)
82:             label = row[0]
83:             label_counts[label] = label_counts.get(label, 0) + 1
84:
85:     return {
86:         "path": _relative_path(path),
87:         "rows": row_count,
88:         "columns": columns,
89:         "size_mb": round(path.stat().st_size / (1024 * 1024), 2),
90:         "sha256": sha256.hexdigest(),
91:         "label_counts": label_counts,
92:     }
93:
94:
95: def _relative_path(path: Path) -> str:
96:     try:
97:         return str(path.relative_to(ROOT_DIR))
98:     except ValueError:
99:         return str(path)
100:
101:
102: @lru_cache(maxsize=1)
103: def dataset_summary() -> dict[str, object]:
104:     classical_train = _dataset_file_summary(CLASSICAL_TRAIN_DATA)
105:     classical_test = _dataset_file_summary(CLASSICAL_TEST_DATA)
106:     dnn_train = _dataset_file_summary(DNN_TRAIN_DATA)
107:     dnn_test = _dataset_file_summary(DNN_TEST_DATA)
108:
109:     classical_train["path"] = _relative_path(CLASSICAL_TRAIN_DATA)
110:     classical_test["path"] = _relative_path(CLASSICAL_TEST_DATA)
111:     dnn_train["path"] = _relative_path(DNN_TRAIN_DATA)
112:     dnn_test["path"] = _relative_path(DNN_TEST_DATA)
113:
114:     return {
115:         "classical_train": classical_train,
116:         "classical_test": classical_test,
117:         "dnn_train": dnn_train,
118:         "dnn_test": dnn_test,
119:         "duplicates": {
120:             "train_files_match": classical_train["sha256"] == dnn_train["sha256"],
121:             "test_files_match": classical_test["sha256"] == dnn_test["sha256"],
122:         },
123:     }

```

Listing 15. Run and job storage primitives

The storage helper is small but strategically important because it standardizes where automation summaries and run metadata live.

```

1: from __future__ import annotations
2:
3: import json
4: from pathlib import Path
5: from typing import Any
6:
7: from .config import AUTOMATION_DIR, JOBS_DIR, LEGACY_DIR, RUNS_DIR
8:
9:
10: def ensure_directories() -> None:
11:     for path in (AUTOMATION_DIR, JOBS_DIR, RUNS_DIR, LEGACY_DIR):

```

```

12:         path.mkdir(parents=True, exist_ok=True)
13:
14:
15: def read_json(path: Path, default: Any = None) -> Any:
16:     if not path.exists():
17:         return default
18:     with path.open("r", encoding="utf-8") as handle:
19:         return json.load(handle)
20:
21:
22: def write_json(path: Path, payload: Any) -> None:
23:     path.parent.mkdir(parents=True, exist_ok=True)
24:     with path.open("w", encoding="utf-8") as handle:
25:         json.dump(payload, handle, indent=2, sort_keys=False)
26:
27:
28: def job_path(job_id: str) -> Path:
29:     return JOBS_DIR / f"{job_id}.json"
30:
31:
32: def run_dir(run_id: str) -> Path:
33:     return RUNS_DIR / run_id
34:
35:
36: def run_summary_path(run_id: str) -> Path:
37:     return run_dir(run_id) / "summary.json"
38:
39:
40: def list_run_summaries(limit: int = 20) -> list[dict[str, Any]]:
41:     ensure_directories()
42:     summaries: list[dict[str, Any]] = []
43:     for path in sorted(RUNS_DIR.glob("*/summary.json"), key=lambda item: item.stat().st_mtime, reverse=True):
44:         payload = read_json(path)
45:         if payload:
46:             summaries.append(payload)
47:             if len(summaries) >= limit:
48:                 break
49:     return summaries
50:

```

Listing 16. API endpoints

This module exposes FastAPI endpoints for jobs and run metadata, but the report also flags it as a dependency surface that is not currently declared in package metadata.

```

1: from __future__ import annotations
2:
3: from pathlib import Path
4: from typing import Literal
5:
6: from fastapi import FastAPI, HTTPException
7: from fastapi.responses import FileResponse
8: from fastapi.staticfiles import StaticFiles
9: from pydantic import BaseModel, Field
10:
11: from .classical import train_classical_suite
12: from .config import CLASSICAL_PROFILES, DNN_PROFILES, FRONTEND_DIR
13: from .data import dataset_summary
14: from .dnn import train_dnn_suite
15: from .jobs import job_manager
16: from .legacy import evaluate_legacy_predictions
17: from .storage import ensure_directories, list_run_summaries, read_json, run_summary_path
18:
19:
20: class ClassicalJobRequest(BaseModel):
21:     profile: Literal["fast", "balanced", "full"] = "fast"
22:     models: list[str] | None = None
23:     train_sample: int | None = Field(default=None, ge=1000)
24:     test_sample: int | None = Field(default=None, ge=1000)
25:     random_state: int = 42
26:
27:
28: class DnnJobRequest(BaseModel):
29:     profile: Literal["fast", "balanced", "full"] = "fast"

```

```

30:     architectures: list[int] | None = None
31:     epochs: int | None = Field(default=None, ge=1, le=100)
32:     batch_size: int | None = Field(default=None, ge=16, le=512)
33:     train_sample: int | None = Field(default=None, ge=1000)
34:     test_sample: int | None = Field(default=None, ge=1000)
35:     random_state: int = 42
36:
37:
38: app = FastAPI(title="IDS Automation Console", version="1.0.0")
39: ensure_directories()
40: app.mount("/static", StaticFiles(directory=FRONTEND_DIR), name="static")
41:
42:
43: @app.get("/")
44: def index() -> FileResponse:
45:     return FileResponse(FRONTEND_DIR / "index.html")
46:
47:
48: @app.get("/api/health")
49: def health() -> dict[str, str]:
50:     return {"status": "ok"}
51:
52:
53: @app.get("/api/overview")
54: def overview() -> dict[str, object]:
55:     return {
56:         "datasets": dataset_summary(),
57:         "legacy": evaluate_legacy_predictions(),
58:         "jobs": job_manager.list(limit=12),
59:         "runs": list_run_summaries(limit=12),
60:         "profiles": {
61:             "classical": CLASSICAL_PROFILES,
62:             "dnn": DNN_PROFILES,
63:         },
64:     }
65:
66:
67: @app.get("/api/jobs")
68: def list_jobs() -> list[dict[str, object]]:
69:     return job_manager.list(limit=40)
70:
71:
72: @app.get("/api/jobs/{job_id}")
73: def get_job(job_id: str) -> dict[str, object]:
74:     payload = job_manager.get(job_id)
75:     if not payload:
76:         raise HTTPException(status_code=404, detail="Job not found")
77:     return payload
78:
79:
80: @app.get("/api/runs")
81: def list_runs() -> list[dict[str, object]]:
82:     return list_run_summaries(limit=40)
83:
84:
85: @app.get("/api/runs/{run_id}")
86: def get_run(run_id: str) -> dict[str, object]:
87:     payload = read_json(run_summary_path(run_id))
88:     if not payload:
89:         raise HTTPException(status_code=404, detail="Run not found")
90:     return payload
91:
92:
93: @app.post("/api/jobs/legacy-evaluation")
94: def launch_legacy_evaluation() -> dict[str, object]:
95:     return job_manager.submit("legacy_evaluation", {}, lambda _job_id, _config: {"run_id": "legacy-snapshot", "kind": "le
96:
97:
98: @app.post("/api/jobs/classical")
99: def launch_classical(request: ClassicalJobRequest) -> dict[str, object]:
100:     return job_manager.submit("classical_train", request.model_dump(exclude_none=True), train_classical_suite)
101:
102:
103: @app.post("/api/jobs/dnn")
104: def launch_dnn(request: DnnJobRequest) -> dict[str, object]:
105:     return job_manager.submit("dnn_train", request.model_dump(exclude_none=True), train_dnn_suite)

```

Listing 17. Legacy terminal interface overview

The legacy terminal predates the productified CLI and still imports the scientific stack at module import time.

```

1: from __future__ import annotations
2:
3: import argparse
4: import csv
5: import json
6: import os
7: import shlex
8: import sys
9: from pathlib import Path
10: from typing import Any
11:
12: from .classical import train_classical_suite
13: from .config import CLASSICAL_PROFILES, DNN_PROFILES, REPORTS_DIR, ROOT_DIR
14: from .data import dataset_summary
15: from .legacy import evaluate_legacy_predictions
16: from .storage import ensure_directories, list_run_summaries
17:
18:
19: BINARY_LABELS = {"0": "normal", "1": "attack"}
20: FEATURE_NAMES = [
21:     "duration",
22:     "protocol_type",
23:     "service",
24:     "flag",
25:     "src_bytes",
26:     "dst_bytes",
27:     "land",
28:     "wrong_fragment",
29:     "urgent",
30:     "hot",
31:     "num_failed_logins",
32:     "logged_in",
33:     "num_compromised",
34:     "root_shell",
35:     "su_attempted",
36:     "num_root",
37:     "num_file_creations",
38:     "num_shells",
39:     "num_access_files",
40:     "num_outbound_cmds",
41:     "is_host_login",
42:     "is_guest_login",
43:     "count",
44:     "srv_count",
45:     "serror_rate",
46:     "srv_serror_rate",
47:     "rerror_rate",
48:     "srv_rerror_rate",
49:     "same_srv_rate",
50:     "diff_srv_rate",
51:     "srv_diff_host_rate",
52:     "dst_host_count",
53:     "dst_host_srv_count",
54:     "dst_host_same_srv_rate",
55:     "dst_host_diff_srv_rate",
56:     "dst_host_same_src_port_rate",
57:     "dst_host_srv_diff_host_rate",
58:     "dst_host_serror_rate",
59:     "dst_host_srv_serror_rate",
60:     "dst_host_rerror_rate",
61:     "dst_host_srv_rerror_rate",
62: ]
63:
64:
65: AUTO_TRAIN_PROFILES = {
66:     "quick": {
67:         "classical": {"profile": "fast", "train_sample": 5000, "test_sample": 2000, "random_state": 42},
68:         "dnn": {

```

```

69:         "profile": "fast",
70:         "architectures": [1],
71:         "epochs": 1,
72:         "train_sample": 5000,
73:         "test_sample": 2000,
74:         "random_state": 42,
75:     },
76: },
77: "standard": {
78:     "classical": {"profile": "fast", "random_state": 42},
79:     "dnn": {"profile": "fast", "random_state": 42},
80: },
81: "full": {
82:     "classical": {"profile": "full", "random_state": 42},
83:     "dnn": {"profile": "full", "random_state": 42},
84: },
85: }
86:
87:
88: def _format_number(value: Any) -> str:
89:     if isinstance(value, float):
90:         return f"{value:.4f}"
91:     if isinstance(value, int):
92:         return f"{value:,}"
93:     return str(value)
94:
95:
96: def _percent(value: float | int) -> str:
97:     return f"{float(value) * 100:.2f}%"
98:
99:
100: def _table(headers: list[str], rows: list[list[Any]]) -> str:
101:     text_rows = [_format_number(cell) for cell in row] for row in rows]
102:     widths = [
103:         max(len(header), *(len(row[index]) for row in text_rows)) if text_rows else len(header)
104:         for index, header in enumerate(headers)
105:     ]
106:     line = " ".join(header.ljust(widths[index]) for index, header in enumerate(headers))
107:     rule = " ".join("-" * width for width in widths)
108:     body = [" ".join(row[index].ljust(widths[index]) for index in range(len(headers))) for row in text_rows]
109:     return "\n".join([line, rule, *body])
110:
111:
112: def _section(title: str) -> None:
113:     print(f"\n{title}")
114:     print("=" * len(title))
115:
116:
117: def _print_json(payload: Any) -> None:
118:     print(json.dumps(payload, indent=2, sort_keys=False))
119:
120:
121: def show_datasets(json_output: bool = False) -> None:
122:     payload = dataset_summary()
123:     if json_output:
124:         _print_json(payload)
125:         return
126:
127:     _section("Datasets")
128:     rows = []
129:     for key in ("classical_train", "classical_test", "dnn_train", "dnn_test"):
130:         item = payload[key]
131:         labels = ", ".join(f"{label}:{count}" for label, count in item["label_counts"].items())
132:         rows.append([item["path"], item["rows"], item["columns"], f"{item['size_mb']} MB", labels])
133:     print(_table(["Path", "Rows", "Cols", "Size", "Labels"], rows))
134:     print()
135:     print(f"Train files match: {payload['duplicates']['train_files_match']}")
136:     print(f"Test files match: {payload['duplicates']['test_files_match']}")
137:
138:
139: def show_attacks(json_output: bool = False) -> None:
140:     payload = dataset_summary()
141:     rows = []
142:     for key in ("classical_train", "classical_test"):
143:         item = payload[key]
144:         total = item["rows"]

```

```

145:         for label, count in sorted(item["label_counts"].items()):
146:             rows.append(
147:                 [
148:                     item["path"],
149:                     label,
150:                     BINARY_LABELS.get(label, "unknown"),
151:                     count,
152:                     f"{{(count / total) * 100:.2f}}%",
153:                 ]
154:             )
155:
156:     if json_output:
157:         _print_json(
158:             {
159:                 "label_meaning": BINARY_LABELS,
160:                 "note": "This dataset is binary encoded. Named attack families are not present in the CSV files.",
161:                 "rows": rows,
162:             }
163:         )
164:     return
165:
166:     _section("Attack Label Distribution")
167:     print(_table(["Dataset", "Label", "Meaning", "Rows", "Share"], rows))
168:     print()
169:     print("The CSV files only contain binary labels here. The app treats 0 as normal and 1 as attack.")
170:
171:
172: def show_features(json_output: bool = False) -> None:
173:     rows = [[0, "label", "binary target: 0=normal, 1=attack"]]
174:     rows.extend([[index, name, "numeric/coded IDS feature"] for index, name in enumerate(FEATURE_NAMES, start=1)])
175:     if json_output:
176:         _print_json({"columns": rows})
177:         return
178:     _section("Feature Columns")
179:     print(_table(["Column", "Name", "Meaning"], rows))
180:
181:
182: def show_legacy(json_output: bool = False) -> None:
183:     payload = evaluate_legacy_predictions()
184:     if json_output:
185:         _print_json(payload)
186:         return
187:
188:     _section("Legacy Classical Results")
189:     classical_rows = [
190:         [
191:             item["label"],
192:             _percent(item["metrics"]["accuracy"]),
193:             _percent(item["metrics"]["precision"]),
194:             _percent(item["metrics"]["recall"]),
195:             _percent(item["metrics"]["f1"]),
196:         ]
197:         for item in payload["classical"]
198:     ]
199:     print(_table(["Model", "Accuracy", "Precision", "Recall", "F1"], classical_rows))
200:
201:     _section("Legacy DNN Results")
202:     dnn_rows = [
203:         [
204:             item["label"],
205:             _percent(item["metrics"]["accuracy"]),
206:             _percent(item["metrics"]["precision"]),
207:             _percent(item["metrics"]["recall"]),
208:             _percent(item["metrics"]["f1"]),
209:             item["history"]["epochs_logged"] if item.get("history") else "n/a",
210:         ]
211:         for item in payload["dnn"]
212:     ]
213:     print(_table(["Model", "Accuracy", "Precision", "Recall", "F1", "Epochs"], dnn_rows))
214:
215:
216: def show_runs(json_output: bool = False, limit: int = 20) -> None:
217:     payload = list_run_summaries(limit=limit)
218:     if json_output:
219:         _print_json(payload)
220:         return

```

Listing 18. Legacy train-and-shell orchestration

This excerpt matters because it is the path that failed during verification when `joblib` was absent.

```
320:         "datasets": dataset_summary(),
321:         "legacy": evaluate_legacy_predictions(),
322:         "runs": list_run_summaries(limit=8),
323:         "profiles": {"classical": CLASSICAL_PROFILES, "dnn": DNN_PROFILES},
324:     }
325:     if json_output:
326:         _print_json(payload)
327:         return
328:     show_datasets()
329:     show_attacks()
330:     show_legacy()
331:     show_runs(limit=8)
332:     show_reports()
333:
334:
335: def train_classical(args: argparse.Namespace) -> None:
336:     config: dict[str, Any] = {"profile": args.profile, "random_state": args.random_state}
337:     if args.train_sample is not None:
338:         config["train_sample"] = args.train_sample
339:     if args.test_sample is not None:
340:         config["test_sample"] = args.test_sample
341:     if args.models:
342:         config["models"] = args.models
343:
344:     _section("Running Classical Train/Test")
345:     summary = train_classical_suite("terminal", config)
346:     _print_train_summary(summary)
347:
348:
349: def train_dnn(args: argparse.Namespace) -> None:
350:     from .dnn import train_dnn_suite
351:
352:     config: dict[str, Any] = {"profile": args.profile, "random_state": args.random_state}
353:     if args.architectures:
354:         config["architectures"] = args.architectures
355:     if args.epochs is not None:
356:         config["epochs"] = args.epochs
357:     if args.batch_size is not None:
358:         config["batch_size"] = args.batch_size
359:     if args.train_sample is not None:
360:         config["train_sample"] = args.train_sample
361:     if args.test_sample is not None:
362:         config["test_sample"] = args.test_sample
363:
364:     _section("Running DNN Train/Test")
365:     summary = train_dnn_suite("terminal", config)
366:     _print_train_summary(summary)
367:
368:
369: def auto_train(args: argparse.Namespace) -> None:
370:     profile_name = args.auto_profile
371:     config = AUTO_TRAIN_PROFILES[profile_name]
372:     _section(f"Auto Train: {profile_name}")
373:     print("Running classical suite...")
374:     classical_summary = train_classical_suite("terminal-auto", dict(config["classical"]))
375:     _print_train_summary(classical_summary)
376:
377:     if args.skip_dnn:
378:         return
379:
380:     print()
381:     print("Running DNN suite...")
382:     from .dnn import train_dnn_suite
383:
384:     dnn_summary = train_dnn_suite("terminal-auto", dict(config["dnn"]))
385:     _print_train_summary(dnn_summary)
386:
387:
388: def _print_train_summary(summary: dict[str, Any]) -> None:
```

```

389:     print(f"Run ID: {summary['run_id']}")
390:     print(f"Train rows: {summary['dataset']['train_rows'],}")
391:     print(f"Test rows: {summary['dataset']['test_rows'],}")
392:     rows = []
393:     for item in summary["results"]:
394:         rows.append(
395:             [
396:                 item["label"],
397:                 f"{item['training_seconds']]s",
398:                 _percent(item["metrics"]["accuracy"]),
399:                 _percent(item["metrics"]["precision"]),
400:                 _percent(item["metrics"]["recall"]),
401:                 _percent(item["metrics"]["f1"]),
402:             ]
403:         )
404:     print()
405:     print(_table(["Model", "Time", "Accuracy", "Precision", "Recall", "F1"], rows))
406:     print()
407:     print(f"Saved summary: automation/runs/{summary['run_id']}/summary.json")
408:
409:
410: def command_shell() -> None:
411:     ensure_directories()
412:     print("IDS command console. Type 'help' for commands, 'exit' to quit.")
413:     while True:
414:         try:
415:             raw = input("ids> ").strip()
416:         except EOFError:
417:             print()
418:             return
419:         if not raw:
420:             continue
421:         try:
422:             should_exit = run_shell_command(raw)
423:         except Exception as exc:
424:             print(f"error: {exc}")
425:             continue
426:         if should_exit:
427:             return
428:
429:
430: def run_shell_command(raw: str) -> bool:
431:     parts = shlex.split(raw)
432:     if not parts:
433:         return False
434:     command, *args = parts
435:     command = command.lower()
436:
437:     if command in {"exit", "quit", "q"}:
438:         return True
439:     if command == "help":
440:         print_shell_help()
441:     elif command == "clear":
442:         os.system("clear")
443:     elif command == "pwd":
444:         print(ROOT_DIR)
445:     elif command == "ls":
446:         list_path(args[0] if args else ".")
447:     elif command == "head":
448:         if not args:
449:             print("usage: head <path> [lines]")
450:         else:
451:             show_head(args[0], int(args[1]) if len(args) > 1 else 5)
452:     elif command in {"overview", "status"}:
453:         show_overview()
454:     elif command in {"data", "datasets"}:
455:         show_datasets()
456:     elif command in {"attack", "attacks"}:
457:         show_attacks()
458:     elif command in {"feature", "features", "schema"}:
459:         show_features()
460:     elif command == "legacy":
461:         show_legacy()
462:     elif command == "reports":
463:         show_reports()
464:     elif command == "runs":

```

```

465:         show_runs(limit=int(args[0]) if args else 20)
466:     elif command == "best":
467:         show_best()
468:     elif command == "train":
469:         run_train_command(args)
470:     else:
471:         print(f"Unknown command: {command}. Type 'help'.")
472:     return False
473:
474:
475: def print_shell_help() -> None:
476:     _section("Commands")
477:     rows = [
478:         ["help", "show this command list"],
479:         ["overview | status", "show datasets, attacks, legacy metrics, runs, reports"],
480:         ["datasets | data", "show CSV sizes, labels, duplicate checks"],
481:         ["attacks", "show attack/normal label distribution"],
482:         ["features | schema", "show the 42 dataset columns"],
483:         ["legacy", "show saved legacy prediction metrics"],
484:         ["runs [limit]", "show completed train/test runs"],
485:         ["best", "show best legacy and automation results"],
486:         ["reports", "list report PDFs"],
487:         ["ls [path]", "list repo files"],
488:         ["head <path> [lines]", "print first lines of a text/CSV file"],
489:         ["train auto [quick|standard|full]", "train classical and DNN automatically"],
490:         ["train classical [quick|fast|balanced|full]", "run classical train/test"],
491:         ["train dnn [quick|fast|balanced|full]", "run DNN train/test"],
492:         ["clear", "clear terminal"],
493:         ["exit", "quit"],
494:     ]
495:     print(_table(["Command", "Action"], rows))
496:
497:
498: def run_train_command(args: list[str]) -> None:
499:     if not args:
500:         print("usage: train <auto|classical|dnn> [quick|fast|standard|balanced|full]")
501:         return
502:     kind = args[0].lower()
503:     profile = args[1].lower() if len(args) > 1 else "quick"
504:
505:     if kind == "auto":
506:         if profile == "fast":
507:             profile = "quick"
508:         if profile not in AUTO_TRAIN_PROFILES:
509:             print("profiles: quick, standard, full")
510:             return
511:         namespace = argparse.Namespace(auto_profile=profile, skip_dnn=False)
512:         auto_train(namespace)
513:     elif kind == "classical":
514:         if profile == "quick":
515:             namespace = argparse.Namespace(profile="fast", train_sample=5000, test_sample=2000, models=None, random_state=None)
516:         else:
517:             if profile == "standard":
518:                 profile = "fast"
519:             if profile not in CLASSICAL_PROFILES:
520:                 print("profiles: quick, fast, balanced, full")
521:                 return
522:             namespace = argparse.Namespace(profile=profile, train_sample=None, test_sample=None, models=None, random_state=None)
523:         train_classical(namespace)
524:     elif kind == "dnn":
525:         if profile == "quick":
526:             namespace = argparse.Namespace(profile="fast", architectures=[1], epochs=1, batch_size=128, train_sample=5000)
527:         else:
528:             if profile == "standard":
529:                 profile = "fast"
530:             if profile not in DNN_PROFILES:
531:                 print("profiles: quick, fast, balanced, full")
532:                 return
533:             namespace = argparse.Namespace(profile=profile, architectures=None, epochs=None, batch_size=None, train_sample=None)
534:         train_dnn(namespace)
535:     else:
536:         print("train kinds: auto, classical, dnn")
537:
538:
539: def list_path(path_text: str) -> None:
540:     path = _resolve_repo_path(path_text)

```

```

541:     if not path.exists():
542:         print(f"Not found: {path_text}")
543:         return
544:     if path.is_file():
545:         print(path.relative_to(ROOT_DIR))
546:         return
547:     entries = []
548:     for child in sorted(path.iterdir(), key=lambda item: (item.is_file(), item.name.lower())):
549:         marker = "/" if child.is_dir() else ""
550:         entries.append([child.name + marker, f"{child.stat().st_size:,"} if child.is_file() else "dir"])
551:     print(_table(["Name", "Size"], entries))
552:
553:
554: def _resolve_repo_path(path_text: str) -> Path:
555:     path = Path(path_text)
556:     if not path.is_absolute():
557:         path = ROOT_DIR / path
558:     resolved = path.resolve()
559:     try:
560:         resolved.relative_to(ROOT_DIR)
561:     except ValueError:
562:         raise ValueError("path must stay inside the project directory")
563:     return resolved
564:
565:
566: def build_parser() -> argparse.ArgumentParser:
567:     parser = argparse.ArgumentParser(description="Terminal-only IDS automation console.")
568:     parser.add_argument("--json", action="store_true", help="Print JSON for commands that support it.")
569:     subparsers = parser.add_subparsers(dest="command")
570:
571:     subparsers.add_parser("overview", help="Show datasets, legacy metrics, runs, and reports.")
572:     subparsers.add_parser("datasets", help="Show CSV file sizes, labels, hashes, and duplicate checks.")
573:     subparsers.add_parser("attacks", help="Show attack/normal label distribution.")
574:     subparsers.add_parser("features", help="Show target and feature columns.")
575:     subparsers.add_parser("legacy", help="Show metrics for existing saved prediction files.")
576:     subparsers.add_parser("best", help="Show best known legacy and generated results.")
577:     subparsers.add_parser("shell", help="Open the interactive IDS command shell.")
578:     runs_parser = subparsers.add_parser("runs", help="Show completed automation runs.")
579:     runs_parser.add_argument("--limit", type=int, default=20)
580:     subparsers.add_parser("reports", help="List PDF reports available in the repo.")
581:
582:     auto_parser = subparsers.add_parser("auto-train", help="Train classical and DNN suites automatically.")
583:     auto_parser.add_argument("--auto-profile", choices=AUTO_TRAIN_PROFILES.keys(), default="quick")
584:     auto_parser.add_argument("--skip-dnn", action="store_true")
585:
586:     classical_parser = subparsers.add_parser("train-classical", help="Run a classical ML train/test suite.")
587:     classical_parser.add_argument("--profile", choices=CLASSICAL_PROFILES.keys(), default="fast")
588:     classical_parser.add_argument("--train-sample", type=int)
589:     classical_parser.add_argument("--test-sample", type=int)
590:     classical_parser.add_argument("--models", nargs="+")
591:     classical_parser.add_argument("--random-state", type=int, default=42)
592:
593:     dnn_parser = subparsers.add_parser("train-dnn", help="Run a DNN train/test suite.")
594:     dnn_parser.add_argument("--profile", choices=DNN_PROFILES.keys(), default="fast")
595:     dnn_parser.add_argument("--architectures", nargs="+", type=int)
596:     dnn_parser.add_argument("--epochs", type=int)
597:     dnn_parser.add_argument("--batch-size", type=int)
598:     dnn_parser.add_argument("--train-sample", type=int)
599:     dnn_parser.add_argument("--test-sample", type=int)
600:     dnn_parser.add_argument("--random-state", type=int, default=42)
601:
602:     return parser
603:
604:
605: def main(argv: list[str] | None = None) -> int:
606:     ensure_directories()
607:     parser = build_parser()
608:     args = parser.parse_args(argv)
609:
610:     try:
611:         if args.command is None:
612:             command_shell()
613:         elif args.command == "shell":
614:             command_shell()
615:         elif args.command == "overview":
616:             show_overview(args.json)

```

```

617:         elif args.command == "datasets":
618:             show_datasets(args.json)
619:         elif args.command == "attacks":
620:             show_attacks(args.json)
621:         elif args.command == "features":
622:             show_features(args.json)
623:         elif args.command == "legacy":
624:             show_legacy(args.json)
625:         elif args.command == "best":
626:             show_best(args.json)
627:         elif args.command == "runs":
628:             show_runs(args.json, args.limit)
629:         elif args.command == "reports":
630:             show_reports(args.json)
631:         elif args.command == "auto-train":
632:             auto_train(args)
633:         elif args.command == "train-classical":
634:             train_classical(args)
635:         elif args.command == "train-dnn":
636:             train_dnn(args)
637:         else:
638:             parser.error(f"Unknown command: {args.command}")
639:     except KeyboardInterrupt:
640:         print("\nInterrupted.")
641:         return 130
642:     return 0
643:
644:
645: if __name__ == "__main__":
646:     raise SystemExit(main(sys.argv[1:]))

```

Listing 19. Installable package metadata and entrypoints

The project metadata successfully defines a distributable CLI and GUI, but it currently omits the optional scientific and API dependencies that parts of the repo assume.

```

1: [build-system]
2: requires = ["setuptools>=69", "wheel"]
3: build-backend = "setuptools.build_meta"
4:
5: [project]
6: name = "ids-sentinel-terminal"
7: version = "0.2.0"
8: description = "Installable CLI and GUI for IDS Sentinel Terminal traffic analysis and local triage."
9: readme = "README.md"
10: requires-python = ">=3.10"
11: authors = [
12:     { name = "IDS Sentinel Terminal Contributors" }
13: ]
14: keywords = ["ids", "cli", "security", "network", "traffic-analysis", "threat-hunting"]
15: classifiers = [
16:     "Development Status :: 4 - Beta",
17:     "Environment :: Console",
18:     "Environment :: Win32 (MS Windows)",
19:     "Intended Audience :: Information Technology",
20:     "Intended Audience :: System Administrators",
21:     "Operating System :: MacOS",
22:     "Operating System :: Microsoft :: Windows",
23:     "Operating System :: POSIX :: Linux",
24:     "Programming Language :: Python :: 3",
25:     "Programming Language :: Python :: 3 :: Only",
26:     "Programming Language :: Python :: 3.10",
27:     "Programming Language :: Python :: 3.11",
28:     "Programming Language :: Python :: 3.12",
29:     "Programming Language :: Python :: 3.13",
30:     "Topic :: Security",
31:     "Topic :: System :: Networking :: Monitoring",
32:     "Topic :: Utilities",
33: ]
34:
35: [project.urls]
36: Homepage = "https://github.com/ankitRaj10022/IDS-Sentinel"
37: Repository = "https://github.com/ankitRaj10022/IDS-Sentinel"
38: Issues = "https://github.com/ankitRaj10022/IDS-Sentinel/issues"

```

```

39:
40: [project.scripts]
41: ids-sentinel = "ids_app.product_app:main"
42: ids-sentinel-terminal = "ids_app.product_app:main"
43: ids-sentinal = "ids_app.product_app:main"
44:
45: [project.gui-scripts]
46: ids-sentinel-gui = "ids_app.product_gui:main"
47:
48: [tool.setuptools.packages.find]
49: include = ["ids_app*"]
50:
51: [tool.setuptools.package-data]
52: ids_app = ["assets/*"]

```

Listing 20. Release and PyPI workflow

The workflow already implements artifact building, a smoke test, GitHub release uploads, and trusted publishing to PyPI.

```

1: name: Release IDS Sentinel Terminal
2:
3: on:
4:   workflow_dispatch:
5:     release:
6:       types: [published]
7:
8: permissions:
9:   contents: read
10:
11: jobs:
12:   release-build:
13:     runs-on: ubuntu-latest
14:     permissions:
15:       contents: write
16:
17:     steps:
18:       - name: Check out repository
19:         uses: actions/checkout@v4
20:
21:       - name: Set up Python
22:         uses: actions/setup-python@v5
23:         with:
24:           python-version: "3.12"
25:
26:       - name: Build Python package and portable archives
27:         run: |
28:           python -m pip install --upgrade pip build twine
29:           python scripts/build_python_package.py
30:           python -m twine check dist/ids_sentinel_terminal-*.whl dist/ids_sentinel_terminal-*.tar.gz
31:           python scripts/build_distributions.py
32:
33:       - name: Smoke-test built wheel
34:         run: |
35:           python -m pip install dist/*.whl
36:           mkdir -p /tmp/ids-sentinel-release-test
37:           cd /tmp/ids-sentinel-release-test
38:           ids-sentinel --version
39:           ids-sentinel status --json >/tmp/ids-sentinel-release-status.json
40:
41:       - name: Upload release artifacts
42:         uses: actions/upload-artifact@v4
43:         with:
44:           name: release-dists
45:           path: dist/
46:
47:       - name: Attach artifacts to GitHub release
48:         if: ${ github.event_name == 'release' }
49:         env:
50:           GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
51:         run: |
52:           gh release upload "${ github.event.release.tag_name }" dist/* --clobber
53:
54:   pypi-publish:
55:     if: ${ github.event_name == 'release' }

```

```

56:     runs-on: ubuntu-latest
57:     needs:
58:       - release-build
59:
60:     permissions:
61:       id-token: write
62:
63:     environment:
64:       name: pypi
65:
66:     steps:
67:       - name: Download release artifacts
68:         uses: actions/download-artifact@v5
69:         with:
70:           name: release-dists
71:           path: dist/
72:
73:       - name: Publish wheel and sdist to PyPI
74:         uses: pypa/gh-action-pypi-publish@release/v1
75:         with:
76:           packages-dir: dist/

```

Listing 21. Wheel and source distribution helper

This helper stages a clean package tree and is especially useful in environments where `python -m build` is not the preferred path.

```

1: from __future__ import annotations
2:
3: import shutil
4: import subprocess
5: import sys
6: from pathlib import Path
7:
8:
9: ROOT = Path(__file__).resolve().parents[1]
10: DIST_DIR = ROOT / "dist"
11: BUILD_SRC_DIR = ROOT / "build" / "python-package-src"
12:
13:
14: def is_python_package_artifact(path: Path) -> bool:
15:     if path.suffix == ".whl" and path.name.startswith("ids_sentinel_terminal-"):
16:         return True
17:     if path.name.startswith(("ids_sentinel_terminal-", "ids-sentinel-terminal-")) and path.name.endswith(".tar.gz"):
18:         suffix = path.name.split("ids_sentinel_terminal-", 1)[-1] if path.name.startswith("ids_sentinel_terminal-") else
19:         return bool(suffix) and suffix[0].isdigit()
20:     return False
21:
22:
23: def clean_previous_outputs() -> None:
24:     if BUILD_SRC_DIR.exists():
25:         shutil.rmtree(BUILD_SRC_DIR)
26:     DIST_DIR.mkdir(parents=True, exist_ok=True)
27:     for path in DIST_DIR.iterdir():
28:         if path.is_dir():
29:             continue
30:         if not path.name.endswith(("whl", ".tar.gz")):
31:             continue
32:         if is_python_package_artifact(path):
33:             path.unlink()
34:
35:
36: def stage_sources() -> Path:
37:     BUILD_SRC_DIR.mkdir(parents=True, exist_ok=True)
38:     shutil.copy2(ROOT / "pyproject.toml", BUILD_SRC_DIR / "pyproject.toml")
39:     shutil.copy2(ROOT / "README.md", BUILD_SRC_DIR / "README.md")
40:     if (ROOT / "MANIFEST.in").exists():
41:         shutil.copy2(ROOT / "MANIFEST.in", BUILD_SRC_DIR / "MANIFEST.in")
42:     shutil.copytree(
43:         ROOT / "ids_app",
44:         BUILD_SRC_DIR / "ids_app",
45:         ignore=shutil.ignore_patterns("__pycache__", "*.pyc", ".pytest_cache"),
46:     )
47:     (BUILD_SRC_DIR / "setup.py").write_text("from setuptools import setup\nsetup()\n", encoding="utf-8", newline="\n")
48:     return BUILD_SRC_DIR

```

```

49:
50:
51: def ensure_build_tools() -> None:
52:     try:
53:         import setuptools # noqa: F401
54:         import wheel # noqa: F401
55:     except ImportError:
56:         subprocess.run([sys.executable, "-m", "pip", "install", "setuptools", "wheel"], cwd=ROOT, check=True)
57:
58:
59: def run_setup(stage_dir: Path, command: str) -> None:
60:     subprocess.run(
61:         [sys.executable, "setup.py", command, "--dist-dir", str(DIST_DIR)],
62:         cwd=stage_dir,
63:         check=True,
64:     )
65:
66:
67: def main() -> int:
68:     clean_previous_outputs()
69:     ensure_build_tools()
70:     stage_dir = stage_sources()
71:     run_setup(stage_dir, "bdist_wheel")
72:     run_setup(stage_dir, "sdist")
73:     for path in sorted(DIST_DIR.iterdir()):
74:         if path.is_file() and is_python_package_artifact(path):
75:             size_mb = path.stat().st_size / (1024 * 1024)
76:             print(f"{path.relative_to(ROOT)} ({size_mb:.2f} MB)")
77:     return 0
78:
79:
80: if __name__ == "__main__":
81:     raise SystemExit(main())

```

Appendix E. Extended Core Listings

Extended core chunk 1: `product\_terminal.py` lines 1-72

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

1: from __future__ import annotations
2:
3: import argparse
4: import csv
5: import fnmatch
6: import hashlib
7: import ipaddress
8: import importlib.resources
9: import json
10: import math
11: import os
12: import re
13: import shlex
14: import shutil
15: import socket
16: import subprocess
17: import sys
18: import time
19: import urllib.request
20: from collections import Counter
21: from dataclasses import dataclass
22: from datetime import datetime, timezone
23: from pathlib import Path
24: from typing import Any, Iterable
25: from uuid import uuid4
26:
27: from . import __version__
28:
29: PACKAGE_DIR = Path(__file__).resolve().parent
30: SOURCE_ROOT = PACKAGE_DIR.parent.resolve()
31: DEFAULT_HOME_DIR = Path.home() / ".ids-sentinel-terminal"

```

```

32: IS_SOURCE_CHECKOUT = (SOURCE_ROOT / ".git").exists() and (SOURCE_ROOT / "kddtrain.csv").exists() and (SOURCE_ROOT / "kddtest.csv").exists()
33: ENV_ROOT = os.environ.get("IDS_PRODUCT_HOME")
34: ROOT_DIR = Path(ENV_ROOT).expanduser().resolve() if ENV_ROOT else (SOURCE_ROOT if IS_SOURCE_CHECKOUT else DEFAULT_HOME_DIR)
35: RUNTIME_MODE = "override" if ENV_ROOT else ("source" if IS_SOURCE_CHECKOUT else "installed")
36: BUNDLED_SEED_FILES = {
37:     "kddtrain.csv": "kddtrain.csv",
38:     "kddtest.csv": "kddtest.csv",
39:     "automation/product/self_learning_model.json": "self_learning_model.json",
40:     "automation/product/iocs.json": "iocs.json",
41: }
42:
43:
44: def _copy_bundled_asset(asset_name: str, destination: Path) -> None:
45:     resource = importlib.resources.files("ids_app").joinpath("assets", asset_name)
46:     destination.parent.mkdir(parents=True, exist_ok=True)
47:     with importlib.resources.as_file(resource) as source_path:
48:         shutil.copy2(source_path, destination)
49:
50:
51: def bootstrap_runtime_home() -> None:
52:     if RUNTIME_MODE == "source":
53:         return
54:     ROOT_DIR.mkdir(parents=True, exist_ok=True)
55:     for relative_target, asset_name in BUNDLED_SEED_FILES.items():
56:         destination = ROOT_DIR / relative_target
57:         if destination.exists():
58:             continue
59:         _copy_bundled_asset(asset_name, destination)
60:
61:
62: bootstrap_runtime_home()
63:
64: TRAIN_CSV = ROOT_DIR / "kddtrain.csv"
65: TEST_CSV = ROOT_DIR / "kddtest.csv"
66: PRODUCT_DIR = ROOT_DIR / "automation" / "product"
67: EXPORTS_DIR = PRODUCT_DIR / "exports"
68: IMPORTS_DIR = PRODUCT_DIR / "imports"
69: CACHE_DIR = PRODUCT_DIR / "cache"
70: INDEX_DIR = CACHE_DIR / "indexes"
71: COMMAND_CACHE_DIR = CACHE_DIR / "commands"
72: LEGACY_INDEX_DIR = PRODUCT_DIR / "indexes"

```

Extended core chunk 2: `product\_terminal.py` lines 73-144

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

73: MODEL_PATH = PRODUCT_DIR / "self_learning_model.json"
74: IOC_PATH = PRODUCT_DIR / "iocs.json"
75:
76: SHELL_STATE = {"cwd": ROOT_DIR, "history": []}
77:
78: EXTERNAL_DATASETS = [
79:     {
80:         "id": "cicids2017",
81:         "name": "CIC-IDS2017",
82:         "source": "Canadian Institute for Cybersecurity, University of New Brunswick",
83:         "url": "https://www.unb.ca/cic/datasets/ids-2017.html",
84:         "format": "PCAP and CICFlowMeter CSV flow data",
85:         "notes": "Contains benign traffic plus FTP/SSH brute force, DoS, Heartbleed, web attacks, infiltration, botnet, DDoS",
86:     },
87:     {
88:         "id": "unsw-nb15",
89:         "name": "UNSW-NB15",
90:         "source": "UNSW Canberra Cyber Range Lab",
91:         "url": "https://research.unsw.edu.au/projects/unsw-nb15-dataset",
92:         "format": "CSV network-flow features with binary and attack-category labels",
93:         "notes": "Contains normal traffic and Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode",
94:     },
95:     {
96:         "id": "unsw-nb15-hf",
97:         "name": "UNSW-NB15 Hugging Face mirror",
98:         "source": "Hugging Face community dataset mirror",
99:         "url": "https://huggingface.co/datasets/lacg030175/UNSW-NB15",
100:         "format": "Dataset hub train/test splits",
101:         "notes": "Useful when the Python datasets package is installed. This terminal can still import downloaded CSV files"

```

```

102:     },
103: ]
104:
105: COMMON_PORT_RISKS = {
106:     20: "FTP data channel. Plaintext file transfer; watch for data exfiltration.",
107:     21: "FTP control. Plaintext credentials; common brute-force target.",
108:     22: "SSH. Remote admin service; watch for brute-force and unusual geos.",
109:     23: "Telnet. Plaintext remote shell; should usually be disabled.",
110:     25: "SMTP. Mail relay; watch for spam abuse and open relay exposure.",
111:     53: "DNS. Watch for tunneling, amplification, and suspicious resolver exposure.",
112:     80: "HTTP. Web service; inspect for web attacks and exposed admin panels.",
113:     110: "POP3. Plaintext mail retrieval unless wrapped; legacy exposure risk.",
114:     135: "MS RPC. Windows lateral-movement surface; restrict to trusted networks.",
115:     139: "NetBIOS. Legacy Windows file-sharing surface.",
116:     143: "IMAP. Mail retrieval; watch authentication abuse.",
117:     443: "HTTPS. Web service; inspect certificates and web attack logs.",
118:     445: "SMB. High-value Windows file-sharing target; restrict heavily.",
119:     1433: "Microsoft SQL Server. Database exposure risk.",
120:     1521: "Oracle database listener. Database exposure risk.",
121:     2049: "NFS. File-sharing service; restrict to trusted hosts.",
122:     2375: "Docker API without TLS. Critical exposure if reachable.",
123:     3306: "MySQL/MariaDB. Database exposure risk.",
124:     3389: "RDP. Common brute-force and ransomware entry point.",
125:     5432: "PostgreSQL. Database exposure risk.",
126:     5900: "VNC. Remote desktop; often weakly protected.",
127:     6379: "Redis. Critical if unauthenticated or internet-exposed.",
128:     8080: "Alternate HTTP/proxy/admin service.",
129:     9200: "Elasticsearch. Data exposure risk if unauthenticated.",
130:     11211: "Memcached. Amplification and data exposure risk.",
131:     27017: "MongoDB. Database exposure risk.",
132: }
133: COMMON_PROBE_PORTS = [21, 22, 23, 25, 53, 80, 110, 135, 139, 143, 443, 445, 1433, 3306, 3389, 5432, 5900, 6379, 8080, 9200,
134: TEXT_SEARCH_MAX_FILE_BYTES = 128 * 1024 * 1024
135: TEXT_SEARCH_SKIP_SUFFIXES = {
136:     ".7z",
137:     ".bin",
138:     ".dll",
139:     ".exe",
140:     ".gz",
141:     ".h5",
142:     ".joblib",
143:     ".jpg",
144:     ".jpeg",

```

Extended core chunk 3: `product\_terminal.py` lines 145-216

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

145:     ".keras",
146:     ".pdf",
147:     ".png",
148:     ".pyc",
149:     ".tar",
150:     ".zip",
151: }
152:
153: SUSPICIOUS_FILE_PATTERNS = [
154:     "powershell -enc",
155:     "powershell.exe -enc",
156:     "frombase64string",
157:     "invoke-expression",
158:     "downloadstring",
159:     "certutil -urlcache",
160:     "bitsadmin",
161:     "rundll32",
162:     "regsvr32",
163:     "wscript.shell",
164:     "mimikatz",
165:     "meterpreter",
166:     "cobalt strike",
167:     "reverse_tcp",
168:     "cmd.exe /c",
169: ]
170:
171: BINARY_LABELS = {"0": "normal", "1": "attack"}

```

```

172: CSV_FEATURE_PREFIX = "feature_"
173: FEATURE_NAMES = [
174:     "duration",
175:     "protocol_type",
176:     "service",
177:     "flag",
178:     "src_bytes",
179:     "dst_bytes",
180:     "land",
181:     "wrong_fragment",
182:     "urgent",
183:     "hot",
184:     "num_failed_logins",
185:     "logged_in",
186:     "num_compromised",
187:     "root_shell",
188:     "su_attempted",
189:     "num_root",
190:     "num_file_creations",
191:     "num_shells",
192:     "num_access_files",
193:     "num_outbound_cmds",
194:     "is_host_login",
195:     "is_guest_login",
196:     "count",
197:     "srv_count",
198:     "serror_rate",
199:     "srv_serror_rate",
200:     "rerror_rate",
201:     "srv_rerror_rate",
202:     "same_srv_rate",
203:     "diff_srv_rate",
204:     "srv_diff_host_rate",
205:     "dst_host_count",
206:     "dst_host_srv_count",
207:     "dst_host_same_srv_rate",
208:     "dst_host_diff_srv_rate",
209:     "dst_host_same_src_port_rate",
210:     "dst_host_srv_diff_host_rate",
211:     "dst_host_serror_rate",
212:     "dst_host_srv_serror_rate",
213:     "dst_host_rerror_rate",
214:     "dst_host_srv_rerror_rate",
215: ]
216:

```

Extended core chunk 4: `product\_terminal.py` lines 217-288

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

217:
218: def ensure_product_dirs() -> None:
219:     for path in (PRODUCT_DIR, EXPORTS_DIR, IMPORTS_DIR, CACHE_DIR, INDEX_DIR, COMMAND_CACHE_DIR):
220:         path.mkdir(parents=True, exist_ok=True)
221:     if LEGACY_INDEX_DIR.exists() and LEGACY_INDEX_DIR.is_dir():
222:         for legacy_file in LEGACY_INDEX_DIR.glob("*.json"):
223:             target = INDEX_DIR / legacy_file.name
224:             if not target.exists():
225:                 shutil.move(str(legacy_file), str(target))
226:
227:
228: def utc_now() -> str:
229:     return datetime.now(timezone.utc).replace(microsecond=0).isoformat()
230:
231:
232: def compact_timestamp() -> str:
233:     return datetime.now().strftime("%Y%m%d_%H%M%S")
234:
235:
236: def relative_path(path: Path) -> str:
237:     try:
238:         return str(path.resolve().relative_to(ROOT_DIR))
239:     except ValueError:
240:         return str(path)
241:

```

```

242:
243: def format_number(value: Any) -> str:
244:     if isinstance(value, float):
245:         if abs(value) >= 1000:
246:             return f"{value:,.0f}"
247:             return f"{value:.4f}"
248:     if isinstance(value, int):
249:         return f"{value:,}"
250:     return str(value)
251:
252:
253: def percent(part: int | float, total: int | float) -> str:
254:     if not total:
255:         return "0.00%"
256:     return f"{{float(part) / float(total)) * 100:.2f}}%"
257:
258:
259: def table(headers: list[str], rows: list[list[Any]]) -> str:
260:     text_rows = [[format_number(cell) for cell in row] for row in rows]
261:     widths = [
262:         max(len(header), *(len(row[index]) for row in text_rows)) if text_rows else len(header)
263:         for index, header in enumerate(headers)
264:     ]
265:     header_line = " ".join(header.ljust(widths[index]) for index, header in enumerate(headers))
266:     rule = " ".join("-" * width for width in widths)
267:     body = [" ".join(row[index].ljust(widths[index]) for index in range(len(headers))) for row in text_rows]
268:     return "\n".join([header_line, rule, *body])
269:
270:
271: def section(title: str) -> None:
272:     print(f"\n{title}")
273:     print("=" * len(title))
274:
275:
276: def print_json(payload: Any) -> None:
277:     print(json.dumps(payload, indent=2, sort_keys=False))
278:
279:
280: def resolve_repo_path(path_text: str | None, default: Path = TEST_CSV) -> Path:
281:     if not path_text:
282:         return default
283:     path = Path(path_text)
284:     if not path.is_absolute():
285:         path = ROOT_DIR / path
286:     resolved = path.resolve()
287:     try:
288:         resolved.relative_to(ROOT_DIR)

```

Extended core chunk 5: `product\_terminal.py` lines 289-360

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

289:     except ValueError:
290:         raise ValueError("path must stay inside the IDS Sentinel home directory")
291:     return resolved
292:
293:
294: def read_json(path: Path, default: Any = None) -> Any:
295:     if not path.exists():
296:         return default
297:     with path.open("r", encoding="utf-8") as handle:
298:         return json.load(handle)
299:
300:
301: def write_json(path: Path, payload: Any) -> None:
302:     path.parent.mkdir(parents=True, exist_ok=True)
303:     with path.open("w", encoding="utf-8") as handle:
304:         json.dump(payload, handle, indent=2, sort_keys=False)
305:
306:
307: def cache_artifact(kind: str, payload: Any) -> Path:
308:     ensure_product_dirs()
309:     safe_kind = re.sub(r"^[A-Za-z0-9_-]+", "_", kind).strip("_") or "artifact"
310:     path = COMMAND_CACHE_DIR / f"{compact_timestamp()}_{safe_kind}_{uuid4().hex[:8]}.json"
311:     write_json(path, {"created_at": utc_now(), "kind": kind, "payload": payload})

```

```

312:     prune_cache_artifacts()
313:     return path
314:
315:
316: def prune_cache_artifacts(max_files: int = 500) -> None:
317:     artifacts = sorted(COMMAND_CACHE_DIR.glob("*.json"), key=lambda item: item.stat().st_mtime, reverse=True)
318:     for stale in artifacts[max_files:]:
319:         stale.unlink(missing_ok=True)
320:
321:
322: def path_cache_key(path: Path) -> str:
323:     resolved = str(path.resolve()).lower()
324:     digest = hashlib.sha1(resolved.encode("utf-8")).hexdigest()[:16]
325:     safe_name = re.sub(r"[^A-Za-z0-9_.-]+", "_", path.name)[:80]
326:     return f"{safe_name}.{digest}"
327:
328:
329: def file_signature(path: Path) -> dict[str, Any]:
330:     stat = path.stat()
331:     return {"size": stat.st_size, "mtime_ns": stat.st_mtime_ns}
332:
333:
334: def cached_json_path(path: Path, suffix: str) -> Path:
335:     return INDEX_DIR / f"{path_cache_key(path)}.{suffix}.json"
336:
337:
338: def is_cache_current(path: Path, payload: dict[str, Any] | None) -> bool:
339:     if not payload:
340:         return False
341:     return payload.get("signature") == file_signature(path)
342:
343:
344: def likely_header(row: list[str]) -> bool:
345:     if not row:
346:         return False
347:     numeric = 0
348:     for value in row:
349:         try:
350:             float(value)
351:             numeric += 1
352:         except ValueError:
353:             pass
354:     return numeric < max(1, len(row) // 2)
355:
356:
357: def all_csv_sources(include_exports: bool = True) -> list[Path]:
358:     sources = [TRAIN_CSV, TEST_CSV]
359:     if IMPORTS_DIR.exists():
360:         sources.extend(sorted(IMPORTS_DIR.glob("*.csv")))

```

Extended core chunk 6: `product\_terminal.py` lines 361-432

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

361:     if include_exports and EXPORTS_DIR.exists():
362:         sources.extend(sorted(EXPORTS_DIR.glob("*.csv")))
363:     return [path for path in sources if path.exists()]
364:
365:
366: def resolve_any_product_path(path_text: str | None, default: Path = TEST_CSV) -> Path:
367:     if not path_text:
368:         return default
369:     path = Path(path_text)
370:     if not path.is_absolute():
371:         shell_cwd = Path(SHELL_STATE.get("cwd", ROOT_DIR))
372:         path = shell_cwd / path
373:     resolved = path.resolve()
374:     try:
375:         resolved.relative_to(ROOT_DIR)
376:     except ValueError:
377:         raise ValueError("path must stay inside the IDS Sentinel home directory")
378:     return resolved
379:
380:
381: def resolve_readable_path(path_text: str | None, default: Path | None = None, base: Path | None = None) -> Path:

```

```

382:     if not path_text:
383:         if default is None:
384:             raise ValueError("path is required")
385:         path = default
386:     else:
387:         path = Path(path_text)
388:         if not path.is_absolute():
389:             path = (base or ROOT_DIR) / path
390:     resolved = path.resolve()
391:     if not resolved.exists() or not resolved.is_file():
392:         raise ValueError(f"not a readable file: {path_text or default}")
393:     return resolved
394:
395:
396: def safe_float(value: str) -> float:
397:     try:
398:         return float(value)
399:     except (TypeError, ValueError):
400:         return 0.0
401:
402:
403: def iter_kdd_rows(path: Path, limit: int | None = None) -> Iterable[tuple[int, str, list[float]]]:
404:     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
405:         reader = csv.reader(handle)
406:         emitted = 0
407:         for row_number, row in enumerate(reader, start=1):
408:             if len(row) < len(FEATURE_NAMES) + 1:
409:                 continue
410:             features = [safe_float(value) for value in row[1 : len(FEATURE_NAMES) + 1]]
411:             yield row_number, row[0].strip(), features
412:             emitted += 1
413:             if limit is not None and emitted >= limit:
414:                 return
415:
416:
417: def iter_generated_rows(path: Path, limit: int | None = None) -> Iterable[tuple[int, str, list[float]]]:
418:     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
419:         reader = csv.DictReader(handle)
420:         emitted = 0
421:         for row_number, row in enumerate(reader, start=2):
422:             label = str(row.get("actual_label") or row.get("label") or "").strip()
423:             if label not in BINARY_LABELS:
424:                 continue
425:             features = [safe_float(row.get(f"{CSV_FEATURE_PREFIX}{name}", "0")) for name in FEATURE_NAMES]
426:             yield row_number, label, features
427:             emitted += 1
428:             if limit is not None and emitted >= limit:
429:                 return
430:
431:
432: @dataclass

```

Extended core chunk 7: `product\_terminal.py` lines 433-504

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

433: class RunningStat:
434:     count: int = 0
435:     mean: float = 0.0
436:     m2: float = 0.0
437:     min_value: float = math.inf
438:     max_value: float = -math.inf
439:
440:     def update(self, value: float) -> None:
441:         self.count += 1
442:         delta = value - self.mean
443:         self.mean += delta / self.count
444:         delta2 = value - self.mean
445:         self.m2 += delta * delta2
446:         self.min_value = min(self.min_value, value)
447:         self.max_value = max(self.max_value, value)
448:
449:     def to_json(self) -> dict[str, float | int]:
450:         variance = self.m2 / max(self.count - 1, 1)
451:         return {

```

```

452:         "count": self.count,
453:         "mean": round(self.mean, 8),
454:         "variance": round(max(variance, 1e-9), 8),
455:         "min": round(self.min_value if self.count else 0.0, 8),
456:         "max": round(self.max_value if self.count else 0.0, 8),
457:     }
458:
459:
460: def empty_label_stats() -> dict[str, list[RunningStat]]:
461:     return {label: [RunningStat() for _ in FEATURE_NAMES] for label in BINARY_LABELS}
462:
463:
464: def update_model_stats(stats: dict[str, list[RunningStat]], labels: Counter[str], label: str, features: list[float]) -> None:
465:     if label not in stats:
466:         return
467:     labels[label] += 1
468:     for index, value in enumerate(features):
469:         stats[label][index].update(value)
470:
471:
472: def generated_export_paths() -> list[Path]:
473:     if not EXPORTS_DIR.exists():
474:         return []
475:     return sorted(EXPORTS_DIR.glob("traffic_analysis_*.csv"))
476:
477:
478: def learn_model(
479:     *,
480:     limit: int | None = None,
481:     include_generated: bool = True,
482:     include_test: bool = False,
483: ) -> dict[str, Any]:
484:     ensure_product_dirs()
485:     stats = empty_label_stats()
486:     labels: Counter[str] = Counter()
487:     sources: list[dict[str, Any]] = []
488:
489:     source_paths = [TRAIN_CSV]
490:     if include_test:
491:         source_paths.append(TEST_CSV)
492:
493:     for path in source_paths:
494:         rows_used = 0
495:         source_labels: Counter[str] = Counter()
496:         for _, label, features in iter_kdd_rows(path, limit):
497:             update_model_stats(stats, labels, label, features)
498:             source_labels[label] += 1
499:             rows_used += 1
500:         sources.append({"path": relative_path(path), "rows_used": rows_used, "label_counts": dict(source_labels)})
501:
502:     if include_generated:
503:         for path in generated_export_paths():
504:             rows_used = 0

```

Extended core chunk 8: `product\_terminal.py` lines 505-576

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

505:         source_labels = Counter()
506:         for _, label, features in iter_generated_rows(path, limit):
507:             update_model_stats(stats, labels, label, features)
508:             source_labels[label] += 1
509:             rows_used += 1
510:         if rows_used:
511:             sources.append({"path": relative_path(path), "rows_used": rows_used, "label_counts": dict(source_labels)})
512:
513:     total_rows = sum(labels.values())
514:     if total_rows == 0 or labels["0"] == 0 or labels["1"] == 0:
515:         raise RuntimeError("not enough labeled normal and attack rows to build a model")
516:
517:     label_payload: dict[str, Any] = {}
518:     for label in BINARY_LABELS:
519:         label_payload[label] = {
520:             "name": BINARY_LABELS[label],
521:             "count": labels[label],

```

```

522:         "prior": labels[label] / total_rows,
523:         "features": [item.to_json() for item in stats[label]],
524:     }
525:
526:     top_indicators = rank_indicators(label_payload)
527:     model = {
528:         "version": 1,
529:         "created_at": utc_now(),
530:         "model_type": "streaming_gaussian_profile",
531:         "description": "Pure-Python self-learning profile built from labeled IDS CSV rows and terminal-generated analysis e
532:         "features": FEATURE_NAMES,
533:         "labels": label_payload,
534:         "total_rows": total_rows,
535:         "sources": sources,
536:         "top_indicators": top_indicators[:12],
537:     }
538:     write_json(MODEL_PATH, model)
539:     cache_artifact("learn", {"model_path": relative_path(MODEL_PATH), "rows_learned": total_rows, "sources": sources})
540:     return model
541:
542:
543: def rank_indicators(label_payload: dict[str, Any]) -> list[dict[str, Any]]:
544:     rows = []
545:     normal_stats = label_payload["0"]["features"]
546:     attack_stats = label_payload["1"]["features"]
547:     for index, name in enumerate(FEATURE_NAMES):
548:         normal = normal_stats[index]
549:         attack = attack_stats[index]
550:         pooled_std = math.sqrt((float(normal["variance"]) + float(attack["variance"]))) / 2.0
551:         score = abs(float(attack["mean"]) - float(normal["mean"])) / max(pooled_std, 1e-6)
552:         rows.append(
553:             {
554:                 "feature": name,
555:                 "separation": round(score, 6),
556:                 "normal_mean": round(float(normal["mean"]), 6),
557:                 "attack_mean": round(float(attack["mean"]), 6),
558:             }
559:         )
560:     rows.sort(key=lambda item: item["separation"], reverse=True)
561:     return rows
562:
563:
564: def load_or_learn_model(auto_learn: bool = True) -> dict[str, Any]:
565:     model = read_json(MODEL_PATH)
566:     if model:
567:         return model
568:     if not auto_learn:
569:         raise RuntimeError("model does not exist yet; run 'learn' first")
570:     return learn_model(limit=None)
571:
572:
573: def gaussian_log_probability(features: list[float], label_model: dict[str, Any], indicator_names: set[str] | None = None) -
574:     logp = math.log(max(float(label_model["prior"]), 1e-12))
575:     for index, value in enumerate(features):
576:         name = FEATURE_NAMES[index]

```

Extended core chunk 9: `product\_terminal.py` lines 577-648

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

577:         if indicator_names is not None and name not in indicator_names:
578:             continue
579:         stat = label_model["features"][index]
580:         mean = float(stat["mean"])
581:         variance = max(float(stat["variance"]), 1e-6)
582:         logp += -0.5 * math.log(2.0 * math.pi * variance)
583:         logp += -((value - mean) ** 2) / (2.0 * variance)
584:     return logp
585:
586:
587: def score_row(model: dict[str, Any], features: list[float]) -> dict[str, Any]:
588:     indicator_names = {item["feature"] for item in model.get("top_indicators", [])[:16]} or None
589:     normal_log = gaussian_log_probability(features, model["labels"]["0"], indicator_names)
590:     attack_log = gaussian_log_probability(features, model["labels"]["1"], indicator_names)
591:     delta = max(min(attack_log - normal_log, 60.0), -60.0)

```

```

592:     attack_probability = 1.0 / (1.0 + math.exp(-delta))
593:     predicted = "1" if attack_probability >= 0.5 else "0"
594:     confidence = abs(attack_probability - 0.5) * 2.0
595:     family, reasons = classify_behavior(features, attack_probability, model)
596:     return {
597:         "predicted_label": predicted,
598:         "predicted_name": BINARY_LABELS[predicted],
599:         "risk_score": round(attack_probability, 6),
600:         "confidence": round(confidence, 6),
601:         "risk_level": risk_level(attack_probability),
602:         "family": family,
603:         "reasons": reasons,
604:     }
605:
606:
607: def feature_map(features: list[float]) -> dict[str, float]:
608:     return dict(zip(FEATURE_NAMES, features))
609:
610:
611: def risk_level(score: float) -> str:
612:     if score >= 0.9:
613:         return "critical"
614:     if score >= 0.75:
615:         return "high"
616:     if score >= 0.55:
617:         return "medium"
618:     return "low"
619:
620:
621: def classify_behavior(features: list[float], risk_score: float, model: dict[str, Any]) -> tuple[str, str]:
622:     values = feature_map(features)
623:     families: list[str] = []
624:     reasons: list[str] = []
625:
626:     if values["count"] >= 80 or values["srv_count"] >= 80 or values["error_rate"] >= 0.5 or values["srv_error_rate"] >= 0:
627:         families.append("dos_flood")
628:         reasons.append("high connection or service-error rate")
629:     if values["diff_srv_rate"] >= 0.35 or values["srv_diff_host_rate"] >= 0.35 or values["dst_host_srv_diff_host_rate"] >= 0:
630:         families.append("probe_scan")
631:         reasons.append("high service or host diversity")
632:     if values["num_failed_logins"] > 0 or values["is_guest_login"] > 0 or values["logged_in"] == 0 and values["hot"] >= 2:
633:         families.append("credential_abuse")
634:         reasons.append("login or credential anomaly")
635:     if values["root_shell"] > 0 or values["su_attempted"] > 0 or values["num_compromised"] > 0 or values["num_root"] > 0:
636:         families.append("privilege_escalation")
637:         reasons.append("compromise or privilege signal")
638:     if values["num_file_creations"] > 0 or values["num_shells"] > 0 or values["num_access_files"] > 0:
639:         families.append("malware_like_activity")
640:         reasons.append("file, shell, or access-file behavior")
641:     if values["wrong_fragment"] > 0 or values["urgent"] > 0 or values["src_bytes"] > 100000 or values["dst_bytes"] > 100000:
642:         families.append("payload_or_exfiltration")
643:         reasons.append("fragment, urgent, or high byte volume")
644:
645:     if risk_score < 0.55 and not families:
646:         return "normal", "close to learned normal profile"
647:
648:     if not families:

```

Extended core chunk 10: `product\_terminal.py` lines 649-720

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

649:         for item in model.get("top_indicators", []):4]:
650:             name = item["feature"]
651:             index = FEATURE_NAMES.index(name)
652:             value = features[index]
653:             normal_mean = float(item["normal_mean"])
654:             attack_mean = float(item["attack_mean"])
655:             if abs(value - attack_mean) < abs(value - normal_mean):
656:                 direction = "high" if attack_mean > normal_mean else "low"
657:                 reasons.append(f"{name} is {direction} versus normal profile")
658:                 families.append("network_attack")
659:
660:     return families[0], "; ".join(reasons[:4]) if reasons else "matches learned attack profile"
661:

```

```

662:
663: def summarize_dataset(path: Path, limit: int | None = None) -> dict[str, Any]:
664:     labels: Counter[str] = Counter()
665:     protocol_counts: Counter[str] = Counter()
666:     service_counts: Counter[str] = Counter()
667:     flag_counts: Counter[str] = Counter()
668:     total_src_bytes = 0.0
669:     total_dst_bytes = 0.0
670:     rows = 0
671:     malformed = 0
672:
673:     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
674:         reader = csv.reader(handle)
675:         for row in reader:
676:             if len(row) < len(FEATURE_NAMES) + 1:
677:                 malformed += 1
678:                 continue
679:                 rows += 1
680:                 labels[row[0].strip()] += 1
681:                 protocol_counts[row[2].strip()] += 1
682:                 service_counts[row[3].strip()] += 1
683:                 flag_counts[row[4].strip()] += 1
684:                 total_src_bytes += safe_float(row[5])
685:                 total_dst_bytes += safe_float(row[6])
686:                 if limit is not None and rows >= limit:
687:                     break
688:
689:     return {
690:         "path": relative_path(path),
691:         "rows": rows,
692:         "malformed_rows": malformed,
693:         "columns": len(FEATURE_NAMES) + 1,
694:         "size_mb": round(path.stat().st_size / (1024 * 1024), 2),
695:         "label_counts": dict(labels),
696:         "attack_share": round(labels["1"] / rows, 6) if rows else 0,
697:         "normal_share": round(labels["0"] / rows, 6) if rows else 0,
698:         "total_src_bytes": int(total_src_bytes),
699:         "total_dst_bytes": int(total_dst_bytes),
700:         "top_protocols": protocol_counts.most_common(5),
701:         "top_services": service_counts.most_common(8),
702:         "top_flags": flag_counts.most_common(8),
703:     }
704:
705:
706: def summarize_dataset_cached(path: Path) -> dict[str, Any]:
707:     cache_path = cached_json_path(path, "summary")
708:     cached = read_json(cache_path)
709:     if is_cache_current(path, cached):
710:         return cached["summary"]
711:     summary = summarize_dataset(path)
712:     write_json(
713:         cache_path,
714:         {
715:             "cached_at": utc_now(),
716:             "path": relative_path(path),
717:             "signature": file_signature(path),
718:             "summary": summary,
719:         },
720:     )

```

Extended core chunk 11: `product\_terminal.py` lines 721-792

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

721:     return summary
722:
723:
724: def summarize_all_datasets() -> dict[str, Any]:
725:     return {"train": summarize_dataset_cached(TRAIN_CSV), "test": summarize_dataset_cached(TEST_CSV)}
726:
727:
728: def inspect_csv(path: Path, limit: int | None = 50000) -> dict[str, Any]:
729:     cache_path = cached_json_path(path, f"inspect-{limit or 'all'}")
730:     cached = read_json(cache_path)
731:     if limit is None and is_cache_current(path, cached):

```

```

732:         return cached["inspection"]
733:
734:     rows = 0
735:     malformed = 0
736:     columns = 0
737:     first_row: list[str] | None = None
738:     header: list[str] | None = None
739:     label_counts: Counter[str] = Counter()
740:     column_counters: list[Counter[str]] = []
741:
742:     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
743:         reader = csv.reader(handle)
744:         for raw_index, row in enumerate(reader):
745:             if not row:
746:                 malformed += 1
747:                 continue
748:             if first_row is None:
749:                 first_row = row
750:                 columns = len(row)
751:                 header = row if likely_header(row) else None
752:                 column_counters = [Counter() for _ in range(columns)]
753:                 if header:
754:                     continue
755:             if columns and len(row) != columns:
756:                 malformed += 1
757:             rows += 1
758:             if row:
759:                 label_counts[row[0].strip()] += 1
760:             for index, value in enumerate(row[:columns]):
761:                 if len(column_counters[index]) < 2000:
762:                     column_counters[index][value.strip()] += 1
763:             if limit is not None and rows >= limit:
764:                 break
765:
766:     field_names = header or [f"column_{index}" for index in range(columns)]
767:     top_values = []
768:     for index, counter in enumerate(column_counters[:20]):
769:         top_values.append(
770:             {
771:                 "column": field_names[index] if index < len(field_names) else f"column_{index}",
772:                 "top": counter.most_common(8),
773:             }
774:         )
775:
776:     inspection = {
777:         "path": relative_path(path),
778:         "rows_scanned": rows,
779:         "scan_limit": limit,
780:         "columns": columns,
781:         "has_header": bool(header),
782:         "size_mb": round(path.stat().st_size / (1024 * 1024), 2),
783:         "malformed_rows": malformed,
784:         "label_counts_first_column": dict(label_counts),
785:         "top_values": top_values,
786:     }
787:     cache_artifact("index", inspection)
788:     if limit is None:
789:         write_json(
790:             cache_path,
791:             {
792:                 "cached_at": utc_now(),

```

Extended core chunk 12: `product\_terminal.py` lines 793-864

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

793:         "path": relative_path(path),
794:         "signature": file_signature(path),
795:         "inspection": inspection,
796:     },
797: )
798: return inspection
799:
800:
801: def analyze_csv(

```

```

802:     source: Path,
803:     *,
804:     limit: int | None = 5000,
805:     export: bool = True,
806:     model: dict[str, Any] | None = None,
807: ) -> dict[str, Any]:
808:     ensure_product_dirs()
809:     model = model or load_or_learn_model()
810:     analysis_id = f"scan-{compact_timestamp()}"
811:     export_csv_path = EXPORTS_DIR / f"traffic_analysis_{compact_timestamp()}.csv"
812:     export_json_path = export_csv_path.with_suffix(".json")
813:
814:     total = 0
815:     malformed = 0
816:     actual_counts: Counter[str] = Counter()
817:     predicted_counts: Counter[str] = Counter()
818:     risk_counts: Counter[str] = Counter()
819:     family_counts: Counter[str] = Counter()
820:     protocol_counts: Counter[str] = Counter()
821:     service_counts: Counter[str] = Counter()
822:     flag_counts: Counter[str] = Counter()
823:     metrics = Counter()
824:     risk_sum = 0.0
825:
826:     fieldnames = [
827:         "analysis_id",
828:         "analyzed_at",
829:         "source_file",
830:         "row_number",
831:         "actual_label",
832:         "actual_name",
833:         "predicted_label",
834:         "predicted_name",
835:         "risk_score",
836:         "confidence",
837:         "risk_level",
838:         "family",
839:         "reasons",
840:         *[f"{CSV_FEATURE_PREFIX}{name}" for name in FEATURE_NAMES],
841:     ]
842:
843:     writer = None
844:     export_handle = None
845:     try:
846:         if export:
847:             export_handle = export_csv_path.open("w", encoding="utf-8", newline="")
848:             writer = csv.DictWriter(export_handle, fieldnames=fieldnames)
849:             writer.writeheader()
850:
851:         with source.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
852:             reader = csv.reader(handle)
853:             for row_number, row in enumerate(reader, start=1):
854:                 if len(row) < len(FEATURE_NAMES) + 1:
855:                     malformed += 1
856:                     continue
857:                 label = row[0].strip()
858:                 features = [safe_float(value) for value in row[1 : len(FEATURE_NAMES) + 1]]
859:                 result = score_row(model, features)
860:                 total += 1
861:                 actual_counts[label] += 1
862:                 predicted_counts[result["predicted_label"]] += 1
863:                 risk_counts[result["risk_level"]] += 1
864:                 family_counts[result["family"]] += 1

```

Extended core chunk 13: `product\_terminal.py` lines 865-936

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

865:         protocol_counts[row[2].strip()] += 1
866:         service_counts[row[3].strip()] += 1
867:         flag_counts[row[4].strip()] += 1
868:         risk_sum += float(result["risk_score"])
869:
870:         if label in BINARY_LABELS:
871:             actual_attack = label == "1"

```

```

872:         predicted_attack = result["predicted_label"] == "1"
873:         if actual_attack and predicted_attack:
874:             metrics["tp"] += 1
875:         elif actual_attack and not predicted_attack:
876:             metrics["fn"] += 1
877:         elif not actual_attack and predicted_attack:
878:             metrics["fp"] += 1
879:         else:
880:             metrics["tn"] += 1
881:
882:     if writer:
883:         output_row = {
884:             "analysis_id": analysis_id,
885:             "analyzed_at": utc_now(),
886:             "source_file": relative_path(source),
887:             "row_number": row_number,
888:             "actual_label": label,
889:             "actual_name": BINARY_LABELS.get(label, "unknown"),
890:             **result,
891:         }
892:         output_row["reasons"] = result["reasons"]
893:         for index, name in enumerate(FEATURE_NAMES):
894:             output_row[f"{CSV_FEATURE_PREFIX}{name}"] = features[index]
895:         writer.writerow(output_row)
896:
897:     if limit is not None and total >= limit:
898:         break
899: finally:
900:     if export_handle:
901:         export_handle.close()
902:
903: precision = metrics["tp"] / max(metrics["tp"] + metrics["fp"], 1)
904: recall = metrics["tp"] / max(metrics["tp"] + metrics["fn"], 1)
905: accuracy = (metrics["tp"] + metrics["tn"]) / max(sum(metrics.values()), 1)
906: f1 = (2 * precision * recall) / max(precision + recall, 1e-12)
907: summary = {
908:     "analysis_id": analysis_id,
909:     "created_at": utc_now(),
910:     "source_file": relative_path(source),
911:     "rows_analyzed": total,
912:     "malformed_rows": malformed,
913:     "limit": limit,
914:     "average_risk_score": round(risk_sum / total, 6) if total else 0,
915:     "actual_counts": dict(actual_counts),
916:     "predicted_counts": dict(predicted_counts),
917:     "risk_counts": dict(risk_counts),
918:     "family_counts": dict(family_counts),
919:     "top_protocols": protocol_counts.most_common(8),
920:     "top_services": service_counts.most_common(8),
921:     "top_flags": flag_counts.most_common(8),
922:     "metrics": {
923:         "accuracy": round(accuracy, 6),
924:         "precision": round(precision, 6),
925:         "recall": round(recall, 6),
926:         "f1": round(f1, 6),
927:         "tp": metrics["tp"],
928:         "tn": metrics["tn"],
929:         "fp": metrics["fp"],
930:         "fn": metrics["fn"],
931:     },
932:     "model_created_at": model.get("created_at"),
933:     "export_csv": relative_path(export_csv_path) if export else None,
934:     "export_json": relative_path(export_json_path) if export else None,
935: }
936: if export:

```

Extended core chunk 14: `product\_terminal.py` lines 937-1008

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

937:     write_json(export_json_path, summary)
938:     cache_artifact("scan", summary)
939:     return summary
940:
941:

```

```

942: def latest_export_summary() -> dict[str, Any] | None:
943:     if not EXPORTS_DIR.exists():
944:         return None
945:     summaries = sorted(EXPORTS_DIR.glob("traffic_analysis_*.json"), key=lambda item: item.stat().st_mtime, reverse=True)
946:     if not summaries:
947:         return None
948:     return read_json(summaries[0])
949:
950:
951: def list_reports(limit: int | None = 20) -> list[dict[str, Any]]:
952:     if not EXPORTS_DIR.exists():
953:         return []
954:     reports = []
955:     for path in sorted(EXPORTS_DIR.glob("*"), key=lambda item: item.stat().st_mtime, reverse=True):
956:         if not path.is_file():
957:             continue
958:         reports.append(
959:             {
960:                 "name": path.name,
961:                 "path": relative_path(path),
962:                 "size_kb": round(path.stat().st_size / 1024, 2),
963:                 "modified": datetime.fromtimestamp(path.stat().st_mtime).isoformat(timespec="seconds"),
964:             }
965:         )
966:         if limit is not None and len(reports) >= limit:
967:             break
968:     return reports
969:
970:
971: def list_cache_artifacts(limit: int = 40) -> list[dict[str, Any]]:
972:     ensure_product_dirs()
973:     artifacts = []
974:     for path in sorted(COMMAND_CACHE_DIR.glob("*.json"), key=lambda item: item.stat().st_mtime, reverse=True):
975:         artifacts.append(
976:             {
977:                 "name": path.name,
978:                 "path": relative_path(path),
979:                 "size_kb": round(path.stat().st_size / 1024, 2),
980:                 "modified": datetime.fromtimestamp(path.stat().st_mtime).isoformat(timespec="seconds"),
981:             }
982:         )
983:         if limit is not None and len(artifacts) >= limit:
984:             break
985:     return artifacts
986:
987:
988: def show_cache(json_output: bool = False, limit: int = 40) -> None:
989:     payload = {
990:         "cache_dir": relative_path(CACHE_DIR),
991:         "index_dir": relative_path(INDEX_DIR),
992:         "command_cache_dir": relative_path(COMMAND_CACHE_DIR),
993:         "artifacts": list_cache_artifacts(limit),
994:     }
995:     if json_output:
996:         print_json(payload)
997:         return
998:     section("IDS Sentinel Terminal Cache")
999:     print(f"Cache:    {payload['cache_dir']}")
1000:     print(f"Indexes: {payload['index_dir']}")
1001:     print(f"Runs:    {payload['command_cache_dir']}")
1002:     if not payload["artifacts"]:
1003:         print("No command cache artifacts yet.")
1004:         return
1005:     print()
1006:     print(table(["Name", "Path", "Size KB", "Modified"], [
1007:         [item["name"], item["path"], item["size_kb"], item["modified"]] for item in payload["artifacts"]
1008:     ]))

```

Extended core chunk 15: `product\_terminal.py` lines 1009-1080

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

1009:
1010:
1011: def list_run_summaries(limit: int | None = 8) -> list[dict[str, Any]]:

```

```

1012:     runs_dir = ROOT_DIR / "automation" / "runs"
1013:     if not runs_dir.exists():
1014:         return []
1015:     rows = []
1016:     for path in sorted(runs_dir.glob("*/summary.json"), key=lambda item: item.stat().st_mtime, reverse=True):
1017:         payload = read_json(path)
1018:         if payload:
1019:             rows.append(payload)
1020:             if limit is not None and len(rows) >= limit:
1021:                 break
1022:     return rows
1023:
1024:
1025: def show_dataset_catalog(json_output: bool = False) -> None:
1026:     payload = {
1027:         "local_sources": [relative_path(path) for path in all_csv_sources(include_exports=False)],
1028:         "external_catalog": EXTERNAL_DATASETS,
1029:     }
1030:     cache_artifact("datasets", payload)
1031:     if json_output:
1032:         print_json(payload)
1033:         return
1034:     section("Dataset Catalog")
1035:     print(table(["ID", "Name", "Source", "Format"], [
1036:         [item["id"], item["name"], item["source"], item["format"]] for item in EXTERNAL_DATASETS
1037:     ]))
1038:     print()
1039:     print(table(["Local CSV", "Size MB"], [
1040:         [relative_path(path), round(path.stat().st_size / (1024 * 1024), 2)] for path in all_csv_sources(include_exports=False)
1041:     ]))
1042:
1043:
1044: def import_csv(source: Path, name: str | None = None) -> Path:
1045:     ensure_product_dirs()
1046:     if not source.exists() or not source.is_file():
1047:         raise ValueError(f"not a file: {source}")
1048:     if source.suffix.lower() != ".csv":
1049:         raise ValueError("only CSV imports are supported in IDS Sentinel Terminal")
1050:     safe_name = re.sub(r"^[A-Za-z0-9_-]+", "_", name or source.name)
1051:     if not safe_name.lower().endswith(".csv"):
1052:         safe_name += ".csv"
1053:     target = IMPORTS_DIR / safe_name
1054:     if source.resolve() != target.resolve():
1055:         shutil.copy2(source, target)
1056:     inspect_csv(target, limit=None)
1057:     cache_artifact("import", {"source": str(source), "target": relative_path(target), "bytes": target.stat().st_size})
1058:     return target
1059:
1060:
1061: def download_url(url: str, name: str | None = None, max_bytes: int = 2 * 1024 * 1024 * 1024) -> Path:
1062:     ensure_product_dirs()
1063:     parsed_name = name or Path(url.split("?", 1)[0]).name or f"download_{compact_timestamp()}"
1064:     safe_name = re.sub(r"^[A-Za-z0-9_-]+", "_", parsed_name)
1065:     target = IMPORTS_DIR / safe_name
1066:     try:
1067:         with urllib.request.urlopen(url, timeout=60) as response, target.open("wb") as handle:
1068:             copied = 0
1069:             while True:
1070:                 chunk = response.read(1024 * 1024)
1071:                 if not chunk:
1072:                     break
1073:                 copied += len(chunk)
1074:                 if copied > max_bytes:
1075:                     raise RuntimeError("download exceeded the 2 GB safety limit")
1076:                 handle.write(chunk)
1077:     except Exception:
1078:         target.unlink(missing_ok=True)
1079:         raise
1080:     cache_artifact("download", {"url": url, "path": relative_path(target), "bytes": target.stat().st_size})

```

Extended core chunk 16: `product\_terminal.py` lines 1081-1152

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

1081:     return target

```

```

1082:
1083:
1084: def show_import(path: Path, json_output: bool = False) -> None:
1085:     payload = {"imported_path": relative_path(path), "inspection": inspect_csv(path, limit=None)}
1086:     if json_output:
1087:         print_json(payload)
1088:         return
1089:     section("CSV Imported")
1090:     print(f"Imported: {payload['imported_path']}")
1091:     show_index(path, json_output=False)
1092:
1093:
1094: def show_index(path: Path, json_output: bool = False, limit: int | None = 50000) -> None:
1095:     payload = inspect_csv(path, limit=limit)
1096:     if json_output:
1097:         print_json(payload)
1098:         return
1099:     section("CSV Index")
1100:     print(f"Path: {payload['path']}")
1101:     print(f"Rows scanned: {payload['rows_scanned']:,} | columns: {payload['columns']} | size: {payload['size_mb']} MB")
1102:     print(f"Header: {payload['has_header']} | malformed rows: {payload['malformed_rows']:,}")
1103:     if payload["label_counts_first_column"]:
1104:         print()
1105:         print(table(["First Column Value", "Rows"], [
1106:             [label, count] for label, count in sorted(payload["label_counts_first_column"].items(), key=lambda item: item[1])
1107:         ]))
1108:         print()
1109:         print(table(["Column", "Top Values"], [
1110:             [item["column"], ", ".join(f"{value}:{count}" for value, count in item["top"][:5])] for item in payload["top_value"]
1111:         ]))
1112:
1113:
1114: def load_services() -> dict[int, str]:
1115:     services: dict[int, str] = {
1116:         20: "ftp-data",
1117:         21: "ftp",
1118:         22: "ssh",
1119:         23: "telnet",
1120:         25: "smtp",
1121:         53: "domain",
1122:         80: "http",
1123:         110: "pop3",
1124:         135: "msrpc",
1125:         139: "netbios-ssn",
1126:         143: "imap",
1127:         443: "https",
1128:         445: "microsoft-ds",
1129:         1433: "ms-sql-s",
1130:         3306: "mysql",
1131:         3389: "ms-wbt-server",
1132:         5432: "postgresql",
1133:         6379: "redis",
1134:         8080: "http-alt",
1135:         9200: "elasticsearch",
1136:         27017: "mongodb",
1137:     }
1138:     services_file = Path(os.environ.get("SystemRoot", "C:\\Windows")) / "System32" / "drivers" / "etc" / "services"
1139:     if services_file.exists():
1140:         with services_file.open("r", encoding="utf-8", errors="ignore") as handle:
1141:             for line in handle:
1142:                 line = line.strip()
1143:                 if not line or line.startswith("#"):
1144:                     continue
1145:                 parts = line.split()
1146:                 if len(parts) < 2 or "/" not in parts[1]:
1147:                     continue
1148:                 port_text, proto = parts[1].split("/", 1)
1149:                 if proto.lower() not in {"tcp", "udp"}:
1150:                     continue
1151:                 try:
1152:                     services.setdefault(int(port_text), parts[0])

```

Extended core chunk 17: `product\_terminal.py` lines 1153-1224

These extended chunks are included because the repository's primary orchestration logic is too central to summarize responsibly in only a few excerpts. They allow a reader to inspect the surrounding implementation context directly.

```

1153:         except ValueError:
1154:             continue
1155:     return services
1156:
1157:
1158: def parse_ports(text: str) -> list[int]:
1159:     if text.lower() in {"common", "top"}:
1160:         return COMMON_PROBE_PORTS
1161:     ports: set[int] = set()
1162:     for part in text.split(","):
1163:         part = part.strip()
1164:         if not part:
1165:             continue
1166:         if "-" in part:
1167:             start_text, end_text = part.split("-", 1)
1168:             start = int(start_text)
1169:             end = int(end_text)
1170:             if end < start:
1171:                 start, end = end, start
1172:             for port in range(start, min(end, start + 127) + 1):
1173:                 if 1 <= port <= 65535:
1174:                     ports.add(port)
1175:         else:
1176:             port = int(part)
1177:             if 1 <= port <= 65535:
1178:                 ports.add(port)
1179:     if len(ports) > 128:
1180:         raise ValueError("port list is capped at 128 ports per probe")
1181:     return sorted(ports)
1182:
1183:
1184: def split_host_port(value: str) -> tuple[str, int | None]:
1185:     value = value.strip()
1186:     if value.startswith("[") and "]" in value:
1187:         host, port_text = value.rsplit(":", 1)
1188:         return host.strip("[]"), int(port_text) if port_text.isdigit() else None
1189:     if ":" in value and value.count(":") == 1:
1190:         host, port_text = value.rsplit(":", 1)
1191:         return host, int(port_text) if port_text.isdigit() else None
1192:     return value, None
1193:
1194:
1195: def parse_netstat() -> list[dict[str, Any]]:
1196:     try:
1197:         completed = subprocess.run(["netstat", "-ano"], capture_output=True, text=True, timeout=15, check=False)
1198:     except FileNotFoundError:
1199:         return []
1200:     rows = []
1201:     for line in completed.stdout.splitlines():
1202:         parts = line.split()
1203:         if not parts or parts[0] not in {"TCP", "UDP"}:
1204:             continue
1205:         proto = parts[0]
1206:         if proto == "TCP" and len(parts) >= 5:
1207:             local_host, local_port = split_host_port(parts[1])
1208:             remote_host, remote_port = split_host_port(parts[2])
1209:             rows.append(
1210:                 {
1211:                     "proto": proto,
1212:                     "local": parts[1],
1213:                     "local_host": local_host,
1214:                     "local_port": local_port,
1215:                     "remote": parts[2],
1216:                     "remote_host": remote_host,
1217:                     "remote_port": remote_port,
1218:                     "state": parts[3],
1219:                     "pid": parts[4],
1220:                 }
1221:             )
1222:         elif proto == "UDP" and len(parts) >= 4:
1223:             local_host, local_port = split_host_port(parts[1])
1224:             rows.append(

```

Appendix F. Live Output Evidence

Status command output

```
status terminal
01 IDS Sentinel Terminal Status
02 =====
03 Version: 0.2.0 | mode: source | home: C:\Users\danny\Desktop\Intrusion-Detection-Systems-master\IDS-MachineLearning
04
05 Set      Path      Rows      Size      Normal  Attack  Attack Share
06 ----      -
07 train  kddtrain.csv  494,021  46.13 MB  97,278  396,743  80.31%
08 test   kddtest.csv   311,029  29.04 MB  60,593  250,436  80.52%
09
10 Self-Learning Model
11 =====
12 Model: streaming_gaussian_profile | rows learned: 505,221 | created: 2026-05-06T04:46:09+00:00
13 Indicator                               Separation  Normal Mean  Attack Mean
14 -----
15 count                                3.6574      8.1411      413.3894
16 srv_count                            2.2142      10.9263     364.7701
17 logged_in                            2.1930      0.7190      0.0081
18 service                              2.0655      12.6203     45.6293
19 protocol_type                        1.7128      1.2239      2.4378
20 dst_host_same_src_port_rate          1.5821      0.1323      0.7220
21
22 Latest Downloadable Analysis
23 =====
24 CSV:  automation\product\exports\traffic_analysis_20260506_101531.csv
25 JSON: automation\product\exports\traffic_analysis_20260506_101531.json
26 Rows: 5,000 | average risk: 0.8381
```

Figure. Status command output rendered from a live command invocation in the report environment.

Attacks command output

```

attacks terminal

01 Attack Distribution
02 =====
03 Set      Label    Rows      Share
04 -----
05 train    normal    97,278    19.69%
06 train    attack    396,743   80.31%
07 test     normal    60,593    19.48%
08 test     attack    250,436   80.52%
09
10 Learned Attack Indicators
11 =====
12 Feature                                Separation    Normal Mean    Attack Mean
13 -----
14 count                                3.6574        8.1411        413.3894
15 srv_count                            2.2142        10.9263       364.7701
16 logged_in                            2.1930        0.7190        0.0081
17 service                              2.0655        12.6203       45.6293
18 protocol_type                        1.7128        1.2239        2.4378
19 dst_host_same_src_port_rate          1.5821        0.1323        0.7220
20 dst_host_count                       1.3940        149.0967      253.0713
21 same_srv_rate                        0.7865        0.9856        0.7496
22 dst_host_srv_serror_rate             0.7346        0.0012        0.2146
23 serror_rate                          0.7334        0.0016        0.2148

```

Figure. Attacks command output rendered from a live command invocation in the report environment.

Ports command output

```

ports terminal

01 Network Connections
02 =====
03 Proto    Local                                Service      Remote       State        PID
04 -----
05 UDP      0.0.0.0:53                          domain       *:.*         UDP          7524
06 TCP      0.0.0.0:135                        msrpc        0.0.0.0:0    LISTENING    1752
07 TCP      [::]:135                          msrpc        [::]:0       LISTENING    1752
08 UDP      172.25.144.1:137                   netbios-ns   *:.*         UDP          4
09 UDP      192.168.1.201:137                  netbios-ns   *:.*         UDP          4
10 UDP      172.25.144.1:138                   netbios-dgm  *:.*         UDP          4
11 UDP      192.168.1.201:138                  netbios-dgm  *:.*         UDP          4
12 TCP      172.25.144.1:139                   netbios-ssn  0.0.0.0:0    LISTENING    4
13 TCP      192.168.1.201:139                  netbios-ssn  0.0.0.0:0    LISTENING    4
14 TCP      0.0.0.0:445                        microsoft-ds  0.0.0.0:0    LISTENING    4
15 TCP      [::]:445                          microsoft-ds  [::]:0       LISTENING    4
16 TCP      127.0.0.1:1042                     0.0.0.0:0    LISTENING    17384
17 TCP      127.0.0.1:1043                     0.0.0.0:0    LISTENING    17384
18 UDP      0.0.0.0:1900                       ssdp         *:.*         UDP          33920
19 UDP      0.0.0.0:1900                       ssdp         *:.*         UDP          33920
20 UDP      127.0.0.1:1900                     ssdp         *:.*         UDP          9328
21 UDP      172.25.144.1:1900                  ssdp         *:.*         UDP          9328
22 UDP      192.168.1.201:1900                 ssdp         *:.*         UDP          9328
23 UDP      [::1]:1900                         ssdp         *:.*         UDP          9328
24 UDP      [fe80::22d6:a544:1365:aaf%45]:1900 ssdp         *:.*         UDP          9328

```

Figure. Ports command output rendered from a live command invocation in the report environment.

Appendix G. Primary JSON Evidence Dumps

status --json

This dump is included verbatim from a live command invocation so that readers can inspect the exact machine-readable structure behind the narrative summaries used earlier in the report.

```
1: {
2:   "installation": {
3:     "version": "0.2.0",
4:     "runtime_mode": "source",
5:     "home_dir": "C:\\Users\\danny\\Desktop\\Intrusion-Detection-Systems-master\\IDS-MachineLearning",
6:     "env_override": false
7:   },
8:   "datasets": {
9:     "train": {
10:      "path": "kddtrain.csv",
11:      "rows": 494021,
12:      "malformed_rows": 0,
13:      "columns": 42,
14:      "size_mb": 46.13,
15:      "label_counts": {
16:        "0": 97278,
17:        "1": 396743
18:      },
19:      "attack_share": 0.803089,
20:      "normal_share": 0.196911,
21:      "total_src_bytes": 1494715024,
22:      "total_dst_bytes": 429073257,
23:      "top_protocols": [
24:        [
25:          "3",
26:          283602
27:        ],
28:        [
29:          "1",
30:          190065
31:        ],
32:        [
33:          "2",
34:          20354
35:        ]
36:      ],
37:      "top_services": [
38:        [
39:          "54",
40:          281400
41:        ],
42:        [
43:          "24",
44:          110893
45:        ],
46:        [
47:          "1",
48:          64293
49:        ],
50:        [
51:          "19",
52:          9723
53:        ],
54:        [
55:          "29",
56:          7237
57:        ],
58:        [
59:          "57",
60:          5863
61:        ],
62:        [
63:          "49",
64:          4721
65:        ],
66:        [
67:          "55",
68:          1642
69:        ]
70:      ],
71:      "top_flags": [
72:        [
73:          "1",
```

```

74:         378440
75:     ],
76:     [
77:         "6",
78:         87007
79:     ],
80:     [
81:         "10",
82:         26875
83:     ],
84:     [
85:         "7",
86:         903
87:     ],
88:     [
89:         "9",
90:         579
91:     ],
92:     [
93:         "2",
94:         107
95:     ],
96:     [
97:         "5",
98:         57
99:     ],
100:    [
101:        "4",
102:        32
103:    ]
104: ],
105: },
106: "test": {
107:     "path": "kddtest.csv",
108:     "rows": 311029,
109:     "malformed_rows": 0,
110:     "columns": 42,
111:     "size_mb": 29.04,
112:     "label_counts": {
113:         "0": 60593,
114:         "1": 250436
115:     },
116:     "attack_share": 0.805185,
117:     "normal_share": 0.194815,
118:     "total_src_bytes": 538609601,
119:     "total_dst_bytes": 232647728,
120:     "top_protocols": [
121:         [
122:             "3",
123:             164969
124:         ],
125:         [
126:             "1",
127:             119357
128:         ],
129:         [
130:             "2",
131:             26703
132:         ]
133:     ],
134:     "top_services": [
135:         [
136:             "54",
137:             164352
138:         ],
139:         [
140:             "24",
141:             78510
142:         ],
143:         [
144:             "1",
145:             41237
146:         ],
147:         [
148:             "19",
149:             8268
150:         ],
151:         [
152:             "26",
153:             3972
154:         ],
155:         [
156:             "57",
157:             3160
158:         ],

```

```

159:      [
160:        "49",
161:        2223
162:      ],
163:      [
164:        "29",
165:        2185
166:      ]
167:    ],
168:    "top_flags": [
169:      [
170:        "1",
171:        248379
172:      ],
173:      [
174:        "10",
175:        41945
176:      ],
177:      [
178:        "6",
179:        18012
180:      ],
181:      [
182:        "9",
183:        1393
184:      ],
185:      [
186:        "7",
187:        872
188:      ],
189:      [
190:        "3",
191:        289
192:      ],
193:      [
194:        "2",
195:        84
196:      ],
197:      [
198:        "5",
199:        27
200:      ]
201:    ]
202:  },
203: },
204: "model": {
205:   "version": 1,
206:   "created_at": "2026-05-06T04:46:09+00:00",
207:   "model_type": "streaming_gaussian_profile",
208:   "description": "Pure-Python self-learning profile built from labeled IDS CSV rows and terminal-generated analysis exports.",
209:   "features": [
210:     "duration",
211:     "protocol_type",
212:     "service",
213:     "flag",
214:     "src_bytes",
215:     "dst_bytes",
216:     "land",
217:     "wrong_fragment",
218:     "urgent",
219:     "hot",
220:     "num_failed_logins",
221:     "logged_in",
222:     "num_compromised",
223:     "root_shell",
224:     "su_attempted",
225:     "num_root",
226:     "num_file_creations",
227:     "num_shells",
228:     "num_access_files",
229:     "num_outbound_cmds",
230:     "is_host_login",
231:     "is_guest_login",
232:     "count",
233:     "srv_count",
234:     "serror_rate",
235:     "srv_serror_rate",
236:     "rerror_rate",
237:     "srv_rerror_rate",
238:     "same_srv_rate",
239:     "diff_srv_rate",
240:     "srv_diff_host_rate",
241:     "dst_host_count",
242:     "dst_host_srv_count",
243:     "dst_host_same_srv_rate",

```

```

244:     "dst_host_diff_srv_rate",
245:     "dst_host_same_src_port_rate",
246:     "dst_host_srv_diff_host_rate",
247:     "dst_host_serror_rate",
248:     "dst_host_srv_serror_rate",
249:     "dst_host_rerror_rate",
250:     "dst_host_srv_rerror_rate"
251: ],
252: "labels": {
253:   "0": {
254:     "name": "normal",
255:     "count": 99541,
256:     "prior": 0.19702466841243732,
257:     "features": [
258:       {
259:         "count": 99541,
260:         "mean": 211.92420209,
261:         "variance": 1807562.559123,
262:         "min": 0.0,
263:         "max": 58329.0
264:       },
265:       {
266:         "count": 99541,
267:         "mean": 1.22390774,
268:         "variance": 0.19999542,
269:         "min": 1.0,
270:         "max": 3.0
271:       },
272:       {
273:         "count": 99541,
274:         "mean": 12.62034739,
275:         "variance": 321.02732116,
276:         "min": 1.0,
277:         "max": 65.0
278:       },
279:       {
280:         "count": 99541,
281:         "mean": 1.49614732,
282:         "variance": 4.1817337,
283:         "min": 1.0,
284:         "max": 10.0
285:       },
286:       {
287:         "count": 99541,
288:         "mean": 1151.5230508,
289:         "variance": 1147351938.380843,
290:         "min": 0.0,
291:         "max": 2194619.0
292:       },
293:       {
294:         "count": 99541,
295:         "mean": 3373.12601842,
296:         "variance": 1385143722.3633213,
297:         "min": 0.0,
298:         "max": 5134218.0
299:       },
300:       {
301:         "count": 99541,
302:         "mean": 1.005e-05,
303:         "variance": 1.005e-05,
304:         "min": 0.0,
305:         "max": 1.0
306:       },
307:       {
308:         "count": 99541,
309:         "mean": 0.0,
310:         "variance": 0.0,
311:         "min": 0.0,
312:         "max": 0.0
313:       },
314:       {
315:         "count": 99541,
316:         "mean": 3.014e-05,
317:         "variance": 9.042e-05,
318:         "min": 0.0,
319:         "max": 3.0
320:       },
321:       {
322:         "count": 99541,
323:         "mean": 0.04475543,
324:         "variance": 0.73797081,
325:         "min": 0.0,
326:         "max": 30.0
327:       },
328:     ]

```

```

329:         "count": 99541,
330:         "mean": 0.00018083,
331:         "variance": 0.00042191,
332:         "min": 0.0,
333:         "max": 4.0
334:     },
335:     {
336:         "count": 99541,
337:         "mean": 0.71896003,
338:         "variance": 0.20205854,
339:         "min": 0.0,
340:         "max": 1.0
341:     },
342:     {
343:         "count": 99541,
344:         "mean": 0.02788801,
345:         "variance": 16.00452665,
346:         "min": 0.0,
347:         "max": 884.0
348:     },
349:     {
350:         "count": 99541,
351:         "mean": 0.00023106,
352:         "variance": 0.00023101,
353:         "min": 0.0,
354:         "max": 1.0
355:     },
356:     {
357:         "count": 99541,
358:         "mean": 0.00017078,
359:         "variance": 0.00029131,
360:         "min": 0.0,
361:         "max": 2.0
362:     },
363:     {
364:         "count": 99541,
365:         "mean": 0.05481159,
366:         "variance": 20.05283251,
367:         "min": 0.0,
368:         "max": 993.0
369:     },
370:     {
371:         "count": 99541,
372:         "mean": 0.00461117,
373:         "variance": 0.04023391,
374:         "min": 0.0,
375:         "max": 28.0
376:     },
377:     {
378:         "count": 99541,
379:         "mean": 0.00043198,
380:         "variance": 0.0004318,
381:         "min": 0.0,
382:         "max": 1.0
383:     },
384:     {
385:         "count": 99541,
386:         "mean": 0.00498287,
387:         "variance": 0.00654539,
388:         "min": 0.0,
389:         "max": 8.0
390:     },
391:     {
392:         "count": 99541,
393:         "mean": 0.0,
394:         "variance": 0.0,
395:         "min": 0.0,
396:         "max": 0.0
397:     },
398:     {
399:         "count": 99541,
400:         "mean": 0.0,
401:         "variance": 0.0,
402:         "min": 0.0,
403:         "max": 0.0
404:     },
405:     {
406:         "count": 99541,
407:         "mean": 0.00385771,
408:         "variance": 0.00384286,
409:         "min": 0.0,
410:         "max": 1.0
411:     },
412:     {
413:         "count": 99541,

```

```

414:         "mean": 8.1410675,
415:         "variance": 308.24767059,
416:         "min": 0.0,
417:         "max": 511.0
418:     },
419:     {
420:         "count": 99541,
421:         "mean": 10.92626154,
422:         "variance": 467.71101828,
423:         "min": 0.0,
424:         "max": 510.0
425:     },
426:     {
427:         "count": 99541,
428:         "mean": 0.00160225,
429:         "variance": 0.00079136,
430:         "min": 0.0,
431:         "max": 1.0
432:     },
433:     {
434:         "count": 99541,
435:         "mean": 0.0018085,
436:         "variance": 0.00072296,
437:         "min": 0.0,
438:         "max": 1.0
439:     },
440:     {
441:         "count": 99541,
442:         "mean": 0.05472951,
443:         "variance": 0.05139123,
444:         "min": 0.0,
445:         "max": 1.0
446:     },
447:     {
448:         "count": 99541,
449:         "mean": 0.05501843,
450:         "variance": 0.05085975,
451:         "min": 0.0,
452:         "max": 1.0
453:     },
454:     {
455:         "count": 99541,
456:         "mean": 0.9856208,
457:         "variance": 0.00843631,
458:         "min": 0.0,
459:         "max": 1.0
460:     },
461:     {
462:         "count": 99541,
463:         "mean": 0.01816337,
464:         "variance": 0.01357084,
465:         "min": 0.0,
466:         "max": 1.0
467:     },
468:     {
469:         "count": 99541,
470:         "mean": 0.13470459,
471:         "variance": 0.07744364,
472:         "min": 0.0,
473:         "max": 1.0
474:     },
475:     {
476:         "count": 99541,
477:         "mean": 149.09667373,
478:         "variance": 10685.50599466,
479:         "min": 0.0,
480:         "max": 255.0
481:     },
482:     {
483:         "count": 99541,
484:         "mean": 202.56859987,
485:         "variance": 7505.25395636,
486:         "min": 0.0,
487:         "max": 255.0
488:     },
489:     {
490:         "count": 99541,
491:         "mean": 0.84665987,
492:         "variance": 0.09222341,
493:         "min": 0.0,
494:         "max": 1.0
495:     },
496:     {
497:         "count": 99541,
498:         "mean": 0.05568992,

```

```

499:         "variance": 0.03187202,
500:         "min": 0.0,
501:         "max": 1.0
502:     },
503:     {
504:         "count": 99541,
505:         "mean": 0.13231563,
506:         "variance": 0.07791054,
507:         "min": 0.0,
508:         "max": 1.0
509:     },
510:     {
511:         "count": 99541,
512:         "mean": 0.02385771,
513:         "variance": 0.00241202,
514:         "min": 0.0,
515:         "max": 1.0
516:     },
517:     {
518:         "count": 99541,
519:         "mean": 0.00214344,
520:         "variance": 0.00087347,
521:         "min": 0.0,
522:         "max": 1.0
523:     },
524:     {
525:         "count": 99541,
526:         "mean": 0.0011756,
527:         "variance": 0.00033946,
528:         "min": 0.0,
529:         "max": 1.0
530:     },
531:     {
532:         "count": 99541,
533:         "mean": 0.05658201,
534:         "variance": 0.04962295,
535:         "min": 0.0,
536:         "max": 1.0
537:     },
538:     {
539:         "count": 99541,
540:         "mean": 0.05468711,
541:         "variance": 0.04695941,
542:         "min": 0.0,
543:         "max": 1.0
544:     }
545: ],
546: "1": {
547:     "name": "attack",
548:     "count": 405680,
549:     "prior": 0.8029753315875626,
550:     "features": [
551:         {
552:             "count": 405680,
553:             "mean": 6.49462384,
554:             "variance": 158498.59359808,
555:             "min": 0.0,
556:             "max": 42448.0
557:         },
558:         {
559:             "count": 405680,
560:             "mean": 2.4377884,
561:             "variance": 0.80456692,
562:             "min": 1.0,
563:             "max": 3.0
564:         },
565:         {
566:             "count": 405680,
567:             "mean": 45.62930142,
568:             "variance": 189.74896777,
569:             "min": 1.0,
570:             "max": 70.0
571:         },
572:         {
573:             "count": 405680,
574:             "mean": 2.57305758,
575:             "variance": 7.34395105,
576:             "min": 1.0,
577:             "max": 10.0
578:         },
579:         {
580:             "count": 405680,
581:             "mean": 3673.10266959,
582:             "variance": 1196368805916.1626,

```

```

584:         "min": 0.0,
585:         "max": 693375640.0
586:     },
587:     {
588:         "count": 405680,
589:         "mean": 248.68854023,
590:         "variance": 989218365.7204053,
591:         "min": 0.0,
592:         "max": 5155468.0
593:     },
594:     {
595:         "count": 405680,
596:         "mean": 5.176e-05,
597:         "variance": 5.176e-05,
598:         "min": 0.0,
599:         "max": 1.0
600:     },
601:     {
602:         "count": 405680,
603:         "mean": 0.00783376,
604:         "variance": 0.02211873,
605:         "min": 0.0,
606:         "max": 3.0
607:     },
608:     {
609:         "count": 405680,
610:         "mean": 9.86e-06,
611:         "variance": 1.479e-05,
612:         "min": 0.0,
613:         "max": 2.0
614:     },
615:     {
616:         "count": 405680,
617:         "mean": 0.03136216,
618:         "variance": 0.56816345,
619:         "min": 0.0,
620:         "max": 28.0
621:     },
622:     {
623:         "count": 405680,
624:         "mean": 0.0001405,
625:         "variance": 0.00018979,
626:         "min": 0.0,
627:         "max": 5.0
628:     },
629:     {
630:         "count": 405680,
631:         "mean": 0.00813696,
632:         "variance": 0.00807077,
633:         "min": 0.0,
634:         "max": 1.0
635:     },
636:     {
637:         "count": 405680,
638:         "mean": 0.00560047,
639:         "variance": 0.01114989,
640:         "min": 0.0,
641:         "max": 38.0
642:     },
643:     {
644:         "count": 405680,
645:         "mean": 7.888e-05,
646:         "variance": 7.887e-05,
647:         "min": 0.0,
648:         "max": 1.0
649:     },
650:     {
651:         "count": 405680,
652:         "mean": 2.46e-06,
653:         "variance": 2.46e-06,
654:         "min": 0.0,
655:         "max": 1.0
656:     },
657:     {
658:         "count": 405680,
659:         "mean": 0.00038207,
660:         "variance": 0.01232733,
661:         "min": 0.0,
662:         "max": 54.0
663:     },
664:     {
665:         "count": 405680,
666:         "mean": 0.00018734,
667:         "variance": 0.00144446,
668:         "min": 0.0,

```

```

669:         "max": 21.0
670:     },
671:     {
672:         "count": 405680,
673:         "mean": 2.711e-05,
674:         "variance": 4.19e-05,
675:         "min": 0.0,
676:         "max": 2.0
677:     },
678:     {
679:         "count": 405680,
680:         "mean": 2.711e-05,
681:         "variance": 3.204e-05,
682:         "min": 0.0,
683:         "max": 2.0
684:     },
685:     {
686:         "count": 405680,
687:         "mean": 0.0,
688:         "variance": 0.0,
689:         "min": 0.0,
690:         "max": 0.0
691:     },
692:     {
693:         "count": 405680,
694:         "mean": 0.0,
695:         "variance": 0.0,
696:         "min": 0.0,
697:         "max": 0.0
698:     },
699:     {
700:         "count": 405680,
701:         "mean": 0.00077401,
702:         "variance": 0.00077341,
703:         "min": 0.0,
704:         "max": 1.0
705:     },
706:     {
707:         "count": 405680,
708:         "mean": 413.38936107,
709:         "variance": 24245.67741018,
710:         "min": 0.0,
711:         "max": 511.0
712:     },
713:     {
714:         "count": 405680,
715:         "mean": 364.77010945,
716:         "variance": 50609.53320716,
717:         "min": 0.0,
718:         "max": 511.0
719:     },
720:     {
721:         "count": 405680,
722:         "mean": 0.21478165,
723:         "variance": 0.16820709,
724:         "min": 0.0,
725:         "max": 1.0
726:     },
727:     {
728:         "count": 405680,
729:         "mean": 0.21463705,
730:         "variance": 0.16853029,
731:         "min": 0.0,
732:         "max": 1.0
733:     },
734:     {
735:         "count": 405680,
736:         "mean": 0.05652603,
737:         "variance": 0.05282404,
738:         "min": 0.0,
739:         "max": 1.0
740:     },
741:     {
742:         "count": 405680,
743:         "mean": 0.05680285,
744:         "variance": 0.05325099,
745:         "min": 0.0,
746:         "max": 1.0
747:     },
748:     {
749:         "count": 405680,
750:         "mean": 0.74963119,
751:         "variance": 0.17164369,
752:         "min": 0.0,
753:         "max": 1.0

```

```

754:     },
755:     {
756:         "count": 405680,
757:         "mean": 0.02117787,
758:         "variance": 0.00497518,
759:         "min": 0.0,
760:         "max": 1.0
761:     },
762:     {
763:         "count": 405680,
764:         "mean": 0.00321019,
765:         "variance": 0.00284733,
766:         "min": 0.0,
767:         "max": 1.0
768:     },
769:     {
770:         "count": 405680,
771:         "mean": 253.07129018,
772:         "variance": 441.56695618,
773:         "min": 1.0,
774:         "max": 255.0
775:     },
776:     {
777:         "count": 405680,
778:         "mean": 186.87488415,
779:         "variance": 11934.84156807,
780:         "min": 1.0,
781:         "max": 255.0
782:     },
783:     {
784:         "count": 405680,
785:         "mean": 0.73728569,
786:         "variance": 0.18223473,
787:         "min": 0.0,
788:         "max": 1.0
789:     },
790:     {
791:         "count": 405680,
792:         "mean": 0.02411672,
793:         "variance": 0.0065874,
794:         "min": 0.0,
795:         "max": 1.0
796:     },
797:     {
798:         "count": 405680,
799:         "mean": 0.72200084,
800:         "variance": 0.19993789,
801:         "min": 0.0,
802:         "max": 1.0
803:     },
804:     {
805:         "count": 405680,
806:         "mean": 0.00235447,
807:         "variance": 0.00148341,
808:         "min": 0.0,
809:         "max": 1.0
810:     },
811:     {
812:         "count": 405680,
813:         "mean": 0.21473548,
814:         "variance": 0.16811924,
815:         "min": 0.0,
816:         "max": 1.0
817:     },
818:     {
819:         "count": 405680,
820:         "mean": 0.21460913,
821:         "variance": 0.16849133,
822:         "min": 0.0,
823:         "max": 1.0
824:     },
825:     {
826:         "count": 405680,
827:         "mean": 0.05694091,
828:         "variance": 0.05268787,
829:         "min": 0.0,
830:         "max": 1.0
831:     },
832:     {
833:         "count": 405680,
834:         "mean": 0.05653187,
835:         "variance": 0.05308389,
836:         "min": 0.0,
837:         "max": 1.0
838:     }

```

```

839:     ]
840:   }
841: },
842: "total_rows": 505221,
843: "sources": [
844:   {
845:     "path": "kddtrain.csv",
846:     "rows_used": 494021,
847:     "label_counts": {
848:       "0": 97278,
849:       "1": 396743
850:     }
851:   },
852:   {
853:     "path": "automation\\product\\exports\\traffic_analysis_20260506_081928.csv",
854:     "rows_used": 100,
855:     "label_counts": {
856:       "0": 77,
857:       "1": 23
858:     }
859:   },
860:   {
861:     "path": "automation\\product\\exports\\traffic_analysis_20260506_081949.csv",
862:     "rows_used": 100,
863:     "label_counts": {
864:       "0": 77,
865:       "1": 23
866:     }
867:   },
868:   {
869:     "path": "automation\\product\\exports\\traffic_analysis_20260506_082152.csv",
870:     "rows_used": 5000,
871:     "label_counts": {
872:       "0": 703,
873:       "1": 4297
874:     }
875:   },
876:   {
877:     "path": "automation\\product\\exports\\traffic_analysis_20260506_084127.csv",
878:     "rows_used": 1000,
879:     "label_counts": {
880:       "0": 703,
881:       "1": 297
882:     }
883:   },
884:   {
885:     "path": "automation\\product\\exports\\traffic_analysis_20260506_101531.csv",
886:     "rows_used": 5000,
887:     "label_counts": {
888:       "0": 703,
889:       "1": 4297
890:     }
891:   }
892: ],
893: "top_indicators": [
894:   {
895:     "feature": "count",
896:     "separation": 3.657428,
897:     "normal_mean": 8.141068,
898:     "attack_mean": 413.389361
899:   },
900:   {
901:     "feature": "srv_count",
902:     "separation": 2.21418,
903:     "normal_mean": 10.926262,
904:     "attack_mean": 364.770109
905:   },
906:   {
907:     "feature": "logged_in",
908:     "separation": 2.192973,
909:     "normal_mean": 0.71896,
910:     "attack_mean": 0.008137
911:   },
912:   {
913:     "feature": "service",
914:     "separation": 2.065529,
915:     "normal_mean": 12.620347,
916:     "attack_mean": 45.629301
917:   },
918:   {
919:     "feature": "protocol_type",
920:     "separation": 1.712784,
921:     "normal_mean": 1.223908,
922:     "attack_mean": 2.437788
923:   }

```

```

924:     {
925:       "feature": "dst_host_same_src_port_rate",
926:       "separation": 1.58209,
927:       "normal_mean": 0.132316,
928:       "attack_mean": 0.722001
929:     },
930:     {
931:       "feature": "dst_host_count",
932:       "separation": 1.393965,
933:       "normal_mean": 149.096674,
934:       "attack_mean": 253.07129
935:     },
936:     {
937:       "feature": "same_srv_rate",
938:       "separation": 0.786457,
939:       "normal_mean": 0.985621,
940:       "attack_mean": 0.749631
941:     },
942:     {
943:       "feature": "dst_host_srv_error_rate",
944:       "separation": 0.734602,
945:       "normal_mean": 0.001176,
946:       "attack_mean": 0.214609
947:     },
948:     {
949:       "feature": "error_rate",
950:       "separation": 0.733363,
951:       "normal_mean": 0.001602,
952:       "attack_mean": 0.214782
953:     },
954:     {
955:       "feature": "srv_error_rate",
956:       "separation": 0.731605,
957:       "normal_mean": 0.001808,
958:       "attack_mean": 0.214637
959:     },
960:     {
961:       "feature": "dst_host_error_rate",
962:       "separation": 0.731355,
963:       "normal_mean": 0.002143,
964:       "attack_mean": 0.214735
965:     }
966:   ]
967: },
968: "latest_export": {
969:   "analysis_id": "scan-20260506_101531",
970:   "created_at": "2026-05-06T04:45:32+00:00",
971:   "source_file": "kddtest.csv",
972:   "rows_analyzed": 5000,
973:   "malformed_rows": 0,
974:   "limit": 5000,
975:   "average_risk_score": 0.8381,
976:   "actual_counts": {
977:     "0": 703,
978:     "1": 4297
979:   },
980:   "predicted_counts": {
981:     "0": 810,
982:     "1": 4190
983:   },
984:   "risk_counts": {
985:     "low": 810,
986:     "critical": 4188,
987:     "medium": 1,
988:     "high": 1
989:   },
990:   "family_counts": {
991:     "normal": 688,
992:     "probe_scan": 115,
993:     "credential_abuse": 2,
994:     "network_attack": 8,
995:     "dos_flood": 4181,
996:     "payload_or_exfiltration": 2,
997:     "malware_like_activity": 3,
998:     "privilege_escalation": 1
999:   },
1000:   "top_protocols": [
1001:     [
1002:       "3",
1003:       4192
1004:     ],
1005:     [
1006:       "1",
1007:       552
1008:     ],

```

```

1009:     [
1010:         "2",
1011:         256
1012:     ],
1013: ],
1014: "top_services": [
1015:     [
1016:         "54",
1017:         4190
1018:     ],
1019:     [
1020:         "1",
1021:         450
1022:     ],
1023:     [
1024:         "24",
1025:         185
1026:     ],
1027:     [
1028:         "57",
1029:         71
1030:     ],
1031:     [
1032:         "19",
1033:         57
1034:     ],
1035:     [
1036:         "49",
1037:         22
1038:     ],
1039:     [
1040:         "65",
1041:         5
1042:     ],
1043:     [
1044:         "3",
1045:         5
1046:     ]
1047: ],
1048: "top_flags": [
1049:     [
1050:         "1",
1051:         4997
1052:     ],
1053:     [
1054:         "7",
1055:         1
1056:     ],
1057:     [
1058:         "5",
1059:         1
1060:     ],
1061:     [
1062:         "10",
1063:         1
1064:     ]
1065: ],
1066: "metrics": {
1067:     "accuracy": 0.9762,
1068:     "precision": 0.998568,
1069:     "recall": 0.973703,
1070:     "f1": 0.985979,
1071:     "tp": 4184,
1072:     "tn": 697,
1073:     "fp": 6,
1074:     "fn": 113
1075: },
1076: "model_created_at": "2026-05-06T04:44:52+00:00",
1077: "export_csv": "automation\\product\\exports\\traffic_analysis_20260506_101531.csv",
1078: "export_json": "automation\\product\\exports\\traffic_analysis_20260506_101531.json"
1079: },
1080: "recent_runs": [
1081:     {
1082:         "run_id": "dnn-1772763347b9",
1083:         "job_id": "terminal-auto",
1084:         "kind": "dnn_train",
1085:         "created_at": "2026-05-05T03:19:14.310697+00:00",
1086:         "config": {
1087:             "architectures": [
1088:                 1
1089:             ],
1090:             "train_sample": 5000,
1091:             "test_sample": 2000,
1092:             "epochs": 1,
1093:             "batch_size": 128,

```

```

1094:     "profile": "fast",
1095:     "random_state": 42
1096: },
1097:     "dataset": {
1098:         "train_rows": 5000,
1099:         "test_rows": 2000,
1100:         "feature_count": 41
1101:     },
1102:     "results": [
1103:         {
1104:             "id": "dnn_1_layer",
1105:             "label": "DNN 1 Layer",
1106:             "training_seconds": 0.47,
1107:             "metrics": {
1108:                 "accuracy": 0.92,
1109:                 "precision": 0.918108,
1110:                 "recall": 0.98882,
1111:                 "f1": 0.952153,
1112:                 "support": 2000,
1113:                 "tp": 1592,
1114:                 "tn": 248,
1115:                 "fp": 142,
1116:                 "fn": 18
1117:             },
1118:             "probability_summary": {
1119:                 "min": 0.168777,
1120:                 "mean": 0.792621,
1121:                 "max": 0.947958
1122:             },
1123:             "history": {
1124:                 "epochs": 1,
1125:                 "best_accuracy": 0.9026,
1126:                 "best_loss": 0.363292,
1127:                 "final_accuracy": 0.9026,
1128:                 "final_loss": 0.363292
1129:             },
1130:             "model_path": "runs/dnn-1772763347b9/models/dnn1.keras"
1131:         }
1132:     ],
1133:     "best_model": "dnn_1_layer"
1134: },
1135: {
1136:     "run_id": "classical-d7b00f74d926",
1137:     "job_id": "terminal-auto",
1138:     "kind": "classical_train",
1139:     "created_at": "2026-05-05T03:19:09.297116+00:00",
1140:     "config": {
1141:         "models": [
1142:             "logistic_regression",
1143:             "naive_bayes",
1144:             "decision_tree",
1145:             "adaboost",
1146:             "random_forest"
1147:         ],
1148:         "train_sample": 5000,
1149:         "test_sample": 2000,
1150:         "profile": "fast",
1151:         "random_state": 42
1152:     },
1153:     "dataset": {
1154:         "train_rows": 5000,
1155:         "test_rows": 2000,
1156:         "feature_count": 41
1157:     },
1158:     "results": [
1159:         {
1160:             "id": "adaboost",
1161:             "label": "AdaBoost",
1162:             "training_seconds": 0.407,
1163:             "metrics": {
1164:                 "accuracy": 0.921,
1165:                 "precision": 0.993207,
1166:                 "recall": 0.908075,
1167:                 "f1": 0.948735,
1168:                 "support": 2000,
1169:                 "tp": 1462,
1170:                 "tn": 380,
1171:                 "fp": 10,
1172:                 "fn": 148
1173:             },
1174:             "probability_summary": {
1175:                 "min": 0.265068,
1176:                 "mean": 0.563496,
1177:                 "max": 0.673029
1178:             }

```

```

1179:     "model_path": "runs/classical-d7b00f74d926/models/adaboost.joblib"
1180: },
1181: {
1182:     "id": "decision_tree",
1183:     "label": "Decision Tree",
1184:     "training_seconds": 0.015,
1185:     "metrics": {
1186:         "accuracy": 0.921,
1187:         "precision": 0.997942,
1188:         "recall": 0.903727,
1189:         "f1": 0.948501,
1190:         "support": 2000,
1191:         "tp": 1455,
1192:         "tn": 387,
1193:         "fp": 3,
1194:         "fn": 155
1195:     },
1196:     "probability_summary": {
1197:         "min": 0.0,
1198:         "mean": 0.729,
1199:         "max": 1.0
1200:     },
1201:     "model_path": "runs/classical-d7b00f74d926/models/decision_tree.joblib"
1202: },
1203: {
1204:     "id": "random_forest",
1205:     "label": "Random Forest",
1206:     "training_seconds": 0.316,
1207:     "metrics": {
1208:         "accuracy": 0.92,
1209:         "precision": 0.995219,
1210:         "recall": 0.904969,
1211:         "f1": 0.947951,
1212:         "support": 2000,
1213:         "tp": 1457,
1214:         "tn": 383,
1215:         "fp": 7,
1216:         "fn": 153
1217:     },
1218:     "probability_summary": {
1219:         "min": 0.0,
1220:         "mean": 0.730015,
1221:         "max": 1.0
1222:     },
1223:     "model_path": "runs/classical-d7b00f74d926/models/random_forest.joblib"
1224: },
1225: {
1226:     "id": "logistic_regression",
1227:     "label": "Logistic Regression",
1228:     "training_seconds": 0.364,
1229:     "metrics": {
1230:         "accuracy": 0.9095,
1231:         "precision": 0.985724,
1232:         "recall": 0.900621,
1233:         "f1": 0.941253,
1234:         "support": 2000,
1235:         "tp": 1450,
1236:         "tn": 369,
1237:         "fp": 21,
1238:         "fn": 160
1239:     },
1240:     "probability_summary": {
1241:         "min": 0.003524,
1242:         "mean": 0.708454,
1243:         "max": 0.999863
1244:     },
1245:     "model_path": "runs/classical-d7b00f74d926/models/logistic_regression.joblib"
1246: },
1247: {
1248:     "id": "naive_bayes",
1249:     "label": "Gaussian Naive Bayes",
1250:     "training_seconds": 0.003,
1251:     "metrics": {
1252:         "accuracy": 0.9095,
1253:         "precision": 0.989049,
1254:         "recall": 0.897516,
1255:         "f1": 0.941062,
1256:         "support": 2000,
1257:         "tp": 1445,
1258:         "tn": 374,
1259:         "fp": 16,
1260:         "fn": 165
1261:     },
1262:     "probability_summary": {
1263:         "min": 0.0,

```

```

1264:         "mean": 0.730647,
1265:         "max": 1.0
1266:     },
1267:     "model_path": "runs/classical-d7b00f74d926/models/naive_bayes.joblib"
1268: }
1269: ],
1270: "best_model": "adaboost"
1271: },
1272: {
1273:     "run_id": "dnn-dcefbda59cdb",
1274:     "job_id": "terminal",
1275:     "kind": "dnn_train",
1276:     "created_at": "2026-05-05T03:10:18.663632+00:00",
1277:     "config": {
1278:         "architectures": [
1279:             1
1280:         ],
1281:         "train_sample": 3000,
1282:         "test_sample": 1200,
1283:         "epochs": 1,
1284:         "batch_size": 128,
1285:         "profile": "fast",
1286:         "random_state": 42
1287:     },
1288:     "dataset": {
1289:         "train_rows": 3000,
1290:         "test_rows": 1200,
1291:         "feature_count": 41
1292:     },
1293:     "results": [
1294:         {
1295:             "id": "dnn_1_layer",
1296:             "label": "DNN 1 Layer",
1297:             "training_seconds": 1.053,
1298:             "metrics": {
1299:                 "accuracy": 0.900833,
1300:                 "precision": 0.893222,
1301:                 "recall": 0.995859,
1302:                 "f1": 0.941752,
1303:                 "support": 1200,
1304:                 "tp": 962,
1305:                 "tn": 119,
1306:                 "fp": 115,
1307:                 "fn": 4
1308:             },
1309:             "probability_summary": {
1310:                 "min": 0.400695,
1311:                 "mean": 0.770569,
1312:                 "max": 0.863311
1313:             },
1314:             "history": {
1315:                 "epochs": 1,
1316:                 "best_accuracy": 0.825667,
1317:                 "best_loss": 0.483427,
1318:                 "final_accuracy": 0.825667,
1319:                 "final_loss": 0.483427
1320:             },
1321:             "model_path": "runs/dnn-dcefbda59cdb/models/dnn1.keras"
1322:         }
1323:     ],
1324:     "best_model": "dnn_1_layer"
1325: },
1326: {
1327:     "run_id": "classical-45fc41fedadd",
1328:     "job_id": "terminal",
1329:     "kind": "classical_train",
1330:     "created_at": "2026-05-05T03:09:43.504542+00:00",
1331:     "config": {
1332:         "models": [
1333:             "logistic_regression",
1334:             "naive_bayes",
1335:             "decision_tree",
1336:             "adaboost",
1337:             "random_forest"
1338:         ],
1339:         "train_sample": 3000,
1340:         "test_sample": 1200,
1341:         "profile": "fast",
1342:         "random_state": 42
1343:     },
1344:     "dataset": {
1345:         "train_rows": 3000,
1346:         "test_rows": 1200,
1347:         "feature_count": 41
1348:     },

```

```

1349: "results": [
1350:   {
1351:     "id": "random_forest",
1352:     "label": "Random Forest",
1353:     "training_seconds": 0.404,
1354:     "metrics": {
1355:       "accuracy": 0.9275,
1356:       "precision": 1.0,
1357:       "recall": 0.909938,
1358:       "f1": 0.952846,
1359:       "support": 1200,
1360:       "tp": 879,
1361:       "tn": 234,
1362:       "fp": 0,
1363:       "fn": 87
1364:     },
1365:     "probability_summary": {
1366:       "min": 0.0,
1367:       "mean": 0.727058,
1368:       "max": 1.0
1369:     },
1370:     "model_path": "runs/classical-45fc41fedadd/models/random_forest.joblib"
1371:   },
1372:   {
1373:     "id": "adaboost",
1374:     "label": "AdaBoost",
1375:     "training_seconds": 0.292,
1376:     "metrics": {
1377:       "accuracy": 0.926667,
1378:       "precision": 0.996606,
1379:       "recall": 0.912008,
1380:       "f1": 0.952432,
1381:       "support": 1200,
1382:       "tp": 881,
1383:       "tn": 231,
1384:       "fp": 3,
1385:       "fn": 85
1386:     },
1387:     "probability_summary": {
1388:       "min": 0.272036,
1389:       "mean": 0.557741,
1390:       "max": 0.664778
1391:     },
1392:     "model_path": "runs/classical-45fc41fedadd/models/adaboost.joblib"
1393:   },
1394:   {
1395:     "id": "logistic_regression",
1396:     "label": "Logistic Regression",
1397:     "training_seconds": 0.476,
1398:     "metrics": {
1399:       "accuracy": 0.911667,
1400:       "precision": 0.985327,
1401:       "recall": 0.903727,
1402:       "f1": 0.942765,
1403:       "support": 1200,
1404:       "tp": 873,
1405:       "tn": 221,
1406:       "fp": 13,
1407:       "fn": 93
1408:     },
1409:     "probability_summary": {
1410:       "min": 0.006769,
1411:       "mean": 0.715482,
1412:       "max": 0.999533
1413:     },
1414:     "model_path": "runs/classical-45fc41fedadd/models/logistic_regression.joblib"
1415:   },
1416:   {
1417:     "id": "naive_bayes",
1418:     "label": "Gaussian Naive Bayes",
1419:     "training_seconds": 0.006,
1420:     "metrics": {
1421:       "accuracy": 0.910833,
1422:       "precision": 0.98641,
1423:       "recall": 0.901656,
1424:       "f1": 0.942131,
1425:       "support": 1200,
1426:       "tp": 871,
1427:       "tn": 222,
1428:       "fp": 12,
1429:       "fn": 95
1430:     },
1431:     "probability_summary": {
1432:       "min": 0.0,
1433:       "mean": 0.735767,

```

```

1434:         "max": 1.0
1435:     },
1436:     "model_path": "runs/classical-45fc41fedadd/models/naive_bayes.joblib"
1437: },
1438: {
1439:     "id": "decision_tree",
1440:     "label": "Decision Tree",
1441:     "training_seconds": 0.011,
1442:     "metrics": {
1443:         "accuracy": 0.856667,
1444:         "precision": 0.99625,
1445:         "recall": 0.825052,
1446:         "f1": 0.902605,
1447:         "support": 1200,
1448:         "tp": 797,
1449:         "tn": 231,
1450:         "fp": 3,
1451:         "fn": 169
1452:     },
1453:     "probability_summary": {
1454:         "min": 0.0,
1455:         "mean": 0.666667,
1456:         "max": 1.0
1457:     },
1458:     "model_path": "runs/classical-45fc41fedadd/models/decision_tree.joblib"
1459: }
1460: ],
1461: "best_model": "random_forest"
1462: },
1463: {
1464:     "run_id": "classical-892cd7c2f6a0",
1465:     "job_id": "39c3f07e95844705b30bb046b7c93701",
1466:     "kind": "classical_train",
1467:     "created_at": "2026-05-04T16:53:48.440056+00:00",
1468:     "config": {
1469:         "models": [
1470:             "logistic_regression",
1471:             "naive_bayes",
1472:             "decision_tree",
1473:             "adaboost",
1474:             "random_forest"
1475:         ],
1476:         "train_sample": 5000,
1477:         "test_sample": 2000,
1478:         "profile": "fast",
1479:         "random_state": 42
1480:     },
1481:     "dataset": {
1482:         "train_rows": 5000,
1483:         "test_rows": 2000,
1484:         "feature_count": 41
1485:     },
1486:     "results": [
1487:         {
1488:             "id": "adaboost",
1489:             "label": "AdaBoost",
1490:             "training_seconds": 0.942,
1491:             "metrics": {
1492:                 "accuracy": 0.921,
1493:                 "precision": 0.993207,
1494:                 "recall": 0.908075,
1495:                 "f1": 0.948735,
1496:                 "support": 2000,
1497:                 "tp": 1462,
1498:                 "tn": 380,
1499:                 "fp": 10,
1500:                 "fn": 148
1501:             },
1502:             "probability_summary": {
1503:                 "min": 0.265068,
1504:                 "mean": 0.563496,
1505:                 "max": 0.673029
1506:             },
1507:             "model_path": "runs/classical-892cd7c2f6a0/models/adaboost.joblib"
1508:         },
1509:         {
1510:             "id": "decision_tree",
1511:             "label": "Decision Tree",
1512:             "training_seconds": 0.036,
1513:             "metrics": {
1514:                 "accuracy": 0.921,
1515:                 "precision": 0.997942,
1516:                 "recall": 0.903727,
1517:                 "f1": 0.948501,
1518:                 "support": 2000,

```

```

1519:         "tp": 1455,
1520:         "tn": 387,
1521:         "fp": 3,
1522:         "fn": 155
1523:     },
1524:     "probability_summary": {
1525:         "min": 0.0,
1526:         "mean": 0.729,
1527:         "max": 1.0
1528:     },
1529:     "model_path": "runs/classical-892cd7c2f6a0/models/decision_tree.joblib"
1530: },
1531: {
1532:     "id": "random_forest",
1533:     "label": "Random Forest",
1534:     "training_seconds": 0.726,
1535:     "metrics": {
1536:         "accuracy": 0.92,
1537:         "precision": 0.995219,
1538:         "recall": 0.904969,
1539:         "f1": 0.947951,
1540:         "support": 2000,
1541:         "tp": 1457,
1542:         "tn": 383,
1543:         "fp": 7,
1544:         "fn": 153
1545:     },
1546:     "probability_summary": {
1547:         "min": 0.0,
1548:         "mean": 0.730015,
1549:         "max": 1.0
1550:     },
1551:     "model_path": "runs/classical-892cd7c2f6a0/models/random_forest.joblib"
1552: },
1553: {
1554:     "id": "logistic_regression",
1555:     "label": "Logistic Regression",
1556:     "training_seconds": 0.704,
1557:     "metrics": {
1558:         "accuracy": 0.9095,
1559:         "precision": 0.985724,
1560:         "recall": 0.900621,
1561:         "f1": 0.941253,
1562:         "support": 2000,
1563:         "tp": 1450,
1564:         "tn": 369,
1565:         "fp": 21,
1566:         "fn": 160
1567:     },
1568:     "probability_summary": {
1569:         "min": 0.003524,
1570:         "mean": 0.708454,
1571:         "max": 0.999863
1572:     },
1573:     "model_path": "runs/classical-892cd7c2f6a0/models/logistic_regression.joblib"
1574: },
1575: {
1576:     "id": "naive_bayes",
1577:     "label": "Gaussian Naive Bayes",
1578:     "training_seconds": 0.004,
1579:     "metrics": {
1580:         "accuracy": 0.9095,
1581:         "precision": 0.989049,
1582:         "recall": 0.897516,
1583:         "f1": 0.941062,
1584:         "support": 2000,
1585:         "tp": 1445,
1586:         "tn": 374,
1587:         "fp": 16,
1588:         "fn": 165
1589:     },
1590:     "probability_summary": {
1591:         "min": 0.0,
1592:         "mean": 0.730647,
1593:         "max": 1.0
1594:     },
1595:     "model_path": "runs/classical-892cd7c2f6a0/models/naive_bayes.joblib"
1596: }
1597: },
1598: "best_model": "adaboost"
1599: },
1600: {
1601:     "run_id": "dnn-29fd1f5f240b",
1602:     "job_id": "e4efa927fdcd4dc2ba58e41ceb8af5ce",
1603:     "kind": "dnn_train",

```

```

1604:     "created_at": "2026-05-04T16:52:33.307024+00:00",
1605:     "config": {
1606:         "architectures": [
1607:             1
1608:         ],
1609:         "train_sample": 5000,
1610:         "test_sample": 2000,
1611:         "epochs": 1,
1612:         "batch_size": 128,
1613:         "profile": "fast",
1614:         "random_state": 42
1615:     },
1616:     "dataset": {
1617:         "train_rows": 5000,
1618:         "test_rows": 2000,
1619:         "feature_count": 41
1620:     },
1621:     "results": [
1622:         {
1623:             "id": "dnn_1_layer",
1624:             "label": "DNN 1 Layer",
1625:             "training_seconds": 1.273,
1626:             "metrics": {
1627:                 "accuracy": 0.916,
1628:                 "precision": 0.912,
1629:                 "recall": 0.991304,
1630:                 "f1": 0.95,
1631:                 "support": 2000,
1632:                 "tp": 1596,
1633:                 "tn": 236,
1634:                 "fp": 154,
1635:                 "fn": 14
1636:             },
1637:             "probability_summary": {
1638:                 "min": 0.210737,
1639:                 "mean": 0.801977,
1640:                 "max": 0.943477
1641:             },
1642:             "history": {
1643:                 "epochs": 1,
1644:                 "best_accuracy": 0.8812,
1645:                 "best_loss": 0.382257,
1646:                 "final_accuracy": 0.8812,
1647:                 "final_loss": 0.382257
1648:             },
1649:             "model_path": "runs/dnn-29fd1f5f240b/models/dnn1.keras"
1650:         }
1651:     ],
1652:     "best_model": "dnn_1_layer"
1653: },
1654: {
1655:     "run_id": "dnn-9612cd3f9981",
1656:     "job_id": "0ef98f5be8294bd4953e2ccc9b61f19c",
1657:     "kind": "dnn_train",
1658:     "created_at": "2026-05-04T16:50:49.902748+00:00",
1659:     "config": {
1660:         "architectures": [
1661:             1,
1662:             3
1663:         ],
1664:         "train_sample": 50000,
1665:         "test_sample": 15000,
1666:         "epochs": 3,
1667:         "batch_size": 128,
1668:         "profile": "fast",
1669:         "random_state": 42
1670:     },
1671:     "dataset": {
1672:         "train_rows": 50000,
1673:         "test_rows": 15000,
1674:         "feature_count": 41
1675:     },
1676:     "results": [
1677:         {
1678:             "id": "dnn_3_layer",
1679:             "label": "DNN 3 Layer",
1680:             "training_seconds": 22.849,
1681:             "metrics": {
1682:                 "accuracy": 0.921467,
1683:                 "precision": 0.997081,
1684:                 "recall": 0.905117,
1685:                 "f1": 0.948876,
1686:                 "support": 15000,
1687:                 "tp": 10932,
1688:                 "tn": 2890,

```

```

1689:         "fp": 32,
1690:         "fn": 1146
1691:     },
1692:     "probability_summary": {
1693:         "min": 0.0,
1694:         "mean": 0.724132,
1695:         "max": 1.0
1696:     },
1697:     "history": {
1698:         "epochs": 3,
1699:         "best_accuracy": 0.99626,
1700:         "best_loss": 0.012686,
1701:         "final_accuracy": 0.99626,
1702:         "final_loss": 0.012686
1703:     },
1704:     "model_path": "runs/dnn-9612cd3f9981/models/dnn3.keras"
1705: },
1706: {
1707:     "id": "dnn_1_layer",
1708:     "label": "DNN 1 Layer",
1709:     "training_seconds": 5.954,
1710:     "metrics": {
1711:         "accuracy": 0.906133,
1712:         "precision": 0.99062,
1713:         "recall": 0.89187,
1714:         "f1": 0.938655,
1715:         "support": 15000,
1716:         "tp": 10772,
1717:         "tn": 2820,
1718:         "fp": 102,
1719:         "fn": 1306
1720:     },
1721:     "probability_summary": {
1722:         "min": 0.000149,
1723:         "mean": 0.69896,
1724:         "max": 0.999992
1725:     },
1726:     "history": {
1727:         "epochs": 3,
1728:         "best_accuracy": 0.98756,
1729:         "best_loss": 0.026306,
1730:         "final_accuracy": 0.98756,
1731:         "final_loss": 0.026306
1732:     },
1733:     "model_path": "runs/dnn-9612cd3f9981/models/dnn1.keras"
1734: }
1735: ],
1736: "best_model": "dnn_3_layer"
1737: },
1738: {
1739:     "run_id": "classical-4676bd9735c7",
1740:     "job_id": "f8ce165380d2480482574ab667d00482",
1741:     "kind": "classical_train",
1742:     "created_at": "2026-05-04T16:49:50.219648+00:00",
1743:     "config": {
1744:         "models": [
1745:             "logistic_regression",
1746:             "naive_bayes",
1747:             "decision_tree",
1748:             "adaboost",
1749:             "random_forest"
1750:         ],
1751:         "train_sample": 40000,
1752:         "test_sample": 15000,
1753:         "profile": "fast",
1754:         "random_state": 42
1755:     },
1756:     "dataset": {
1757:         "train_rows": 40000,
1758:         "test_rows": 15000,
1759:         "feature_count": 41
1760:     },
1761:     "results": [
1762:         {
1763:             "id": "random_forest",
1764:             "label": "Random Forest",
1765:             "training_seconds": 1.339,
1766:             "metrics": {
1767:                 "accuracy": 0.9232,
1768:                 "precision": 0.998267,
1769:                 "recall": 0.906193,
1770:                 "f1": 0.950004,
1771:                 "support": 15000,
1772:                 "tp": 10945,
1773:                 "tn": 2903,

```

```

1774:         "fp": 19,
1775:         "fn": 1133
1776:     },
1777:     "probability_summary": {
1778:         "min": 0.0,
1779:         "mean": 0.733314,
1780:         "max": 1.0
1781:     },
1782:     "model_path": "runs/classical-4676bd9735c7/models/random_forest.joblib"
1783: },
1784: {
1785:     "id": "decision_tree",
1786:     "label": "Decision Tree",
1787:     "training_seconds": 0.479,
1788:     "metrics": {
1789:         "accuracy": 0.9228,
1790:         "precision": 0.997903,
1791:         "recall": 0.906027,
1792:         "f1": 0.949748,
1793:         "support": 15000,
1794:         "tp": 10943,
1795:         "tn": 2899,
1796:         "fp": 23,
1797:         "fn": 1135
1798:     },
1799:     "probability_summary": {
1800:         "min": 0.0,
1801:         "mean": 0.731067,
1802:         "max": 1.0
1803:     },
1804:     "model_path": "runs/classical-4676bd9735c7/models/decision_tree.joblib"
1805: },
1806: {
1807:     "id": "adaboost",
1808:     "label": "AdaBoost",
1809:     "training_seconds": 6.218,
1810:     "metrics": {
1811:         "accuracy": 0.921733,
1812:         "precision": 0.995006,
1813:         "recall": 0.907352,
1814:         "f1": 0.94916,
1815:         "support": 15000,
1816:         "tp": 10959,
1817:         "tn": 2867,
1818:         "fp": 55,
1819:         "fn": 1119
1820:     },
1821:     "probability_summary": {
1822:         "min": 0.415657,
1823:         "mean": 0.518662,
1824:         "max": 0.592831
1825:     },
1826:     "model_path": "runs/classical-4676bd9735c7/models/adaboost.joblib"
1827: },
1828: {
1829:     "id": "naive_bayes",
1830:     "label": "Gaussian Naive Bayes",
1831:     "training_seconds": 0.023,
1832:     "metrics": {
1833:         "accuracy": 0.9114,
1834:         "precision": 0.989972,
1835:         "recall": 0.899073,
1836:         "f1": 0.942335,
1837:         "support": 15000,
1838:         "tp": 10859,
1839:         "tn": 2812,
1840:         "fp": 110,
1841:         "fn": 1219
1842:     },
1843:     "probability_summary": {
1844:         "min": 0.0,
1845:         "mean": 0.732066,
1846:         "max": 1.0
1847:     },
1848:     "model_path": "runs/classical-4676bd9735c7/models/naive_bayes.joblib"
1849: },
1850: {
1851:     "id": "logistic_regression",
1852:     "label": "Logistic Regression",
1853:     "training_seconds": 0.945,
1854:     "metrics": {
1855:         "accuracy": 0.902333,
1856:         "precision": 0.988134,
1857:         "recall": 0.889386,
1858:         "f1": 0.936163,

```

```

1859:         "support": 15000,
1860:         "tp": 10742,
1861:         "tn": 2793,
1862:         "fp": 129,
1863:         "fn": 1336
1864:     },
1865:     "probability_summary": {
1866:         "min": 0.000107,
1867:         "mean": 0.694586,
1868:         "max": 1.0
1869:     },
1870:     "model_path": "runs/classical-4676bd9735c7/models/logistic_regression.joblib"
1871: },
1872: ],
1873: "best_model": "random_forest"
1874: }
1875: ]
1876: }

```

attacks --json

This dump is included verbatim from a live command invocation so that readers can inspect the exact machine-readable structure behind the narrative summaries used earlier in the report.

```

1: {
2:   "datasets": {
3:     "train": {
4:       "path": "kddtrain.csv",
5:       "rows": 494021,
6:       "malformed_rows": 0,
7:       "columns": 42,
8:       "size_mb": 46.13,
9:       "label_counts": {
10:        "0": 97278,
11:        "1": 396743
12:      },
13:       "attack_share": 0.803089,
14:       "normal_share": 0.196911,
15:       "total_src_bytes": 1494715024,
16:       "total_dst_bytes": 429073257,
17:       "top_protocols": [
18:         [
19:           "3",
20:           283602
21:         ],
22:         [
23:           "1",
24:           190065
25:         ],
26:         [
27:           "2",
28:           20354
29:         ]
30:       ],
31:       "top_services": [
32:         [
33:           "54",
34:           281400
35:         ],
36:         [
37:           "24",
38:           110893
39:         ],
40:         [
41:           "1",
42:           64293
43:         ],
44:         [
45:           "19",
46:           9723
47:         ],
48:         [
49:           "29",
50:           7237
51:         ],
52:         [
53:           "57",
54:           5863
55:         ],
56:         [
57:           "49",
58:           4721

```

```

59:     ],
60:     [
61:         "55",
62:         1642
63:     ]
64: ],
65: "top_flags": [
66:     [
67:         "1",
68:         378440
69:     ],
70:     [
71:         "6",
72:         87007
73:     ],
74:     [
75:         "10",
76:         26875
77:     ],
78:     [
79:         "7",
80:         903
81:     ],
82:     [
83:         "9",
84:         579
85:     ],
86:     [
87:         "2",
88:         107
89:     ],
90:     [
91:         "5",
92:         57
93:     ],
94:     [
95:         "4",
96:         32
97:     ]
98: ],
99: },
100: "test": {
101:     "path": "kddtest.csv",
102:     "rows": 311029,
103:     "malformed_rows": 0,
104:     "columns": 42,
105:     "size_mb": 29.04,
106:     "label_counts": {
107:         "0": 60593,
108:         "1": 250436
109:     },
110:     "attack_share": 0.805185,
111:     "normal_share": 0.194815,
112:     "total_src_bytes": 538609601,
113:     "total_dst_bytes": 232647728,
114:     "top_protocols": [
115:         [
116:             "3",
117:             164969
118:         ],
119:         [
120:             "1",
121:             119357
122:         ],
123:         [
124:             "2",
125:             26703
126:         ]
127:     ],
128:     "top_services": [
129:         [
130:             "54",
131:             164352
132:         ],
133:         [
134:             "24",
135:             78510
136:         ],
137:         [
138:             "1",
139:             41237
140:         ],
141:         [
142:             "19",
143:             8268

```

```

144:     ],
145:     [
146:         "26",
147:         3972
148:     ],
149:     [
150:         "57",
151:         3160
152:     ],
153:     [
154:         "49",
155:         2223
156:     ],
157:     [
158:         "29",
159:         2185
160:     ]
161: ],
162: "top_flags": [
163:     [
164:         "1",
165:         248379
166:     ],
167:     [
168:         "10",
169:         41945
170:     ],
171:     [
172:         "6",
173:         18012
174:     ],
175:     [
176:         "9",
177:         1393
178:     ],
179:     [
180:         "7",
181:         872
182:     ],
183:     [
184:         "3",
185:         289
186:     ],
187:     [
188:         "2",
189:         84
190:     ],
191:     [
192:         "5",
193:         27
194:     ]
195: ]
196: }
197: },
198: "model_indicators": [
199:     {
200:         "feature": "count",
201:         "separation": 3.657428,
202:         "normal_mean": 8.141068,
203:         "attack_mean": 413.389361
204:     },
205:     {
206:         "feature": "srv_count",
207:         "separation": 2.21418,
208:         "normal_mean": 10.926262,
209:         "attack_mean": 364.770109
210:     },
211:     {
212:         "feature": "logged_in",
213:         "separation": 2.192973,
214:         "normal_mean": 0.71896,
215:         "attack_mean": 0.008137
216:     },
217:     {
218:         "feature": "service",
219:         "separation": 2.065529,
220:         "normal_mean": 12.620347,
221:         "attack_mean": 45.629301
222:     },
223:     {
224:         "feature": "protocol_type",
225:         "separation": 1.712784,
226:         "normal_mean": 1.223908,
227:         "attack_mean": 2.437788
228:     },

```

```

229:     {
230:         "feature": "dst_host_same_src_port_rate",
231:         "separation": 1.58209,
232:         "normal_mean": 0.132316,
233:         "attack_mean": 0.722001
234:     },
235:     {
236:         "feature": "dst_host_count",
237:         "separation": 1.393965,
238:         "normal_mean": 149.096674,
239:         "attack_mean": 253.07129
240:     },
241:     {
242:         "feature": "same_srv_rate",
243:         "separation": 0.786457,
244:         "normal_mean": 0.985621,
245:         "attack_mean": 0.749631
246:     },
247:     {
248:         "feature": "dst_host_srv_error_rate",
249:         "separation": 0.734602,
250:         "normal_mean": 0.001176,
251:         "attack_mean": 0.214609
252:     },
253:     {
254:         "feature": "error_rate",
255:         "separation": 0.733363,
256:         "normal_mean": 0.001602,
257:         "attack_mean": 0.214782
258:     },
259:     {
260:         "feature": "srv_error_rate",
261:         "separation": 0.731605,
262:         "normal_mean": 0.001808,
263:         "attack_mean": 0.214637
264:     },
265:     {
266:         "feature": "dst_host_error_rate",
267:         "separation": 0.731355,
268:         "normal_mean": 0.002143,
269:         "attack_mean": 0.214735
270:     }
271: ]
272: }

```

datasets --json

This dump is included verbatim from a live command invocation so that readers can inspect the exact machine-readable structure behind the narrative summaries used earlier in the report.

```

1: {
2:   "local_sources": [
3:     "kddtrain.csv",
4:     "kddtest.csv"
5:   ],
6:   "external_catalog": [
7:     {
8:       "id": "cicids2017",
9:       "name": "CIC-IDS2017",
10:      "source": "Canadian Institute for Cybersecurity, University of New Brunswick",
11:      "url": "https://www.unb.ca/cic/datasets/ids-2017.html",
12:      "format": "PCAP and CICFlowMeter CSV flow data",
13:      "notes": "Contains benign traffic plus FTP/SSH brute force, DoS, Heartbleed, web attacks, infiltration, botnet, DDoS, and port scan."
14:    },
15:    {
16:      "id": "unsw-nb15",
17:      "name": "UNSW-NB15",
18:      "source": "UNSW Canberra Cyber Range Lab",
19:      "url": "https://research.unsw.edu.au/projects/unsw-nb15-dataset",
20:      "format": "CSV network-flow features with binary and attack-category labels",
21:      "notes": "Contains normal traffic and Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms."
22:    },
23:    {
24:      "id": "unsw-nb15-hf",
25:      "name": "UNSW-NB15 Hugging Face mirror",
26:      "source": "Hugging Face community dataset mirror",
27:      "url": "https://huggingface.co/datasets/lacg030175/UNSW-NB15",
28:      "format": "Dataset hub train/test splits",
29:      "notes": "Useful when the Python datasets package is installed. This terminal can still import downloaded CSV files directly."
30:    }
31:   ]
32: }

```

runs --json

This dump is included verbatim from a live command invocation so that readers can inspect the exact machine-readable structure behind the narrative summaries used earlier in the report.

```
1: [
2:   {
3:     "run_id": "dnn-1772763347b9",
4:     "job_id": "terminal-auto",
5:     "kind": "dnn_train",
6:     "created_at": "2026-05-05T03:19:14.310697+00:00",
7:     "config": {
8:       "architectures": [
9:         1
10:      ],
11:      "train_sample": 5000,
12:      "test_sample": 2000,
13:      "epochs": 1,
14:      "batch_size": 128,
15:      "profile": "fast",
16:      "random_state": 42
17:    },
18:    "dataset": {
19:      "train_rows": 5000,
20:      "test_rows": 2000,
21:      "feature_count": 41
22:    },
23:    "results": [
24:      {
25:        "id": "dnn_1_layer",
26:        "label": "DNN 1 Layer",
27:        "training_seconds": 0.47,
28:        "metrics": {
29:          "accuracy": 0.92,
30:          "precision": 0.918108,
31:          "recall": 0.98882,
32:          "f1": 0.952153,
33:          "support": 2000,
34:          "tp": 1592,
35:          "tn": 248,
36:          "fp": 142,
37:          "fn": 18
38:        },
39:        "probability_summary": {
40:          "min": 0.168777,
41:          "mean": 0.792621,
42:          "max": 0.947958
43:        },
44:        "history": {
45:          "epochs": 1,
46:          "best_accuracy": 0.9026,
47:          "best_loss": 0.363292,
48:          "final_accuracy": 0.9026,
49:          "final_loss": 0.363292
50:        },
51:        "model_path": "runs/dnn-1772763347b9/models/dnn1.keras"
52:      }
53:    ],
54:    "best_model": "dnn_1_layer"
55:  },
56:  {
57:    "run_id": "classical-d7b00f74d926",
58:    "job_id": "terminal-auto",
59:    "kind": "classical_train",
60:    "created_at": "2026-05-05T03:19:09.297116+00:00",
61:    "config": {
62:      "models": [
63:        "logistic_regression",
64:        "naive_bayes",
65:        "decision_tree",
66:        "adaboost",
67:        "random_forest"
68:      ],
69:      "train_sample": 5000,
70:      "test_sample": 2000,
71:      "profile": "fast",
72:      "random_state": 42
73:    },
74:    "dataset": {
75:      "train_rows": 5000,
76:      "test_rows": 2000,
77:      "feature_count": 41
78:    },
79:    "results": [
80:      {
81:        "id": "classical_1",
82:        "label": "Classical 1",
83:        "training_seconds": 0.1,
84:        "metrics": {
85:          "accuracy": 0.8,
86:          "precision": 0.8,
87:          "recall": 0.8,
88:          "f1": 0.8,
89:          "support": 2000,
90:          "tp": 1600,
91:          "tn": 400,
92:          "fp": 400,
93:          "fn": 400
94:        },
95:        "probability_summary": {
96:          "min": 0.1,
97:          "mean": 0.5,
98:          "max": 0.9
99:        },
100:        "history": {
101:          "epochs": 1,
102:          "best_accuracy": 0.8,
103:          "best_loss": 0.2,
104:          "final_accuracy": 0.8,
105:          "final_loss": 0.2
106:        },
107:        "model_path": "runs/classical-d7b00f74d926/models/classical1.keras"
108:      }
109:    ],
110:    "best_model": "classical_1"
111:  }
112: ]
```

```

78: },
79: "results": [
80:   {
81:     "id": "adaboost",
82:     "label": "AdaBoost",
83:     "training_seconds": 0.407,
84:     "metrics": {
85:       "accuracy": 0.921,
86:       "precision": 0.993207,
87:       "recall": 0.908075,
88:       "f1": 0.948735,
89:       "support": 2000,
90:       "tp": 1462,
91:       "tn": 380,
92:       "fp": 10,
93:       "fn": 148
94:     },
95:     "probability_summary": {
96:       "min": 0.265068,
97:       "mean": 0.563496,
98:       "max": 0.673029
99:     },
100:     "model_path": "runs/classical-d7b00f74d926/models/adaboost.joblib"
101:   },
102:   {
103:     "id": "decision_tree",
104:     "label": "Decision Tree",
105:     "training_seconds": 0.015,
106:     "metrics": {
107:       "accuracy": 0.921,
108:       "precision": 0.997942,
109:       "recall": 0.903727,
110:       "f1": 0.948501,
111:       "support": 2000,
112:       "tp": 1455,
113:       "tn": 387,
114:       "fp": 3,
115:       "fn": 155
116:     },
117:     "probability_summary": {
118:       "min": 0.0,
119:       "mean": 0.729,
120:       "max": 1.0
121:     },
122:     "model_path": "runs/classical-d7b00f74d926/models/decision_tree.joblib"
123:   },
124:   {
125:     "id": "random_forest",
126:     "label": "Random Forest",
127:     "training_seconds": 0.316,
128:     "metrics": {
129:       "accuracy": 0.92,
130:       "precision": 0.995219,
131:       "recall": 0.904969,
132:       "f1": 0.947951,
133:       "support": 2000,
134:       "tp": 1457,
135:       "tn": 383,
136:       "fp": 7,
137:       "fn": 153
138:     },
139:     "probability_summary": {
140:       "min": 0.0,
141:       "mean": 0.730015,
142:       "max": 1.0
143:     },
144:     "model_path": "runs/classical-d7b00f74d926/models/random_forest.joblib"
145:   },
146:   {
147:     "id": "logistic_regression",
148:     "label": "Logistic Regression",
149:     "training_seconds": 0.364,
150:     "metrics": {
151:       "accuracy": 0.9095,
152:       "precision": 0.985724,
153:       "recall": 0.900621,
154:       "f1": 0.941253,
155:       "support": 2000,
156:       "tp": 1450,
157:       "tn": 369,
158:       "fp": 21,
159:       "fn": 160
160:     },
161:     "probability_summary": {
162:       "min": 0.003524,

```

```

163:         "mean": 0.708454,
164:         "max": 0.999863
165:     },
166:     "model_path": "runs/classical-d7b00f74d926/models/logistic_regression.joblib"
167: },
168: {
169:     "id": "naive_bayes",
170:     "label": "Gaussian Naive Bayes",
171:     "training_seconds": 0.003,
172:     "metrics": {
173:         "accuracy": 0.9095,
174:         "precision": 0.989049,
175:         "recall": 0.897516,
176:         "f1": 0.941062,
177:         "support": 2000,
178:         "tp": 1445,
179:         "tn": 374,
180:         "fp": 16,
181:         "fn": 165
182:     },
183:     "probability_summary": {
184:         "min": 0.0,
185:         "mean": 0.730647,
186:         "max": 1.0
187:     },
188:     "model_path": "runs/classical-d7b00f74d926/models/naive_bayes.joblib"
189: },
190: ],
191: "best_model": "adaboost"
192: },
193: {
194:     "run_id": "dnn-dcefbda59cdb",
195:     "job_id": "terminal",
196:     "kind": "dnn_train",
197:     "created_at": "2026-05-05T03:10:18.663632+00:00",
198:     "config": {
199:         "architectures": [
200:             1
201:         ],
202:         "train_sample": 3000,
203:         "test_sample": 1200,
204:         "epochs": 1,
205:         "batch_size": 128,
206:         "profile": "fast",
207:         "random_state": 42
208:     },
209:     "dataset": {
210:         "train_rows": 3000,
211:         "test_rows": 1200,
212:         "feature_count": 41
213:     },
214:     "results": [
215:         {
216:             "id": "dnn_1_layer",
217:             "label": "DNN 1 Layer",
218:             "training_seconds": 1.053,
219:             "metrics": {
220:                 "accuracy": 0.900833,
221:                 "precision": 0.893222,
222:                 "recall": 0.995859,
223:                 "f1": 0.941752,
224:                 "support": 1200,
225:                 "tp": 962,
226:                 "tn": 119,
227:                 "fp": 115,
228:                 "fn": 4
229:             },
230:             "probability_summary": {
231:                 "min": 0.400695,
232:                 "mean": 0.770569,
233:                 "max": 0.863311
234:             },
235:             "history": {
236:                 "epochs": 1,
237:                 "best_accuracy": 0.825667,
238:                 "best_loss": 0.483427,
239:                 "final_accuracy": 0.825667,
240:                 "final_loss": 0.483427
241:             },
242:             "model_path": "runs/dnn-dcefbda59cdb/models/dnn1.keras"
243:         }
244:     ],
245:     "best_model": "dnn_1_layer"
246: },
247: {

```

```

248: "run_id": "classical-45fc41fedadd",
249: "job_id": "terminal",
250: "kind": "classical_train",
251: "created_at": "2026-05-05T03:09:43.504542+00:00",
252: "config": {
253:   "models": [
254:     "logistic_regression",
255:     "naive_bayes",
256:     "decision_tree",
257:     "adaboost",
258:     "random_forest"
259:   ],
260:   "train_sample": 3000,
261:   "test_sample": 1200,
262:   "profile": "fast",
263:   "random_state": 42
264: },
265: "dataset": {
266:   "train_rows": 3000,
267:   "test_rows": 1200,
268:   "feature_count": 41
269: },
270: "results": [
271:   {
272:     "id": "random_forest",
273:     "label": "Random Forest",
274:     "training_seconds": 0.404,
275:     "metrics": {
276:       "accuracy": 0.9275,
277:       "precision": 1.0,
278:       "recall": 0.909938,
279:       "f1": 0.952846,
280:       "support": 1200,
281:       "tp": 879,
282:       "tn": 234,
283:       "fp": 0,
284:       "fn": 87
285:     },
286:     "probability_summary": {
287:       "min": 0.0,
288:       "mean": 0.727058,
289:       "max": 1.0
290:     },
291:     "model_path": "runs/classical-45fc41fedadd/models/random_forest.joblib"
292:   },
293:   {
294:     "id": "adaboost",
295:     "label": "AdaBoost",
296:     "training_seconds": 0.292,
297:     "metrics": {
298:       "accuracy": 0.926667,
299:       "precision": 0.996606,
300:       "recall": 0.912008,
301:       "f1": 0.952432,
302:       "support": 1200,
303:       "tp": 881,
304:       "tn": 231,
305:       "fp": 3,
306:       "fn": 85
307:     },
308:     "probability_summary": {
309:       "min": 0.272036,
310:       "mean": 0.557741,
311:       "max": 0.664778
312:     },
313:     "model_path": "runs/classical-45fc41fedadd/models/adaboost.joblib"
314:   },
315:   {
316:     "id": "logistic_regression",
317:     "label": "Logistic Regression",
318:     "training_seconds": 0.476,
319:     "metrics": {
320:       "accuracy": 0.911667,
321:       "precision": 0.985327,
322:       "recall": 0.903727,
323:       "f1": 0.942765,
324:       "support": 1200,
325:       "tp": 873,
326:       "tn": 221,
327:       "fp": 13,
328:       "fn": 93
329:     },
330:     "probability_summary": {
331:       "min": 0.006769,
332:       "mean": 0.715482,

```

```

333:         "max": 0.999533
334:     },
335:     "model_path": "runs/classical-45fc41fedadd/models/logistic_regression.joblib"
336: },
337: {
338:     "id": "naive_bayes",
339:     "label": "Gaussian Naïve Bayes",
340:     "training_seconds": 0.006,
341:     "metrics": {
342:         "accuracy": 0.910833,
343:         "precision": 0.98641,
344:         "recall": 0.901656,
345:         "f1": 0.942131,
346:         "support": 1200,
347:         "tp": 871,
348:         "tn": 222,
349:         "fp": 12,
350:         "fn": 95
351:     },
352:     "probability_summary": {
353:         "min": 0.0,
354:         "mean": 0.735767,
355:         "max": 1.0
356:     },
357:     "model_path": "runs/classical-45fc41fedadd/models/naive_bayes.joblib"
358: },
359: {
360:     "id": "decision_tree",
361:     "label": "Decision Tree",
362:     "training_seconds": 0.011,
363:     "metrics": {
364:         "accuracy": 0.856667,
365:         "precision": 0.99625,
366:         "recall": 0.825052,
367:         "f1": 0.902605,
368:         "support": 1200,
369:         "tp": 797,
370:         "tn": 231,
371:         "fp": 3,
372:         "fn": 169
373:     },
374:     "probability_summary": {
375:         "min": 0.0,
376:         "mean": 0.666667,
377:         "max": 1.0
378:     },
379:     "model_path": "runs/classical-45fc41fedadd/models/decision_tree.joblib"
380: },
381: ],
382: "best_model": "random_forest"
383: },
384: {
385:     "run_id": "classical-892cd7c2f6a0",
386:     "job_id": "39c3f07e95844705b30bb046b7c93701",
387:     "kind": "classical_train",
388:     "created_at": "2026-05-04T16:53:48.440056+00:00",
389:     "config": {
390:         "models": [
391:             "logistic_regression",
392:             "naive_bayes",
393:             "decision_tree",
394:             "adaboost",
395:             "random_forest"
396:         ],
397:         "train_sample": 5000,
398:         "test_sample": 2000,
399:         "profile": "fast",
400:         "random_state": 42
401:     },
402:     "dataset": {
403:         "train_rows": 5000,
404:         "test_rows": 2000,
405:         "feature_count": 41
406:     },
407:     "results": [
408:         {
409:             "id": "adaboost",
410:             "label": "AdaBoost",
411:             "training_seconds": 0.942,
412:             "metrics": {
413:                 "accuracy": 0.921,
414:                 "precision": 0.993207,
415:                 "recall": 0.908075,
416:                 "f1": 0.948735,
417:                 "support": 2000,

```

```

418:         "tp": 1462,
419:         "tn": 380,
420:         "fp": 10,
421:         "fn": 148
422:     },
423:     "probability_summary": {
424:         "min": 0.265068,
425:         "mean": 0.563496,
426:         "max": 0.673029
427:     },
428:     "model_path": "runs/classical-892cd7c2f6a0/models/adaboost.joblib"
429: },
430: {
431:     "id": "decision_tree",
432:     "label": "Decision Tree",
433:     "training_seconds": 0.036,
434:     "metrics": {
435:         "accuracy": 0.921,
436:         "precision": 0.997942,
437:         "recall": 0.903727,
438:         "f1": 0.948501,
439:         "support": 2000,
440:         "tp": 1455,
441:         "tn": 387,
442:         "fp": 3,
443:         "fn": 155
444:     },
445:     "probability_summary": {
446:         "min": 0.0,
447:         "mean": 0.729,
448:         "max": 1.0
449:     },
450:     "model_path": "runs/classical-892cd7c2f6a0/models/decision_tree.joblib"
451: },
452: {
453:     "id": "random_forest",
454:     "label": "Random Forest",
455:     "training_seconds": 0.726,
456:     "metrics": {
457:         "accuracy": 0.92,
458:         "precision": 0.995219,
459:         "recall": 0.904969,
460:         "f1": 0.947951,
461:         "support": 2000,
462:         "tp": 1457,
463:         "tn": 383,
464:         "fp": 7,
465:         "fn": 153
466:     },
467:     "probability_summary": {
468:         "min": 0.0,
469:         "mean": 0.730015,
470:         "max": 1.0
471:     },
472:     "model_path": "runs/classical-892cd7c2f6a0/models/random_forest.joblib"
473: },
474: {
475:     "id": "logistic_regression",
476:     "label": "Logistic Regression",
477:     "training_seconds": 0.704,
478:     "metrics": {
479:         "accuracy": 0.9095,
480:         "precision": 0.985724,
481:         "recall": 0.900621,
482:         "f1": 0.941253,
483:         "support": 2000,
484:         "tp": 1450,
485:         "tn": 369,
486:         "fp": 21,
487:         "fn": 160
488:     },
489:     "probability_summary": {
490:         "min": 0.003524,
491:         "mean": 0.708454,
492:         "max": 0.999863
493:     },
494:     "model_path": "runs/classical-892cd7c2f6a0/models/logistic_regression.joblib"
495: },
496: {
497:     "id": "naive_bayes",
498:     "label": "Gaussian Naive Bayes",
499:     "training_seconds": 0.004,
500:     "metrics": {
501:         "accuracy": 0.9095,
502:         "precision": 0.989049,

```

```

503:         "recall": 0.897516,
504:         "f1": 0.941062,
505:         "support": 2000,
506:         "tp": 1445,
507:         "tn": 374,
508:         "fp": 16,
509:         "fn": 165
510:     },
511:     "probability_summary": {
512:         "min": 0.0,
513:         "mean": 0.730647,
514:         "max": 1.0
515:     },
516:     "model_path": "runs/classical-892cd7c2f6a0/models/naive_bayes.joblib"
517: },
518: ],
519: "best_model": "adaboost"
520: },
521: {
522:     "run_id": "dnn-29fd1f5f240b",
523:     "job_id": "e4efa927fcd4dc2ba58e41ceb8af5ce",
524:     "kind": "dnn_train",
525:     "created_at": "2026-05-04T16:52:33.307024+00:00",
526:     "config": {
527:         "architectures": [
528:             1
529:         ],
530:         "train_sample": 5000,
531:         "test_sample": 2000,
532:         "epochs": 1,
533:         "batch_size": 128,
534:         "profile": "fast",
535:         "random_state": 42
536:     },
537:     "dataset": {
538:         "train_rows": 5000,
539:         "test_rows": 2000,
540:         "feature_count": 41
541:     },
542:     "results": [
543:         {
544:             "id": "dnn_1_layer",
545:             "label": "DNN 1 Layer",
546:             "training_seconds": 1.273,
547:             "metrics": {
548:                 "accuracy": 0.916,
549:                 "precision": 0.912,
550:                 "recall": 0.991304,
551:                 "f1": 0.95,
552:                 "support": 2000,
553:                 "tp": 1596,
554:                 "tn": 236,
555:                 "fp": 154,
556:                 "fn": 14
557:             },
558:             "probability_summary": {
559:                 "min": 0.210737,
560:                 "mean": 0.801977,
561:                 "max": 0.943477
562:             },
563:             "history": {
564:                 "epochs": 1,
565:                 "best_accuracy": 0.8812,
566:                 "best_loss": 0.382257,
567:                 "final_accuracy": 0.8812,
568:                 "final_loss": 0.382257
569:             },
570:             "model_path": "runs/dnn-29fd1f5f240b/models/dnn1.keras"
571:         }
572:     ],
573:     "best_model": "dnn_1_layer"
574: },
575: {
576:     "run_id": "dnn-9612cd3f9981",
577:     "job_id": "0ef98f5be8294bd4953e2ccc9b61f19c",
578:     "kind": "dnn_train",
579:     "created_at": "2026-05-04T16:50:49.902748+00:00",
580:     "config": {
581:         "architectures": [
582:             1,
583:             3
584:         ],
585:         "train_sample": 50000,
586:         "test_sample": 15000,
587:         "epochs": 3,

```

```

588:     "batch_size": 128,
589:     "profile": "fast",
590:     "random_state": 42
591: },
592: "dataset": {
593:     "train_rows": 50000,
594:     "test_rows": 15000,
595:     "feature_count": 41
596: },
597: "results": [
598:     {
599:         "id": "dnn_3_layer",
600:         "label": "DNN 3 Layer",
601:         "training_seconds": 22.849,
602:         "metrics": {
603:             "accuracy": 0.921467,
604:             "precision": 0.997081,
605:             "recall": 0.905117,
606:             "f1": 0.948876,
607:             "support": 15000,
608:             "tp": 10932,
609:             "tn": 2890,
610:             "fp": 32,
611:             "fn": 1146
612:         },
613:         "probability_summary": {
614:             "min": 0.0,
615:             "mean": 0.724132,
616:             "max": 1.0
617:         },
618:         "history": {
619:             "epochs": 3,
620:             "best_accuracy": 0.99626,
621:             "best_loss": 0.012686,
622:             "final_accuracy": 0.99626,
623:             "final_loss": 0.012686
624:         },
625:         "model_path": "runs/dnn-9612cd3f9981/models/dnn3.keras"
626:     },
627:     {
628:         "id": "dnn_1_layer",
629:         "label": "DNN 1 Layer",
630:         "training_seconds": 5.954,
631:         "metrics": {
632:             "accuracy": 0.906133,
633:             "precision": 0.99062,
634:             "recall": 0.89187,
635:             "f1": 0.938655,
636:             "support": 15000,
637:             "tp": 10772,
638:             "tn": 2820,
639:             "fp": 102,
640:             "fn": 1306
641:         },
642:         "probability_summary": {
643:             "min": 0.000149,
644:             "mean": 0.69896,
645:             "max": 0.999992
646:         },
647:         "history": {
648:             "epochs": 3,
649:             "best_accuracy": 0.98756,
650:             "best_loss": 0.026306,
651:             "final_accuracy": 0.98756,
652:             "final_loss": 0.026306
653:         },
654:         "model_path": "runs/dnn-9612cd3f9981/models/dnn1.keras"
655:     }
656: ],
657: "best_model": "dnn_3_layer"
658: },
659: {
660:     "run_id": "classical-4676bd9735c7",
661:     "job_id": "f8ce165380d2480482574ab667d00482",
662:     "kind": "classical_train",
663:     "created_at": "2026-05-04T16:49:50.219648+00:00",
664:     "config": {
665:         "models": [
666:             "logistic_regression",
667:             "naive_bayes",
668:             "decision_tree",
669:             "adaboost",
670:             "random_forest"
671:         ],
672:         "train_sample": 40000,

```

```

673:     "test_sample": 15000,
674:     "profile": "fast",
675:     "random_state": 42
676: },
677: "dataset": {
678:     "train_rows": 40000,
679:     "test_rows": 15000,
680:     "feature_count": 41
681: },
682: "results": [
683:     {
684:         "id": "random_forest",
685:         "label": "Random Forest",
686:         "training_seconds": 1.339,
687:         "metrics": {
688:             "accuracy": 0.9232,
689:             "precision": 0.998267,
690:             "recall": 0.906193,
691:             "f1": 0.950004,
692:             "support": 15000,
693:             "tp": 10945,
694:             "tn": 2903,
695:             "fp": 19,
696:             "fn": 1133
697:         },
698:         "probability_summary": {
699:             "min": 0.0,
700:             "mean": 0.733314,
701:             "max": 1.0
702:         },
703:         "model_path": "runs/classical-4676bd9735c7/models/random_forest.joblib"
704:     },
705:     {
706:         "id": "decision_tree",
707:         "label": "Decision Tree",
708:         "training_seconds": 0.479,
709:         "metrics": {
710:             "accuracy": 0.9228,
711:             "precision": 0.997903,
712:             "recall": 0.906027,
713:             "f1": 0.949748,
714:             "support": 15000,
715:             "tp": 10943,
716:             "tn": 2899,
717:             "fp": 23,
718:             "fn": 1135
719:         },
720:         "probability_summary": {
721:             "min": 0.0,
722:             "mean": 0.731067,
723:             "max": 1.0
724:         },
725:         "model_path": "runs/classical-4676bd9735c7/models/decision_tree.joblib"
726:     },
727:     {
728:         "id": "adaboost",
729:         "label": "AdaBoost",
730:         "training_seconds": 6.218,
731:         "metrics": {
732:             "accuracy": 0.921733,
733:             "precision": 0.995006,
734:             "recall": 0.907352,
735:             "f1": 0.94916,
736:             "support": 15000,
737:             "tp": 10959,
738:             "tn": 2867,
739:             "fp": 55,
740:             "fn": 1119
741:         },
742:         "probability_summary": {
743:             "min": 0.415657,
744:             "mean": 0.518662,
745:             "max": 0.592831
746:         },
747:         "model_path": "runs/classical-4676bd9735c7/models/adaboost.joblib"
748:     },
749:     {
750:         "id": "naive_bayes",
751:         "label": "Gaussian Naive Bayes",
752:         "training_seconds": 0.023,
753:         "metrics": {
754:             "accuracy": 0.9114,
755:             "precision": 0.989972,
756:             "recall": 0.899073,
757:             "f1": 0.942335,

```

```

758:         "support": 15000,
759:         "tp": 10859,
760:         "tn": 2812,
761:         "fp": 110,
762:         "fn": 1219
763:     },
764:     "probability_summary": {
765:         "min": 0.0,
766:         "mean": 0.732066,
767:         "max": 1.0
768:     },
769:     "model_path": "runs/classical-4676bd9735c7/models/naive_bayes.joblib"
770: },
771: {
772:     "id": "logistic_regression",
773:     "label": "Logistic Regression",
774:     "training_seconds": 0.945,
775:     "metrics": {
776:         "accuracy": 0.902333,
777:         "precision": 0.988134,
778:         "recall": 0.889386,
779:         "f1": 0.936163,
780:         "support": 15000,
781:         "tp": 10742,
782:         "tn": 2793,
783:         "fp": 129,
784:         "fn": 1336
785:     },
786:     "probability_summary": {
787:         "min": 0.000107,
788:         "mean": 0.694586,
789:         "max": 1.0
790:     },
791:     "model_path": "runs/classical-4676bd9735c7/models/logistic_regression.joblib"
792: }
793: ],
794: "best_model": "random_forest"
795: }
796: ]

```

Appendix H. Annotated Bibliography

[NIST-800-94] Guide to Intrusion Detection and Prevention Systems (IDPS)

Karen Scarfone and Peter Mell. 2007. <https://csrc.nist.gov/pubs/sp/800/94/final>

Accessed May 15, 2026. Operational foundation for how IDS and IPS technologies should be understood, designed, monitored, and maintained.

[NIST-800-61R2] Computer Security Incident Handling Guide

Paul Cichonski, Thomas Millar, Tim Grance, Karen Scarfone. 2012. <https://csrc.nist.gov/pubs/sp/800/61/r2/final>

Accessed May 15, 2026; page notes the document was withdrawn on April 3, 2025 and superseded by Rev. 3. Used to frame the product as part of an incident handling workflow rather than as a standalone algorithm.

[NIST-800-83R1] Guide to Malware Incident Prevention and Handling for Desktops and Laptops

Murugiah Souppaya and Karen Scarfone. 2013. <https://csrc.nist.gov/pubs/sp/800/83/r1/final>

Accessed May 15, 2026. Supports the report's discussion of file triage, malware-like indicators, and incident response preparedness.

[NIST-800-137] Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations

Kelley Dempsey et al.. 2011. <https://csrc.nist.gov/pubs/sp/800/137/final>

Accessed May 15, 2026. Provides the conceptual frame for continuous monitoring and why persistent telemetry matters.

[KDD-1999] KDD Cup 1999: Computer Network Intrusion Detection

SIGKDD Cup organizers. 1999. <https://www.kdd.org/kdd-cup/view/kdd-cup-1999/Data>

Accessed May 15, 2026. Primary source for the benchmark dataset bundled with this repository.

[UCI-KDD99] KDD Cup 1999 Data

Salvatore Stolfo, Wei Fan, Wenke Lee, Andreas Prodromidis, Philip Chan. 1999. <https://archive.ics.uci.edu/dataset/130/kdd+cup+1999+data>

Accessed May 15, 2026. Used for formal citation metadata, instance counts, and licensing context.

[CIC-IDS2017] Intrusion Detection Evaluation Dataset (CIC-IDS2017)

Canadian Institute for Cybersecurity, University of New Brunswick. 2017. <https://www.unb.ca/cic/datasets/ids-2017.html>

Accessed May 15, 2026. Authoritative dataset description for modern attacks, five-day collection schedule, and flow feature generation.

[UNSW-NB15] The UNSW-NB15 Dataset

UNSW Canberra at ADFA. 2015-2024 page snapshot. <https://research.unsw.edu.au/projects/unswnb15-dataset>

Accessed May 15, 2026. Source for the dataset composition, attack families, record counts, and split sizes.

[DENNING-1987] An Intrusion-Detection Model

Dorothy E. Denning. 1987. <https://doi.org/10.1109/TSE.1987.232894>

Accessed via DOI and search metadata on May 15, 2026. Seminal conceptual basis for anomaly-oriented intrusion detection built around profiles and deviations.

[LEE-1999] A Data Mining Framework for Building Intrusion Detection Models

Wenke Lee, Salvatore Stolfo, Kui Mok. 1999. <https://doi.org/10.1109/SECPRI.1999.766909>

Accessed via DOI-linked search metadata on May 15, 2026. Seminal bridge from audit data toward data-mining-based intrusion detection workflows.

[LIU-LANG-2019] Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey

Hongyu Liu and Bo Lang. 2019. <https://www.mdpi.com/2076-3417/9/20/4396>

Accessed May 15, 2026. Broad survey used to situate this repository inside the ML and DL IDS landscape.

[CANTONE-2024] On the Cross-Dataset Generalization of Machine Learning for Network Intrusion Detection

Marco Cantone, Claudio Marrocco, Alessandro Bria. 2024. <https://arxiv.org/abs/2402.10974>

Accessed May 15, 2026. Modern evidence that same-dataset accuracy can mask serious generalization failure across network environments.

[MITRE-ATTACK] MITRE ATT&CK;

MITRE. Living knowledge base. <https://attack.mitre.org/index.html>

Accessed May 15, 2026; landing page advertised ATT&CK v19. Used to connect the repository's hunt and IOC features to contemporary detection engineering language.

[PY-ZIPAPP] zipapp - Manage executable Python zip archives

Python Software Foundation. 2026 documentation snapshot. <https://docs.python.org/3.14/library/zipapp.html>

Accessed May 15, 2026. Supports the report's packaging discussion around distributable Python applications.

[PY-TKINTER] tkinter - Python interface to Tcl/Tk

Python Software Foundation. 2026 documentation snapshot. <https://docs.python.org/3/library/tkinter.html>

Accessed May 15, 2026. Used to contextualize the GUI layer and its cross-platform baseline.

[GH-OIDC-PYPI] Configuring OpenID Connect in PyPI

GitHub Docs. 2026. <https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-pypi>

Accessed May 15, 2026. Supports the release automation analysis for trusted package publishing.

[PYPI-TRUSTED-PUBLISHER] Creating a PyPI project with a Trusted Publisher

PyPI Documentation. 2026. <https://docs.pypi.org/trusted-publishers/creating-a-project-through-oidc/>

Accessed May 15, 2026. Used in the packaging chapter to describe zero-token project bootstrapping.